



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería del Software

**Transpilación Cuántica Híbrida mediante
Optimización Multiobjetivo y Aprendizaje por
Refuerzo**

**Hybrid Quantum Transpilation through
Multi-Objective Optimization and Reinforcement
Learning**

Realizado por

Eduardo González Bautista

Tutorizado por

Gabriel Jesús Luque Polo, Abdelmoiz Zakaria Dahi

Departamento

Departamento de Lenguajes y Ciencias de la Computación

Málaga, Junio, 2026

Resumen

La computación cuántica es un paradigma en crecimiento en el que los programas se representan mediante circuitos cuánticos. Sin embargo, para ejecutar estos circuitos en ordenadores cuánticos reales es necesario adaptarlos a las restricciones concretas del *hardware*, como la conectividad entre cúbits, el conjunto de puertas disponibles y la presencia de ruido. Este proceso de adaptación se conoce como transpilación cuántica y condiciona directamente la calidad del circuito ejecutable.

Este Trabajo de Fin de Grado estudia una estrategia híbrida para mejorar el proceso de transpilación mediante la combinación de optimización multiobjetivo, aprendizaje por refuerzo e integración con Qiskit. La propuesta se centra especialmente en dos decisiones relevantes: la selección del *layout* inicial, que asigna cúbits lógicos a cúbits físicos, y el *routing*, que introduce operaciones adicionales cuando las interacciones del circuito no son compatibles con la conectividad del dispositivo.

La solución se diseña de forma modular para separar responsabilidades, facilitar la comparación entre técnicas y permitir futuras extensiones. El módulo `qiskit_interface` proporciona la capa de interacción con Qiskit, el cálculo de métricas y las transpilaciones de referencia. `mo_module` aplica algoritmos evolutivos para explorar *layouts* iniciales atendiendo a varias métricas de calidad. `rl_module` entrena agentes de aprendizaje por refuerzo para tomar decisiones de *routing* o *synthesis*. Finalmente, `integration` coordina estos componentes y permite comparar escenarios como Baseline, MO_Only, RL_Only y MO+RL.

La propuesta permite analizar cada técnica por separado y también su combinación dentro de un flujo experimental reproducible. De este modo, el trabajo estudia si una selección más informada del *layout* y una política aprendida de *routing* pueden mejorar la adaptación de circuitos cuánticos frente a estrategias de referencia.

Palabras clave: transpilación cuántica, optimización multiobjetivo, aprendizaje por refuerzo, Qiskit, *routing*

Abstract

Quantum computing is a growing paradigm in which programs are represented as quantum circuits. However, executing these circuits on real quantum computers requires adapting them to the specific constraints of the hardware, such as qubit connectivity, the available gate set, and the presence of noise. This adaptation process is known as quantum transpilation and directly affects the quality of the executable circuit.

This Bachelor's Thesis studies a hybrid strategy to improve the transpilation process by combining multi-objective optimization, reinforcement learning, and integration with Qiskit. The proposal focuses especially on two relevant decisions: the selection of the initial layout, which assigns logical qubits to physical qubits, and routing, which introduces additional operations when the circuit interactions are not compatible with the device connectivity.

The solution is designed in a modular way to separate responsibilities, facilitate comparison between techniques, and support future extensions. The `qiskit_interface` module provides the interaction layer with Qiskit, metric computation, and reference transpilations. `mo_module` applies evolutionary algorithms to explore initial layouts according to several quality metrics. `rl_module` trains reinforcement learning agents to make routing or synthesis decisions. Finally, `integration` coordinates these components and enables the comparison of scenarios such as `Baseline`, `MO_Only`, `RL_Only`, and `MO+RL`.

The proposal makes it possible to analyze each technique separately and also their combination within a reproducible experimental flow. In this way, the thesis studies whether a more informed layout selection and a learned routing policy can improve the adaptation of quantum circuits compared with reference strategies.

Keywords: quantum transpilation, multi-objective optimization, reinforcement learning, Qiskit, routing

Agradecimientos

En primer lugar, quiero agradecer a mis tutores, Gabriel Jesús y Zakaria. Durante estos meses, vuestra orientación y apoyo han sido fundamentales para el desarrollo de este trabajo. Gracias por ayudarme siempre que lo he necesitado, por explicarme con paciencia conceptos que en algunos momentos sentía que me sobrepasaban, y por estar disponibles prácticamente en cualquier momento. Vuestra implicación y cercanía han hecho que este proceso haya sido mucho más llevadero.

También quiero agradecer profundamente a mis padres, María José y Eduardo, por haberme convertido en la persona que soy hoy. Sin la maravillosa educación que me habéis dado y sin vuestro apoyo constante, no habría podido vivir una experiencia tan buena en un grado tan demandante como este. Habéis estado siempre conmigo, tanto en las celebraciones por los buenos resultados como consolándome cuando las cosas no salían como esperaba. Gracias por confiar en mí, por acompañarme en cada etapa y por ser siempre un apoyo fundamental en mi vida.

También quiero agradecer al resto de mi familia, por el cariño, el apoyo y la confianza que me habéis dado durante todos estos años. En especial, quiero agradecerle a ti, Javi, por haberme transmitido tu vocación y por haber sido un referente en este campo siempre que lo he necesitado. Tu presencia ha sido muy importante para mí, y este logro también tiene una parte de ti.

A mis amigos de la universidad: Juan Manuel, Artur, Jorge, Rubén y David. Vosotros sois, probablemente, lo más bonito que me llevo de estos últimos cuatro años. Gracias por todos los buenos momentos que hemos compartido y por haber puesto siempre el listón tan alto. Habéis hecho que disfrutara cada día intentando dar la mejor versión de mí, y vuestra compañía ha convertido esta etapa en una experiencia mucho más especial.

Por último, quiero agradecer a mi novia, Laura. Me has acompañado durante casi todo este camino, motivándome, empujándome a ser mejor cada día y dándome todo el apoyo que he necesitado en cada momento. Ahora eres tú quien empieza su camino en la universidad, y si consigo darte al menos la mitad del apoyo que tú me has dado a mí, me daré por satisfecho. Jamás te rindas: tienes un talento enorme, y estoy seguro de que te llevará a conseguir cosas grandes.

Resumen	3
Abstract	5
Agradecimientos	7
1 Introducción	19
1.1 Motivación	19
1.2 Objetivos	20
1.3 Alcance y contribuciones del TFG	20
1.4 Estructura de la memoria	21
2 Antecedentes y fundamentos	23
2.1 Computación cuántica y paradigma basado en puertas	23
2.2 Transpilación cuántica: <i>layout</i> , <i>routing</i> y <i>synthesis</i>	27
2.3 Optimización multiobjetivo aplicada a transpilación	29
2.4 Aprendizaje por refuerzo aplicado a transpilación	30
2.5 Estado del arte y trabajos relacionados	32
3 Tecnologías y entorno experimental	35
3.1 Pila de <i>software</i> principal	35
3.2 Qiskit 2.x y <i>backends</i> simulados	36
3.3 Herramientas para optimización multiobjetivo y aprendizaje por refuerzo	38
3.4 Reproducibilidad y limitaciones del entorno	39
4 Arquitectura y diseño de la solución	41
4.1 Visión global del sistema	41
4.2 Contrato compartido de <i>layout</i>	43
4.3 Responsabilidades por módulo	45
4.4 Flujo híbrido y escenarios de evaluación	46
4.5 Decisiones de diseño y alcance actual	48
5 Desarrollo del módulo <code>qiskit_interface</code>	51
5.1 Estructura general del módulo	51
5.1.1 Visión global	51
5.1.2 Bloques funcionales y contratos	52

5.1.3	Diagrama de componentes	53
5.1.4	Flujo principal	54
5.2	Gestión y generación de circuitos	55
5.3	Abstracción de <i>backends</i> y propiedades <i>hardware</i>	57
5.4	Métricas de circuitos y transpilación <i>baseline</i>	59
5.5	Evaluación con <i>layouts</i> externos	60
5.6	Evaluación con <i>routing</i> externo	62
6	Desarrollo del módulo <i>mo_module</i>	65
6.1	Estructura general del módulo	65
6.1.1	Visión global	65
6.1.2	Bloques funcionales y contratos	65
6.1.3	Modelo estático del optimizador	65
6.1.4	Flujo de optimización multiobjetivo	67
6.2	Formulación del problema y representación del <i>layout</i>	67
6.3	Objetivos y evaluación de fitness	70
6.4	Algoritmos evolutivos y operadores especializados	72
6.5	Análisis del frente de Pareto y selección de soluciones	74
6.6	Ajuste de hiperparámetros, bancos de pruebas y reproducibilidad	76
6.7	Limitaciones actuales y aportación del módulo	77
7	Desarrollo del módulo <i>rl_module</i>	81
7.1	Formulación del problema y arquitectura del entorno	81
7.2	Estructura general del módulo	82
7.2.1	Visión global	82
7.2.2	Bloques funcionales y contratos	82
7.2.3	Modelo estático del entorno	82
7.2.4	Ciclo de episodio	84
7.2.5	Secuencia de un paso de <i>routing</i>	85
7.2.6	Flujo principal	85
7.3	Representación interna del estado y del <i>layout</i>	86
7.4	Modo <i>routing</i> como alcance base	87
7.5	Modo de síntesis como extensión del alcance base	88
7.6	Entrenamiento, evaluación e interpretabilidad	89

7.7	Limitaciones actuales y aportación del módulo	91
8	Desarrollo del módulo integration	93
8.1	Papel y alcance de la capa de integración	93
8.2	Estructura general del módulo	94
8.2.1	Dos niveles de orquestación	94
8.2.2	Bloques funcionales	94
8.2.3	Diagrama de componentes	94
8.3	Contratos y DTOs de integración	94
8.4	Escenarios de evaluación	97
8.4.1	Baseline	98
8.4.2	MO_Only	98
8.4.3	RL_Only	99
8.4.4	MO+RL	99
8.5	Reconstrucción y evaluación tras <i>routing</i>	100
8.6	Campañas reproducibles	100
8.7	Comparabilidad actual entre escenarios	102
8.8	Alcance actual y ampliaciones	102
9	Ampliaciones realizadas	103
9.1	Motivación y criterio de ampliación	103
9.2	Lectura de circuitos en QASM	104
9.3	<i>Routing</i> enmascarado con MaskablePPO	104
9.4	Modo <i>synthesis</i>	106
9.5	Modelo de ajuste para optimización multiobjetivo	106
9.6	Estrategias avanzadas de selección híbrida	107
9.6.1	<code>hybrid_probe</code>	107
9.6.2	<code>rl_guided</code>	108
9.7	Subgrafos de <i>routing</i>	108
9.8	Ampliaciones de campañas y trazabilidad	110
10	Validación y diseño experimental	111
10.1	Preguntas de investigación e hipótesis	111
10.2	Circuitos de prueba	112
10.3	Topologías sintéticas seleccionadas	113

10.4	Métricas e indicadores	114
10.5	Protocolo experimental	115
10.6	Amenazas a la validez	117
11	Análisis experimental y discusión	119
11.1	Criterios de lectura de los resultados	119
11.2	Resultados en circuitos GHZ	120
11.2.1	GHZ de 5 cúbits	120
11.2.2	GHZ de 8 cúbits	121
11.2.3	GHZ de 11 cúbits	121
11.2.4	Resumen de resultados en GHZ	122
11.3	Resultados en circuitos QFT	122
11.3.1	QFT de 5 cúbits	123
11.3.2	QFT de 8 cúbits	123
11.3.3	QFT de 11 cúbits	124
11.3.4	Resumen de resultados en QFT	125
11.4	Comparación transversal entre configuraciones	126
11.4.1	Efecto de la familia de circuito	126
11.4.2	Efecto del tamaño	126
11.4.3	Efecto de la topología	126
11.4.4	Efecto de los escenarios de transpilación	126
11.4.5	Efecto de los modos de selección MO	127
11.5	Discusión crítica	127
12	Conclusiones y líneas futuras	129
12.1	Conclusiones	129
12.2	Líneas futuras	131
A	Guía de Despliegue	133
A.1	Requisitos de instalación	133
A.2	Ejecución básica	133
A.3	Uso de la CLI	134

2.1	Circuito de preparación de un estado de Bell generado con la función <code>draw</code> de Qiskit.	25
2.2	Representación de <i>SWAP</i> y de su descomposición habitual en puertas <i>CX</i> , generadas con Qiskit.	26
3.1	Lenguajes y núcleo de ejecución.	36
3.2	Aprendizaje por refuerzo, optimización y seguimiento experimental.	36
3.3	Análisis científico y visualización.	36
3.4	Mapas de acoplamiento de los <i>fake backends</i> principales. Elaboración propia a partir de las coordenadas y acoplamientos incluidos en sus instantáneas de Qiskit.	37
3.5	Familias de topologías sintéticas de acoplamiento consideradas, incluida la variante <i>t</i> balanceada. Elaboración propia a partir de mapas de acoplamiento generados con Qiskit y de la especificación sintética de <i>integration</i>	38
4.1	Visión global de la arquitectura modular del sistema.	42
4.2	Interpretación del contrato compartido de <i>layout</i>	44
4.3	Flujo general de escenarios coordinado por <i>integration</i>	47
5.1	Componentes internos y contratos principales de <i>qiskit_interface</i>	54
5.2	Circuito <i>ghz</i> de cinco cúbits generado por la biblioteca interna de <i>qiskit_interface</i> .	56
5.3	Circuito <i>qft</i> de cinco cúbits generado por la biblioteca interna de <i>qiskit_interface</i> .	56
5.4	Circuito <i>clifford</i> de cinco cúbits generado con semilla 42 por la biblioteca interna de <i>qiskit_interface</i>	57
5.5	Secuencia de evaluación de un <i>layout</i> externo en <i>qiskit_interface</i>	62
6.1	Vista de componentes y fronteras funcionales de <i>mo_module</i>	66
6.2	Modelo UML de clases de los contratos principales de <i>mo_module</i>	67
6.3	Actividad principal de optimización multiobjetivo de <i>layouts</i> en <i>mo_module</i>	68
6.4	Flujo de validación y reparación de un <i>layout</i> candidato en <i>mo_module</i>	70
6.5	Captura de la interfaz gráfica de banco de pruebas y ajuste de hiperparámetros de <i>mo_module</i>	78
7.1	Modelo UML de clases de la arquitectura principal de <i>rl_module</i>	83

7.2	Diagrama UML de estados del ciclo de episodio en <code>QuantumTranspilationEnv</code> . .	84
7.3	Secuencia de interacción entre agente, entorno, frontera y recompensa en un paso de <i>routing</i>	85
7.4	Flujo de entrenamiento, persistencia y evaluación de modelos en <code>rl_module</code>	90
7.5	Captura de la interfaz gráfica de <code>rl_module</code>	91
8.1	Diagrama UML de componentes principales de <i>integration</i>	96
8.2	Actividad comparativa de los escenarios <i>Baseline</i> , <i>MO_Only</i> , <i>RL_Only</i> y <i>MO+RL</i> . .	97
8.3	Secuencia del traspaso <i>MO+RL</i> coordinado por <i>integration</i>	101
8.4	Actividad principal de una campaña reproducible en <i>integration</i>	101
9.1	Derivación de un subgrafo de <i>routing</i> expandido por caminos dentro de una campaña híbrida.	109
10.1	Topologías sintéticas <i>ring</i> y <i>t</i> utilizadas en el protocolo experimental para 5, 8 y 11 cúbits.	113
10.2	Composición de la matriz experimental principal.	115
10.3	Desglose de las ocho ejecuciones evaluadas para cada combinación experimental. . .	115

4.1	Responsabilidades y límites principales de cada módulo.	45
4.2	Resumen de los escenarios de evaluación definidos por <i>integration</i>	48
5.1	Contratos centrales expuestos por <i>qiskit_interface</i>	52
5.2	Circuitos disponibles en la biblioteca interna de <i>qiskit_interface</i>	55
5.3	Interpretación de las métricas básicas extraídas por <i>extract_metrics</i>	60
5.4	Comprobaciones básicas aplicadas a un <i>layout</i> externo antes de evaluarlo.	61
5.5	Entradas relevantes de la evaluación con <i>routing</i> externo.	62
6.1	Bloques funcionales de <i>mo_module</i> y contratos que fijan su superficie pública.	66
6.2	Reglas básicas de factibilidad aplicadas a un <i>layout</i> candidato.	69
6.3	Operadores evolutivos especializados empleados por <i>mo_module</i>	73
6.4	Instrumentos de análisis y selección disponibles para el frente de Pareto de <i>mo_module</i>	75
7.1	Bloques funcionales de <i>rl_module</i> y relación con el alcance base de <i>routing</i>	83
7.2	Campos principales de la observación en el modo <i>routing</i>	87
7.3	Diferencias entre el alcance base de <i>routing</i> y la extensión <i>synthesis</i>	89
8.1	Bloques funcionales principales de <i>integration</i>	95
8.2	DTOs públicos utilizados por la capa <i>integration</i>	95
8.3	Comparación funcional de los cuatro escenarios públicos.	98
8.4	Modos históricos de selección de <i>layout</i> disponibles para <i>MO_Only</i>	99
8.5	Artefactos persistidos por una campaña base.	102
9.1	Versiones de <i>routing</i> enmascarado	105
10.1	Factores principales del protocolo experimental.	116
11.1	Resultados agregados para <i>ghz</i> de 5 cúbits, separados por topología y modo de selección MO.	120
11.2	Resultados agregados para <i>ghz</i> de 8 cúbits, separados por topología y modo de selección MO.	121

11.3	Resultados agregados para ghz de 11 cúbits, separados por topología y modo de selección MO.	122
11.4	Resultados agregados para qft de 5 cúbits, separados por topología y modo de selección MO.	123
11.5	Resultados agregados para qft de 8 cúbits, separados por topología y modo de selección MO.	124
11.6	Resultados agregados para qft de 11 cúbits, separados por topología y modo de selección MO.	125

Nomenclatura

$layout[i]$ Cúbit físico asignado al cúbit lógico i

Baseline Escenario de referencia basado en Qiskit

MO+RL Escenario híbrido que combina layout multiobjetivo y routing con aprendizaje por refuerzo

MO_Only Escenario que evalúa el layout seleccionado por optimización multiobjetivo

RL_Only Escenario que evalúa el routing aprendido sin layout procedente de MO

CNOT Controlled-NOT, puerta cuántica controlada de dos cúbits

DAG Grafo acíclico dirigido

DQN Deep Q-Network

GHZ Greenberger-Horne-Zeilinger

HV Hipervolumen del frente de Pareto

MaskablePPO Variante de PPO con máscaras de acciones válidas

MO Optimización multiobjetivo

MOEA/D Multi-Objective Evolutionary Algorithm based on Decomposition

NISQ Noisy Intermediate-Scale Quantum

NSGA-II Non-dominated Sorting Genetic Algorithm II

PPO Proximal Policy Optimization

QASM Quantum Assembly Language

QFT Transformada Cuántica de Fourier

RL Aprendizaje por refuerzo

SABRE Heurística de routing basada en búsqueda bidireccional con puertas SWAP

SB3 Stable-Baselines3

SWAP Puerta que intercambia el estado de dos cúbits

1

Introducción

1.1 Motivación

La computación cuántica propone un paradigma de cálculo distinto al clásico. Mientras que la computación convencional trabaja con bits que toman valores discretos, la computación cuántica utiliza cúbits, sistemas capaces de representar estados cuánticos y de combinarse mediante operaciones específicas. En la práctica, los programas cuánticos se describen como circuitos: una secuencia organizada de puertas que actúan sobre cúbits lógicos para expresar el cálculo que se desea realizar.

Sin embargo, un circuito diseñado de forma abstracta no se ejecuta sin más en cualquier ordenador cuántico real. Antes debe adaptarse a las restricciones concretas del dispositivo disponible. Este proceso de adaptación se conoce como transpilación cuántica y tiene en cuenta, entre otros factores, qué cúbits físicos existen en la máquina, qué conexiones hay entre ellos, qué puertas nativas admite el *hardware* y qué limitaciones introduce el ruido. En los dispositivos actuales, frecuentemente descritos como NISQ, estas restricciones condicionan de forma directa la calidad del circuito finalmente ejecutable.

Dos decisiones son especialmente importantes dentro de este proceso. La primera es el *layout* inicial. Un circuito trabaja con cúbits lógicos, pero el ordenador cuántico dispone de cúbits físicos concretos; por tanto, hay que decidir qué cúbit físico representará a cada cúbit lógico. La segunda es el *routing*. Aunque se haya elegido una asignación inicial, no todos los cúbits físicos están conectados entre sí. Cuando dos cúbits que deben interactuar no son vecinos en la máquina, el transpilador debe introducir operaciones adicionales para mover la información cuántica hasta una posición compatible con la conectividad del dispositivo.

Estas decisiones pueden aumentar la profundidad del circuito, el número de puertas de dos cúbits y, en consecuencia, la exposición al error. Por ello, resulta relevante estudiar estrategias que ayuden a

seleccionar mejores *layouts* iniciales y a tomar decisiones de *routing* de forma más informada. Este TFG se sitúa en ese punto: analiza si el proceso de transpilación puede mejorarse combinando técnicas de optimización multiobjetivo y aprendizaje por refuerzo, manteniendo como referencia comparativa los métodos disponibles en Qiskit.

1.2 Objetivos

El objetivo general de este TFG es estudiar si el proceso de transpilación cuántica puede mejorarse combinando técnicas de optimización multiobjetivo y aprendizaje por refuerzo. En concreto, el trabajo se centra en dos decisiones relevantes del proceso: la selección del *layout* inicial y la introducción de operaciones de *routing* para adaptar el circuito a la conectividad del *hardware*.

Para alcanzar ese objetivo general, se plantean los siguientes objetivos específicos:

- Construir una base de comparación sobre Qiskit para disponer de resultados de referencia frente a los que evaluar la propuesta.
- Desarrollar `qiskit_interface` para aislar la interacción con Qiskit, los *backends*, las métricas y las transpilaciones de referencia.
- Desarrollar `mo_module` para explorar *layouts* iniciales optimizando varios criterios simultáneamente, de forma que la elección no dependa de una única métrica.
- Desarrollar `rl_module` para estudiar decisiones de *routing* mediante políticas aprendidas que actúan paso a paso sobre el estado del circuito y la conectividad disponible.
- Definir `integration` para orquestar los escenarios comparativos y conectar de forma homogénea los resultados producidos por los módulos anteriores.
- Establecer un flujo reproducible de evaluación para comparar los escenarios del proyecto bajo criterios comunes.

1.3 Alcance y contribuciones del TFG

El TFG se articula en cuatro módulos desacoplados: `qiskit_interface`, `mo_module`, `rl_module` e `integration`. Cada uno fija una responsabilidad principal: entrada y métricas de Qiskit, búsqueda de *layouts*, decisiones de *routing* y orquestación experimental.

La primera contribución del trabajo es, por tanto, el establecimiento de un contrato común entre módulos para intercambiar *layouts* iniciales y resultados de evaluación. Ese contrato permite conectar

el productor de *layouts* con el consumidor de *routing* sin acoplar la implementación interna de ambos, lo que facilita comparar estrategias distintas sobre una misma base experimental.

La segunda contribución es la construcción de un flujo de evaluación reproducible sobre escenarios comparables. A través de *integration*, el proyecto contrasta *Baseline* con variantes que incorporan optimización multiobjetivo y aprendizaje por refuerzo, manteniendo configuraciones y artefactos que permiten auditar cada ejecución.

Como contribución adicional, el trabajo reúne herramientas orientadas a la interpretación de resultados, al seguimiento de experimentos y a la trazabilidad de los cambios introducidos por cada módulo. Esto resulta especialmente útil en un dominio como la transpilación cuántica, donde pequeñas variaciones en el *layout* o en el *routing* pueden tener un impacto significativo en las métricas finales.

En cuanto al alcance, la versión actual del sistema se centra en escenarios de *routing* y en la comparación entre configuraciones de transpilación. El objetivo de la memoria es describir esa arquitectura y mostrar cómo los módulos cooperan para explorar una estrategia híbrida de mejora, sin convertir el proyecto en una reimplementación completa del compilador de Qiskit.

1.4 Estructura de la memoria

La memoria se organiza de forma progresiva, desde los fundamentos teóricos hasta la validación experimental. Tras esta introducción, el Capítulo 2 presenta los antecedentes de computación cuántica, transpilación, optimización multiobjetivo y aprendizaje por refuerzo, además de situar el trabajo frente a investigaciones relacionadas. El Capítulo 3 describe las tecnologías empleadas, con especial atención al ecosistema de Qiskit y al entorno *software* y *hardware* en el que se desarrolla el proyecto.

A continuación, el Capítulo 4 expone la arquitectura y el diseño de la solución, definiendo la visión global del sistema, el contrato compartido de *layout* y las responsabilidades de cada módulo. Sobre esa base, los Capítulos 5, 6, 7 y 8 desarrollan, respectivamente, *qiskit_interface*, *mo_module*, *rl_module* e *integration*, detallando sus componentes internos, sus entradas y salidas, y la forma en que cooperan para dar soporte a los escenarios *Baseline*, *MO_Only*, *RL_Only* y *MO+RL*.

El tramo final recoge las ampliaciones realizadas, la validación, la discusión de resultados y las conclusiones. El Capítulo 9 describe las ampliaciones incorporadas durante el desarrollo; el Capítulo 10 define la validación y el diseño experimental; el Capítulo 11 analiza los resultados obtenidos; y el Capítulo 12 sintetiza las conclusiones y las líneas futuras. Por último, un único apéndice recopila información complementaria sobre el despliegue y la reproducción operativa del proyecto.

2

Antecedentes y fundamentos

2.1 Computación cuántica y paradigma basado en puertas

En computación clásica, la información se representa mediante bits, que solo pueden tomar los valores 0 o 1. En computación cuántica, la unidad básica de información es el cúbit. Un cúbit también se expresa a partir de dos estados base, $|0\rangle$ y $|1\rangle$, pero su estado no tiene por qué coincidir con uno solo de ellos antes de la medición. Puede describirse como una combinación de ambos estados, propiedad conocida como superposición.

De forma matemática, el estado de un cúbit puede escribirse como

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1.$$

En esta expresión, $|\psi\rangle$ denota el estado del cúbit, y α y β son amplitudes complejas. La condición de normalización, $|\alpha|^2 + |\beta|^2 = 1$, garantiza que las probabilidades de todos los resultados posibles sumen uno. Cuando se mide el cúbit en la base computacional, el resultado observado es 0 con probabilidad $|\alpha|^2$ y 1 con probabilidad $|\beta|^2$ [13].

Esta descripción se generaliza a registros de varios cúbits. En un registro de n cúbits, el estado puede expresarse como

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \quad \sum_x |\alpha_x|^2 = 1,$$

donde x recorre todas las cadenas binarias de longitud n , $|x\rangle$ representa el estado base asociado a una de esas cadenas y α_x es su amplitud compleja. El crecimiento exponencial aparece porque existen 2^n estados base posibles, y el estado general necesita una amplitud asociada a cada uno de ellos. La

condición $\sum_x |\alpha_x|^2 = 1$, por su parte, solo expresa que las probabilidades de medida sobre todos esos estados base suman uno.

En registros de varios cúbits aparecen fenómenos que no se reducen a la descripción independiente de cada cúbit. El entrelazamiento describe una correlación cuántica entre varios cúbits que no puede explicarse como la combinación independiente de estados individuales. Este concepto ayuda a entender por qué un circuito cuántico no debe interpretarse como una simple colección de operaciones booleanas.

Existen distintos modelos de computación cuántica, entre ellos el modelo basado en puertas, la computación cuántica adiabática o modelos basados en medidas. En esta memoria se considera el modelo basado en puertas, por ser el utilizado por Qiskit y el más directamente relacionado con el proceso de transpilación estudiado. En este paradigma, un programa cuántico se expresa como un circuito ordenado de operaciones elementales sobre cúbits. La documentación oficial de IBM Quantum describe este modelo como una secuencia de instrucciones que actúan sobre datos cuánticos y que pueden incluir puertas, mediciones y reinicios, todo ello materializado en Qiskit mediante la clase `QuantumCircuit` [14, 7].

Desde el punto de vista formal, cada puerta es una transformación unitaria, es decir, reversible, y puede actuar sobre uno o varios cúbits. Las puertas de un solo cúbit, como X, H, Z, S, T o las rotaciones parametrizadas RX, RY y RZ, modifican un cúbit individual y permiten construir transformaciones locales. Por su parte, las puertas de dos cúbits, como CX, CZ o ECR, actúan como bloques de interacción entre líneas cuánticas y son las que, en la práctica, hacen posible generar entrelazamiento. Esa distinción no es menor: en *hardware* real, las operaciones de dos cúbits suelen ser notablemente más costosas y más sensibles al ruido que las operaciones locales.

La lectura intuitiva de estas puertas ayuda a entender el comportamiento del circuito. La puerta X intercambia los estados $|0\rangle$ y $|1\rangle$, mientras que la puerta H transforma un estado base en una superposición equiprobable. Cuando estas puertas se combinan con CX, el circuito deja de describir solamente cambios locales independientes y puede codificar correlaciones entre cúbits. El modelo de puertas especifica las transformaciones aplicadas, su orden y las dependencias entre líneas cuánticas, aspectos esenciales para la compilación posterior.

Un ejemplo clásico es la preparación de un estado de Bell. Partiendo de $|00\rangle$, se aplica una puerta de Hadamard sobre el primer cúbit y, a continuación, una puerta CX con ese cúbit como control y el segundo como objetivo. El resultado es el estado

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}},$$

En esta notación, $|\Phi^+\rangle$ nombra uno de los cuatro estados de Bell, $|00\rangle$ y $|11\rangle$ son estados de dos cúbits en la base computacional, y el factor $1/\sqrt{2}$ normaliza la superposición para que la suma de probabilidades sea uno. El resultado constituye un par máximamente entrelazado [10]. En un circuito, esta preparación se expresa de forma compacta como

$$|00\rangle \xrightarrow{H \otimes I} \frac{|00\rangle + |10\rangle}{\sqrt{2}} \xrightarrow{CX} \frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

Aquí H es la puerta de Hadamard aplicada al primer cúbit, I es la identidad aplicada al segundo, $H \otimes I$ representa la acción conjunta sobre el registro de dos cúbits y CX es la puerta controlada que usa el primer cúbit como control y el segundo como objetivo. La Figura 2.1 muestra el circuito correspondiente a la preparación de un estado de Bell.

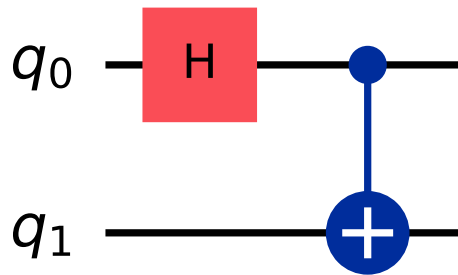


Figura 2.1. Circuito de preparación de un estado de Bell generado con la función *draw* de Qiskit.

La interpretación física del ejemplo es clara: al medir el sistema, solo aparecen los resultados 00 o 11, cada uno con probabilidad $1/2$, pero no una combinación mixta como 01 o 10. Esta correlación perfecta entre ambos cúbits no puede explicarse como una mera combinación de probabilidades clásicas independientes y ejemplifica por qué el entrelazamiento es un recurso central en computación cuántica.

Otra operación muy ilustrativa es **SWAP**, porque muestra con claridad la diferencia entre el modelo lógico y la ejecución física. A nivel abstracto, **SWAP** intercambia el contenido de dos cúbits,

$$\text{SWAP } |a, b\rangle = |b, a\rangle,$$

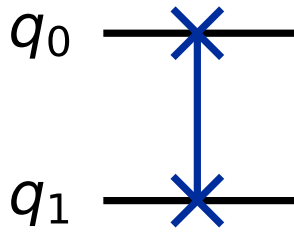
donde a y b representan los valores lógicos almacenados en los dos cúbits antes del intercambio. Qiskit la expone como una instrucción de dos cúbits [15]. Sin embargo, cuando el *backend*, entendido como el

dispositivo o simulador objetivo sobre el que se prepara la ejecución, no la incluye como puerta nativa, el transpilador la descompone habitualmente en tres CX consecutivas,

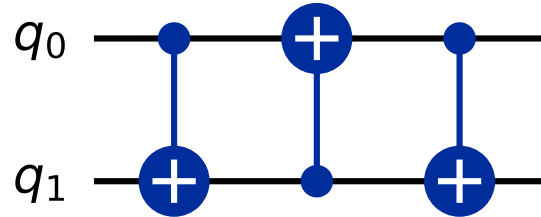
$$CX(q_0, q_1) CX(q_1, q_0) CX(q_0, q_1),$$

donde q_0 y q_1 son los dos cúbits implicados y, en cada $CX(q_i, q_j)$, el primer argumento actúa como control y el segundo como objetivo. Esta descomposición convierte la **SWAP** en una operación relativamente cara sobre dispositivos ruidosos [17]. Este detalle es importante para la memoria porque el enrutado de circuitos consiste, en gran medida, en introducir intercambios lógicos equivalentes a **SWAP** hasta conseguir que las interacciones entre cúbits sean compatibles con la conectividad física del *backend*.

La Figura 2.2 muestra la operación **SWAP** abstracta y una descomposición equivalente en tres puertas CX.



(a) *SWAP* abstracta.



(b) Descomposición equivalente en tres CX.

Figura 2.2. Representación de **SWAP** y de su descomposición habitual en puertas CX, generadas con Qiskit.

Otra consecuencia práctica del modelo de circuitos es que permite definir métricas estructurales útiles para comparar compilaciones. La profundidad del circuito cuantifica cuántas capas secuenciales son necesarias cuando se explotan todos los paralelismos posibles, mientras que la anchura o *width* refleja el número de cúbits implicados en la representación. En el contexto de *hardware* actual, estas magnitudes no son meramente descriptivas: una mayor profundidad suele traducirse en una exposición más larga a decoherencia, es decir, a la pérdida progresiva de las propiedades cuánticas del sistema debido a su interacción con el entorno. Además, un mayor número de puertas de dos cúbits suele incrementar la probabilidad de fallo acumulado. De ahí que, en problemas de transpilación, reducir

profundidad y contener el coste de las interacciones no locales sea una estrategia tan relevante como preservar la corrección lógica del circuito.

Este punto se vuelve especialmente importante en la era *Noisy Intermediate-Scale Quantum* (NISQ), término popularizado por Preskill para describir dispositivos intermedios con decenas de cúbits y sin corrección de errores a gran escala [25]. En ese escenario, la limitación principal no es solo el tamaño abstracto del circuito, sino su capacidad de ejecutarse con fidelidad suficiente sobre un *backend* concreto. La topología física, los errores de puerta, los tiempos de coherencia y las restricciones de conectividad convierten la ejecución de un circuito lógico en un problema de adaptación al *hardware*. Por eso, la transpilación actúa como una etapa de optimización que condiciona de forma directa la calidad del resultado final.

El paradigma basado en puertas ofrece el lenguaje formal con el que se describe el algoritmo, pero no agota el problema práctico de llevarlo a un dispositivo físico. La memoria parte de esa distancia entre descripción lógica y ejecución real: primero se representa el cálculo como circuito, después se estudia cómo adaptarlo a una topología limitada y, finalmente, se comparan estrategias para reducir el coste de esa adaptación.

2.2 Transpilación cuántica: *layout, routing y synthesis*

La transpilación convierte un circuito cuántico abstracto en una versión compatible con la arquitectura concreta de un *backend*, es decir, el dispositivo o simulador objetivo sobre el que se prepara la ejecución. IBM Quantum la define como el proceso de reescritura de un circuito para adaptarlo a la topología del dispositivo y optimizar su ejecución en *hardware* ruidoso. En Qiskit, este proceso se organiza como un flujo por etapas, normalmente materializado mediante un `StagedPassManager`, en el que destacan las fases `init`, `layout`, `routing`, `translation` y `optimization` [11, 17]. Dentro de ese flujo, tres decisiones son especialmente relevantes: situar los cúbits lógicos sobre cúbits físicos, insertar las permutaciones necesarias cuando las interacciones no son ejecutables directamente y traducir finalmente las puertas a las primitivas nativas del *backend*. La ISA, del inglés *Instruction Set Architecture*, recoge las instrucciones, puertas y restricciones que definen qué operaciones puede ejecutar un *backend* concreto. El `coupling_map`, por su parte, representa la conectividad física del dispositivo, es decir, qué pares de cúbits físicos pueden ejecutar directamente operaciones de dos cúbits.

La primera decisión es el *layout*. Esta etapa establece una correspondencia uno a uno entre cúbits virtuales o lógicos y cúbits físicos, y Qiskit la representa mediante un objeto `Layout` que forma parte de la ISA del *backend* [17]. Un buen *layout* sitúa cerca, dentro de la topología física, aquellos cúbits

lógicos que necesitan interactuar en el circuito. De este modo, se reduce la necesidad de introducir operaciones **SWAP** para desplazar la información cuántica hasta posiciones físicamente conectadas. Por eso, los pasos preconfigurados del transpilador de Qiskit intentan primero hallar un *layout* que evite permutaciones innecesarias y, si eso no es posible, recurren a heurísticas; en Qiskit, estos pasos reciben el nombre de *passes*. Entre las estrategias habituales están **TrivialLayout**, **VF2Layout**, **DenseLayout** y **SabreLayout**, cada una con un criterio diferente de selección y distinta relación entre coste y calidad [17]. Esta diversidad muestra que el *layout* no es un paso meramente administrativo, sino una primera fase de optimización que condiciona el resto de la transpilación.

La segunda decisión es el *routing*. Una vez fijado el *layout*, esta etapa inserta operaciones **SWAP** cuando una interacción de dos cúbits no puede ejecutarse de forma directa sobre una arista física del **coupling_map**. Cada **SWAP** es costosa y ruidosa, y el problema de minimizar su número es difícil de resolver exactamente; por eso Qiskit recurre a heurísticas estocásticas como **SabreSwap** para encontrar un mapeo razonable, aunque no necesariamente óptimo [17]. La consecuencia práctica es que el mismo circuito puede generar salidas distintas entre ejecuciones, con distribuciones diferentes de profundidad y número de puertas. En términos de transpilación, el *routing* es la fase que adapta el circuito lógico a la conectividad física disponible, sin alterar la lógica computacional que el circuito implementa.

La tercera pieza es la *synthesis*, entendida aquí como la reducción del circuito a las primitivas nativas del *backend*. Una vez que la conectividad física ya está resuelta, el transpilador reescribe las puertas para que pertenezcan al conjunto de instrucciones soportado por el *target*; en Qiskit, esta idea aparece en la etapa **translation**, y en niveles más agresivos también en operaciones como **UnitarySynthesis** dentro de la fase de optimización [17]. Este paso suele incrementar la profundidad y el número de puertas, pero es imprescindible para ejecutar el circuito sobre *hardware* real. El ejemplo de **SWAP** vuelve a ser ilustrativo: si no es nativa del *backend*, debe descomponerse en tres **CX**, de modo que una operación aparentemente elemental se convierte en una secuencia más cara y más sensible al ruido. En general, la síntesis es la etapa que garantiza que cada instrucción del circuito final pertenezca al repertorio soportado por el dispositivo.

Esta descomposición en *layout*, *routing* y *synthesis* es útil porque separa tres fuentes de complejidad distintas. El *layout* decide dónde se sitúan inicialmente los cúbits lógicos, el *routing* adapta las interacciones a la conectividad disponible y la *synthesis* garantiza que el resultado final use solo instrucciones nativas del dispositivo. A partir de esta división, puede entenderse la transpilación como un problema de adaptación progresiva: primero se asignan los recursos físicos, después se resuelven

las restricciones geométricas y finalmente se baja el circuito al lenguaje nativo del *hardware*.

2.3 Optimización multiobjetivo aplicada a transpilación

En muchos problemas de optimización no existe un único criterio de calidad. En transpilación cuántica, por ejemplo, puede interesar reducir la profundidad del circuito y, al mismo tiempo, minimizar el número de puertas de dos cúbits. Estos objetivos pueden entrar en conflicto: una asignación inicial que favorece interacciones cortas puede no ser la que produce menor profundidad global, y una reducción local de puertas puede desplazar coste a otras partes del circuito. Por ello, resulta natural formular el problema como una optimización multiobjetivo.

En esta clase de problemas no suele existir una solución que sea simultáneamente óptima para todos los criterios. La formulación habitual considera una función vectorial $f(x) = (f_1(x), \dots, f_m(x))$, donde x es una solución candidata, m es el número de objetivos y cada $f_i(x)$ mide el valor de la solución en el objetivo i . Esta función se quiere minimizar sobre un espacio de búsqueda dado, y la dominancia de Pareto se define mediante la relación

$$x \prec y \iff (\forall i, f_i(x) \leq f_i(y)) \wedge (\exists j, f_j(x) < f_j(y)).$$

En esta relación, $x \prec y$ significa que x domina a y ; el índice i recorre todos los objetivos y el índice j identifica al menos un objetivo en el que x mejora estrictamente a y . Las soluciones para las que no existe otra que las domine forman el frente de Pareto. Ese frente no representa un único punto “mejor”, sino un conjunto de alternativas no dominadas que expresan distintos compromisos entre objetivos conflictivos. En transpilación cuántica esta idea es especialmente natural: reducir la profundidad del circuito suele exigir aceptar más operaciones de dos cúbits o una estructura de interacción menos directa, mientras que minimizar el coste de dos cúbits puede forzar recorridos más largos y, por tanto, circuitos más profundos. El problema consiste entonces en localizar un equilibrio razonable entre métricas incompatibles, más que en encontrar un “óptimo absoluto”.

Para explorar ese espacio de compromisos se emplean con frecuencia algoritmos evolutivos de ordenación no dominada. NSGA-II es el referente clásico porque separa la población por frentes y preserva diversidad mediante *crowding distance*, de manera que la búsqueda no se concentra en una sola región del espacio objetivo [4]. La biblioteca *pymoo* integra esta familia de métodos y ofrece una interfaz orientada a experimentación, visualización y análisis de decisiones [3]. La ventaja práctica de este enfoque es que, con una sola ejecución, se obtiene una aproximación del frente de Pareto en lugar de un único punto dependiente de una ponderación fijada de antemano.

Una vez aproximado el frente, la decisión final sobre una solución concreta se traslada a un problema de *multi-criteria decision making* (MCDM). En esa fase suele ser necesario normalizar los objetivos, porque cada métrica puede vivir en una escala diferente y, sin esa precaución, el criterio de mayor magnitud dominaría la comparación. Sobre los valores normalizados se aplica después una función de descomposición o un esquema de compromiso para identificar el punto que mejor refleja las preferencias del decisor; la literatura clásica de MCDM formaliza este paso como la selección de una solución de compromiso paretiana dentro del conjunto de alternativas no dominadas [5]. De forma esquemática, puede escribirse como

$$x^* = \arg \min_{x \in \mathcal{P}} g(\hat{f}_1(x), \dots, \hat{f}_m(x); w),$$

donde x^* es la solución finalmente seleccionada, $\arg \min$ indica que se elige el candidato con menor valor de compromiso, x recorre el frente de Pareto aproximado $\mathcal{P}$, $\hat{f}_i$ son los objetivos normalizados, w recoge las preferencias relativas y g es la función de compromiso o descomposición elegida. Esta separación entre generación del conjunto de soluciones y selección final es conceptualmente importante: la optimización produce diversidad, mientras que la decisión elige una solución representativa de esa diversidad.

El módulo multiobjetivo adopta esa lógica. Cada individuo de la búsqueda representa un *layout* inicial, es decir, una asignación candidata de cúbits lógicos a cúbits físicos. Sobre cada candidato se evalúan dos objetivos activos: `depth` y `cnot_count`. El primero cuantifica la profundidad del circuito transpilado, mientras que el segundo no debe interpretarse como un simple conteo literal de puertas `cx` visibles en el circuito original, sino como un coste equivalente en `cx` derivado de la transpilación final. En la práctica, esto convierte a `cnot_count` en un *proxy* del esfuerzo de dos cúbits del circuito. Así se penalizan descomposiciones como `SWAP` y otras transformaciones que, aunque lógicamente equivalentes, incrementan el coste físico de ejecución. Bajo esta formulación, el resultado del módulo no es un único *layout* “óptimo” en sentido absoluto, sino un conjunto de *layouts* no dominados entre los que luego puede elegirse una solución de compromiso según el equilibrio deseado entre profundidad y coste de dos cúbits.

2.4 Aprendizaje por refuerzo aplicado a transpilación

El aprendizaje por refuerzo (RL) modela la interacción entre un agente y un entorno como un proceso secuencial de decisión. Formalmente, suele expresarse mediante un proceso de decisión de Markov $\langle S, A, P, R, \gamma \rangle$, donde S es el conjunto de estados, A el conjunto de acciones, P la dinámica de

transición, R la función de recompensa y γ el factor de descuento. En cada instante t , el agente se encuentra en un estado s_t , selecciona una acción a_t según una política $\pi(a_t | s_t)$, recibe una recompensa r_{t+1} y transita a un nuevo estado o a una observación [2]. Aquí $\pi(a_t | s_t)$ representa la probabilidad de elegir la acción a_t condicionada al estado s_t , y r_{t+1} es la recompensa obtenida tras ejecutar esa acción. El objetivo no es maximizar una recompensa instantánea, sino la suma descontada de recompensas futuras a lo largo del episodio. Cuando el agente no dispone del estado completo, sino solo de una observación codificada del problema, se habla de un entorno parcialmente observable; esta distinción es importante porque muchos problemas reales no exponen toda la información relevante de forma directa [2]. En RL, además, el diseño de la recompensa es delicado: una señal demasiado escasa dificulta el aprendizaje, mientras que una señal demasiado local puede inducir comportamientos cortoplacistas que no mejoran la solución global.

La transpilación cuántica encaja bien en ese marco porque puede formularse como una secuencia de decisiones discretas bajo restricciones fuertes. En cada paso, una acción modifica una parte del circuito, la conectividad o la forma en la que el problema se aproxima al *hardware* objetivo; por tanto, el interés no está en una decisión aislada, sino en la política que transforma una entrada inicial en una secuencia válida y eficiente. Esta manera de plantear el problema ya ha aparecido en la literatura de optimización combinatoria, donde las políticas aprendidas sustituyen o complementan heurísticas diseñadas a mano [1]. En el ámbito cuántico, la idea también se ha trasladado a entornos específicos de síntesis y transpilación, con resultados que muestran que RL puede adaptarse tanto a la construcción de circuitos como al enrutado sobre conectividad limitada [18, 19].

En esta memoria, esa abstracción se concreta en dos modos de uso. En **routing**, el agente opera sobre un estado asociado al circuito parcial y a la conectividad disponible, y su acción consiste en decidir qué **SWAP** introducir para hacer posibles las interacciones pendientes sobre el **coupling_map**. La recompensa debe favorecer que el circuito avance hacia una configuración físicamente ejecutable, penalizando al mismo tiempo las permutaciones innecesarias y el aumento del coste de dos cúbits. En **synthesis**, el problema cambia: el agente recibe una representación del objetivo y decide qué primitivas nativas aplicar para aproximarse a él bajo un conjunto de puertas permitido. Aquí la recompensa premia el progreso hacia el circuito deseado y penaliza secuencias largas, retrocesos o acciones que no aportan avance estructural. En ambos casos, la calidad de la política se evalúa a lo largo del episodio completo, no por el valor de una acción individual.

Este punto de vista explica también por qué el comportamiento del entorno debe definirse con cuidado. Si la señal de recompensa es demasiado escasa, el agente puede tardar en recibir informa-

ción útil; si es demasiado detallada, puede aprender atajos que mejoran la métrica local pero empeoran el resultado final. Por ello, la formulación de RL en transpilación no sustituye a la compilación clásica, sino que introduce una política aprendida para resolver decisiones locales dentro de un problema combinatorio sometido a restricciones de *hardware*. En términos prácticos, se busca una secuencia de decisiones que reduzca el coste total del circuito, preserve su equivalencia lógica y se adapte a las limitaciones del dispositivo.

2.5 Estado del arte y trabajos relacionados

El punto de referencia más extendido en compilación NISQ para *layout* y *routing* es SABRE. El método de Li, Ding y Xie formula el mapeo como una búsqueda heurística bidireccional que explora rutas hacia delante y hacia atrás, reutiliza la permutación inducida por los **SWAPs** para refinar el *layout* inicial e introduce un efecto de decaimiento para equilibrar profundidad y número de puertas de dos cúbits [21]. La documentación de IBM Quantum incorpora esa familia de heurísticas en las etapas **SabreLayout** y **SabreSwap**, lo que confirma que se trata de una referencia práctica y no solo teórica [17]. Sin embargo, su salida sigue siendo esencialmente una decisión puntual: para una entrada concreta produce un *layout* y un enrutado concretos, pero no ofrece una exploración explícita de varios compromisos posibles entre objetivos.

A partir de esa base, otros trabajos han tratado de enriquecer la selección del *layout* con información del *hardware*. Niu, Suau, Staffelbach y Todri-Sanial proponen un heurístico *hardware-aware* que incorpora datos de calibración para mejorar la fidelidad global del circuito y reducir el coste introducido por el mapeo [22]. Esta línea es relevante porque muestra que el *layout* no debe entenderse solo como una asignación topológica, sino también como una decisión sensible a la calidad relativa de los cúbits y de las puertas disponibles. Aun así, el resultado sigue siendo, por lo general, una única solución seleccionada por una heurística o por una función objetivo agregada, de modo que el espacio de compromisos queda oculto tras una decisión previa.

El aprendizaje por refuerzo ofrece una alternativa diferente. Pozzi, Herbert, Sengupta y Mullins plantean el *routing* como un problema de *deep Q-learning* y muestran que una política aprendida puede superar a compiladores cuánticos avanzados en circuitos aleatorios y realistas [24]. Más recientemente, Kölle et al. han desarrollado un entorno de RL para síntesis dirigida de circuitos, mientras que Kremer et al. han mostrado que RL puede mejorar tanto la síntesis como el *routing* y reducir profundidad y coste de dos cúbits frente a heurísticas como SABRE [18, 19]. La ventaja de este enfoque es que la política aprende a tomar decisiones secuenciales a partir de la observación del estado, pero su eficacia depende fuertemente del diseño del entorno, de la representación del estado y del moldeado

de la recompensa.

En esta memoria, la combinación $MO \rightarrow RL$ se adopta como una hipótesis de diseño. La optimización multiobjetivo permite explorar varios *layouts* iniciales no dominados y exponer explícitamente el frente de Pareto, en lugar de colapsar el problema en una única decisión temprana. RL, por su parte, aborda las decisiones secuenciales posteriores sobre *routing* o *synthesis*, donde el valor de cada acción depende del estado alcanzado hasta ese momento. Separar ambas fases permite estudiar si un punto de partida elegido con criterios multiobjetivo facilita después la política aprendida, sin presentar la combinación como una conclusión general del campo.

3

Tecnologías y entorno experimental

3.1 Pila de *software* principal

El prototipo se apoya en una cadena de herramientas que combina ejecución cuántica, aprendizaje por refuerzo, optimización multiobjetivo y análisis numérico. El núcleo de ejecución lo forman Python 3.10.8 y Qiskit 2.3.0, que proporcionan el entorno de desarrollo y la abstracción de circuitos, *backends* y transpilación. Sobre esa base se integra PyTorch 2.5.1 con soporte CUDA 12.1 para los componentes entrenables, mientras que Gymnasium 1.2.3 y Stable-Baselines3 2.7.1 actúan como interfaz y motor del entrenamiento por refuerzo.

La capa de búsqueda y selección de soluciones se apoya en *pymoo* 0.6.1.6 y Optuna. La primera aporta algoritmos evolutivos multiobjetivo, como NSGA-II y MOEA/D, y la segunda facilita el ajuste sistemático de hiperparámetros. TensorBoard se emplea para monitorizar la evolución del entrenamiento, y NumPy, SciPy, pandas y matplotlib cubren el tratamiento de datos, el cálculo científico y la visualización de resultados.

Para mantener una composición más limpia, las Figuras 3.1, 3.2 y 3.3 agrupan varios logotipos por fila, conservan su identificación individual y acompañan cada bloque con una breve explicación de su papel en el prototipo.

Lenguajes y núcleo de ejecución

La Figura 3.1 recoge el lenguaje de desarrollo, el SDK cuántico y el *backend* de aprendizaje profundo sobre el que se construye el resto de la arquitectura [20].



Figura 3.1. Lenguajes y núcleo de ejecución.

Aprendizaje por refuerzo y optimización

La Figura 3.2 agrupa la interfaz del entorno, la biblioteca de entrenamiento de agentes, la optimización multiobjetivo, el ajuste de hiperparámetros y la monitorización de experimentos.

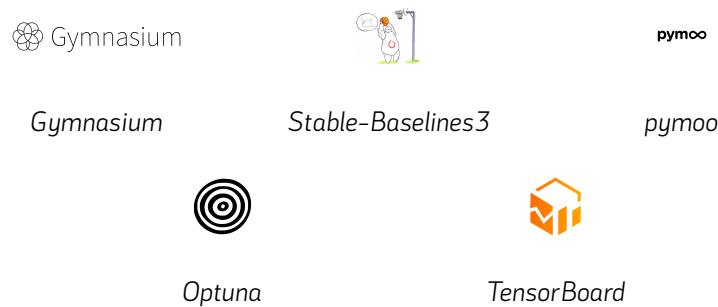


Figura 3.2. Aprendizaje por refuerzo, optimización y seguimiento experimental.

Análisis científico y visualización

La Figura 3.3 reúne las bibliotecas que se utilizan para el tratamiento numérico de datos, el cálculo científico y la generación de gráficas.



Figura 3.3. Análisis científico y visualización.

3.2 Qiskit 2.x y backends simulados

El prototipo se apoya en Qiskit 2.x como capa de ejecución cuántica y de transpilación de referencia, siguiendo un modelo centrado en la descripción explícita de *targets* y de las etapas del proceso de *transpilation* [11, 17]. Sobre esa base, el proyecto evita depender de credenciales reales de IBM Quantum y trabaja con *fake backends* del ecosistema de Qiskit. Estos *backends* reproducen instantáneas

de sistemas reales y conservan información *hardware* relevante, como el `coupling_map`, las puertas nativas y propiedades del dispositivo que influyen en la calidad de la compilación [9].

Los *backends* más relevantes para la memoria son tres:

- `fake_torino`, que representa un dispositivo Heron de 133 cúbits y aporta un patrón de conectividad distinto al de las referencias Eagle.
- `fake_sherbrooke`, una referencia Eagle de 127 cúbits para estudiar la transpilación sobre una topología de gran escala.
- `fake_brisbane`, otra referencia Eagle de 127 cúbits para contrastar una segunda instantánea de *backend*.

A alto nivel, los tres representan el mismo principio: una conectividad física limitada en la que la distancia lógica entre cúbits condiciona el coste final de la transpilación. La Figura 3.4 muestra los mapas de acoplamiento correspondientes y permite comparar la estructura física de cada dispositivo simulado.

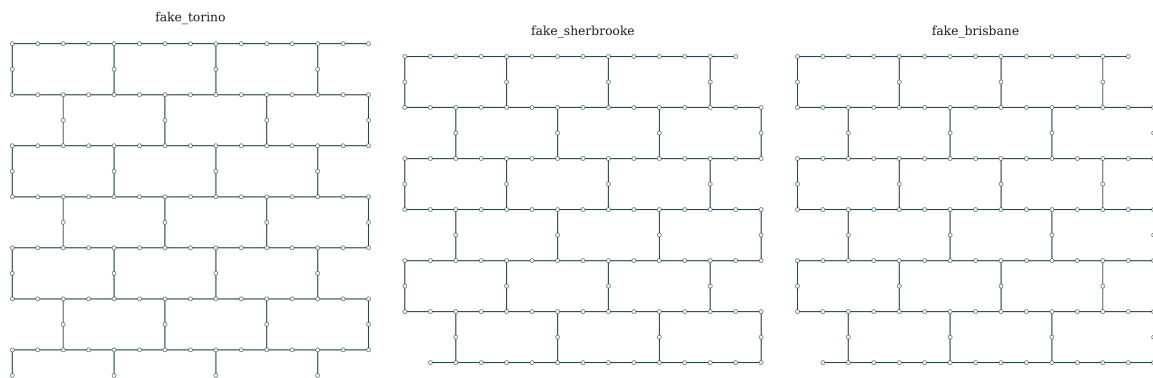


Figura 3.4. Mapas de acoplamiento de los *fake backends* principales. Elaboración propia a partir de las coordenadas y acoplamientos incluidos en sus instantáneas de Qiskit.

Las topologías sintéticas, por su parte, son mapas de acoplamiento idealizados. Sirven para estudiar cómo cambia el problema cuando se modifica la forma de la conectividad, desde estructuras muy densas hasta patrones más esparsos y regulares [8]. Entre las familias más representativas están `full`, `line`, `ring`, `t`, `grid`, `heavy_hex`, `heavy_square` y `hexagonal_lattice`. `full` modela conectividad completa; `line` y `ring` describen cadenas unidimensionales, con y sin cierre; y `t` introduce una estructura con barra superior y tallo central balanceados, útil para estudiar un punto

intermedio entre linealidad y ramificación. Por su parte, `grid` y `hexagonal_lattice` corresponden a retículas bidimensionales, mientras que `heavy_hex` y `heavy_square` representan variantes esparsas inspiradas en diseños de *hardware* reales. En la implementación, la familia `t` se identifica como `synthetic_t_Nq`, exige al menos cinco cúbits físicos y reparte los nodos entre barra y tallo con un criterio balanceado, manteniendo ambos segmentos con longitud mínima de tres cúbits. La Figura 3.5 resume visualmente estas familias y facilita la comparación entre conectividades densas, lineales, ramificadas y reticulares.

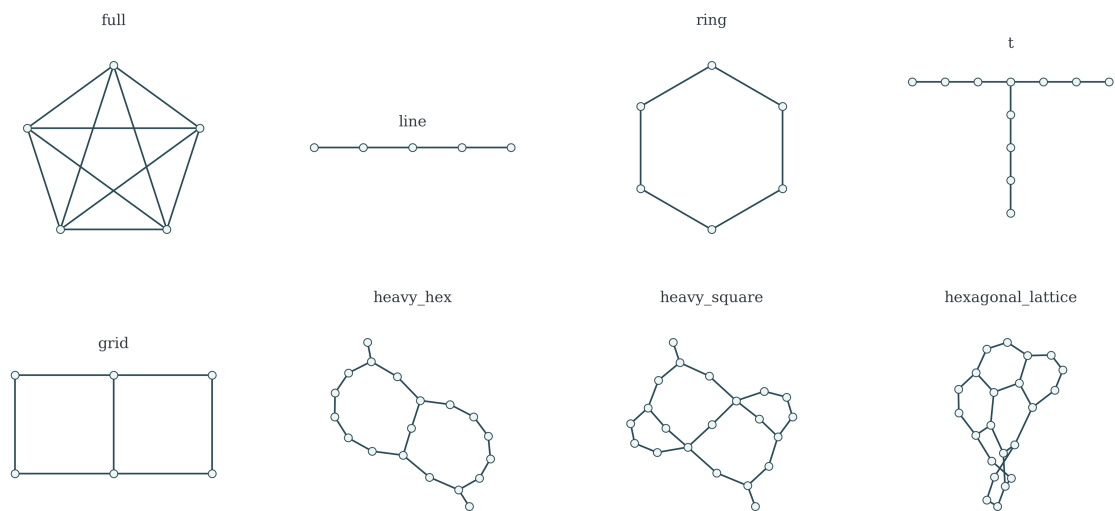


Figura 3.5. Familias de topologías sintéticas de acoplamiento consideradas, incluida la variante *t* balanceada. Elaboración propia a partir de mapas de acoplamiento generados con *Qiskit* y de la especificación sintética de *integration*.

En conjunto, estas familias cubren desde conectividades densas hasta geometrías más restrictivas. Esa variedad permite analizar el efecto de la topología sobre la transpilación sin depender de un único dispositivo ni de un único patrón de conectividad.

3.3 Herramientas para optimización multiobjetivo y aprendizaje por refuerzo

La parte de optimización multiobjetivo se apoya principalmente en `pymoo`. Esta biblioteca proporciona una base flexible para definir problemas de optimización, ejecutar algoritmos evolutivos y adaptar los operadores de búsqueda a representaciones específicas [3, 26]. En el proyecto resulta adecuada porque la elección de un *layout* inicial no se reduce a mejorar una única métrica: interesa explorar compromisos entre criterios de calidad de la transpilación y conservar un conjunto de soluciones no

dominadas. Sobre esta base se emplean NSGA-II y MOEA/D como algoritmos de referencia para obtener frentes de Pareto a partir de poblaciones de *layouts* candidatos.

Optuna complementa ese bloque como herramienta de ajuste de hiperparámetros. Su función no es sustituir al optimizador multiobjetivo, sino recorrer configuraciones posibles del proceso de búsqueda y comparar su rendimiento experimental de forma sistemática [23]. Esta separación permite mantener, por un lado, el problema de selección de *layouts* y, por otro, la elección de parámetros que condicionan el comportamiento del algoritmo evolutivo. En consecuencia, *pymoo* actúa como motor de optimización y Optuna como soporte para afinar ese motor cuando se estudian configuraciones alternativas.

El bloque de aprendizaje por refuerzo utiliza Gymnasium como interfaz de entorno. Gymnasium fija una API común para representar observaciones, acciones, recompensas y transiciones mediante operaciones como `reset()` y `step()` [6]. Esa abstracción encaja con la formulación secuencial de la transpilación: el estado del problema se presenta al agente, el agente selecciona una acción y el entorno devuelve la evolución resultante junto con la señal de recompensa.

Sobre ese contrato, Stable-Baselines3 proporciona el marco de entrenamiento de políticas. La biblioteca reúne implementaciones y utilidades de aprendizaje por refuerzo que facilitan entrenar y evaluar agentes sin reimplementar desde cero el ciclo de optimización de la política [27]. En el prototipo sirve de soporte para los agentes empleados por el módulo RL, mientras que TensorBoard se utiliza como herramienta de seguimiento para registrar y visualizar la evolución de las métricas durante el entrenamiento [29, 28]. De este modo, Gymnasium define la interacción agente–entorno, Stable-Baselines3 ejecuta el aprendizaje y TensorBoard aporta observabilidad sobre el proceso experimental.

3.4 Reproducibilidad y limitaciones del entorno

La reproducibilidad es un requisito transversal en el entorno experimental del proyecto. La comparación entre estrategias de transpilación solo resulta interpretable si se controlan, en la medida de lo posible, las fuentes de variación que afectan a la generación de soluciones, a la evaluación de los circuitos y al entrenamiento de los agentes. Por ello, el diseño adopta una política orientada a semillas: las semillas forman parte de la configuración de los experimentos y se propagan a los bloques que incorporan decisiones estocásticas.

En la interfaz con Qiskit, la semilla se transmite al proceso de transpilación para reducir la variabilidad introducida por las etapas internas del compilador. Además, los experimentos reutilizan descripciones de *backend* coherentes dentro de una misma comparación. Esta decisión es especialmente relevante cuando se trabaja con *fake backends*, pues permite mantener fija la topología, el conjunto

de puertas y las propiedades del dispositivo simulado mientras se estudia el efecto de una estrategia de *layout* o de *routing*.

La optimización multiobjetivo sigue el mismo criterio. Los algoritmos evolutivos operan sobre poblaciones y operadores de búsqueda que pueden depender de factores estocásticos, de modo que la semilla queda asociada a la configuración de optimización y a la evaluación de los candidatos. Cuando interesa valorar la robustez de una configuración, no basta con observar una única ejecución favorable: el entorno contempla la repetición sobre varias semillas para estimar mejor el comportamiento medio del proceso de búsqueda.

En aprendizaje por refuerzo, la preparación del entrenamiento fija semillas para las bibliotecas principales y para el reinicio del entorno. A ello se suma la selección automática del dispositivo de cómputo: si PyTorch detecta una GPU compatible con CUDA, el agente puede entrenarse sobre ella; en caso contrario, se mantiene la ejecución sobre CPU. Esta adaptación evita imponer un *hardware* concreto como requisito para ejecutar el prototipo, aunque el tiempo de entrenamiento pueda variar de forma notable entre entornos.

Estas medidas mejoran la comparabilidad de los resultados, pero no deben interpretarse como una garantía de repetición *bit a bit* en cualquier circunstancia. La transpilación, la optimización evolutiva y el entrenamiento de políticas combinan bibliotecas, operadores personalizados y, potencialmente, dispositivos de cómputo distintos. Mientras existan fuentes de aleatoriedad no completamente sincronizadas en todos esos niveles, la reproducibilidad estricta puede no ser perfecta. En consecuencia, los resultados experimentales deben acompañarse de la semilla utilizada, la configuración del entorno y, cuando proceda, agregaciones sobre varias ejecuciones.

4

Arquitectura y diseño de la solución

4.1 Visión global del sistema

La solución desarrollada se organiza como una arquitectura modular alrededor del proceso de transpilación cuántica. En lugar de construir un único bloque monolítico, el sistema separa la interacción con Qiskit, la búsqueda de *layouts*, el aprendizaje por refuerzo y la comparación de escenarios en cuatro módulos: `qiskit_interface`, `mo_module`, `rl_module` e `integration`. Esta división permite estudiar cada técnica de forma aislada y, al mismo tiempo, combinarlas mediante contratos explícitos cuando se ejecutan flujos híbridos.

La Figura 4.1 resume esta organización y muestra los principales flujos de información entre módulos.

El módulo `qiskit_interface` actúa como capa de entrada al ecosistema de Qiskit. Su responsabilidad es cargar o generar circuitos, resolver *backends* simulados, extraer información de *hardware* y ejecutar transpilaciones de referencia. También proporciona métricas estructuradas del circuito antes y después de la transpilación, como profundidad, número de puertas, puertas de dos cúbits y datos asociados al *backend*. De este modo, el resto del sistema no necesita depender directamente de los detalles internos de Qiskit para obtener circuitos, topologías o resultados comparables.

Sobre esa base, `mo_module` aborda la selección del *layout* inicial como un problema de optimización multiobjetivo. El módulo recibe un circuito y una descripción del *backend*, construye un espacio de búsqueda de asignaciones lógico-físicas y evalúa *layouts* candidatos mediante objetivos como la profundidad y el número de puertas de dos cúbits tras la transpilación. Su salida principal no

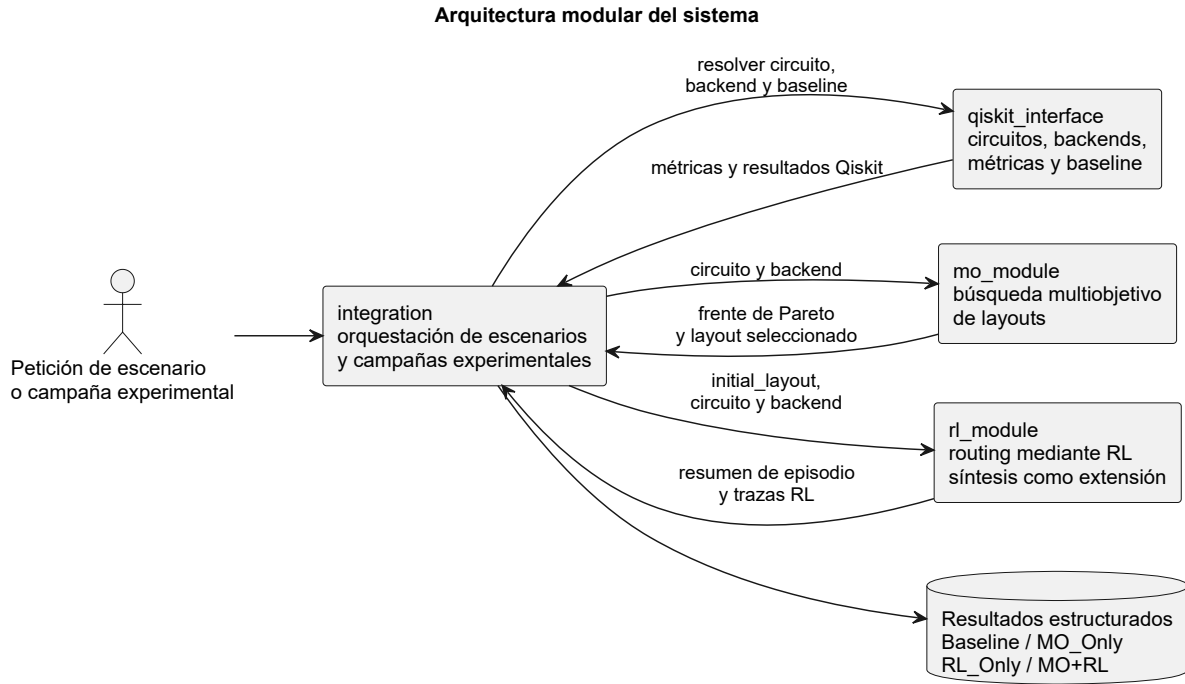


Figura 4.1. Visión global de la arquitectura modular del sistema.

es una única solución fija, sino un frente de Pareto junto con mecanismos de selección, por ejemplo un *layout* de compromiso, que pueden ser consumidos por otros componentes.

El módulo `rl_module` formula parte de la transpilación como un problema de decisión secuencial. En el modo de *routing*, el agente decide qué operaciones `SWAP` aplicar sobre el mapa de acoplamiento para desbloquear puertas del circuito bajo un *layout* dado. El módulo se centra en el entorno, las observaciones, las recompensas, el entrenamiento y la evaluación de episodios; no genera *layouts* iniciales por sí mismo.

Aunque el alcance base analizado en los escenarios del proyecto es el *routing*, la arquitectura de `rl_module` contempla una extensión de síntesis que se describe posteriormente en el capítulo de ampliaciones.

La capa `integration` conecta las piezas anteriores y actúa como punto común desde el que se comparan los escenarios del proyecto. Su función es resolver la entrada del experimento, seleccionar el circuito y el *backend*, invocar el módulo que corresponda y devolver resultados estructurados para los escenarios `Baseline`, `MO_Only`, `RL_Only` y `MO+RL`. En las rutas con aprendizaje por refuerzo, esos resultados pueden contener métricas finales cuando el *routing* completa, o limitarse al resumen del episodio y sus incidencias cuando no existe circuito *post-routing* evaluable. En el flujo híbrido, esta capa valida el *layout* producido por `mo_module` y lo entrega a `rl_module` como *layout* inicial.

Desde el punto de vista arquitectónico, cada ejecución entra por *integration*, que interpreta la petición del escenario o de la campaña experimental y delega el trabajo en los módulos especializados. En la implementación se denomina *Campaign* a un conjunto reproducible de ejecuciones experimentales, normalmente compuesto por varios casos circuito-*backend*. Los circuitos y *backends* se normalizan mediante *qiskit_interface*; los *layouts* candidatos se exploran en *mo_module*; y las decisiones secuenciales de *routing* se estudian en *rl_module*. Esta estructura facilita aislar el efecto de cada componente: *Baseline* mide el comportamiento de Qiskit sin *layout* externo, *MO_Only* evalúa el impacto de un *layout* optimizado, *RL_Only* analiza el comportamiento del agente de refuerzo y *MO+RL* estudia la combinación de un *layout* multiobjetivo con una política de RL aplicada al *routing*.

La arquitectura no pretende sustituir al transpilador de Qiskit completo, sino construir una infraestructura experimental para analizar decisiones concretas del flujo de transpilación. En particular, la versión actual debe interpretarse con cautela: *integration* coordina escenarios centrados en *routing* y obtiene métricas homogéneas cuando el episodio RL completa y la evaluación *post-routing* finaliza correctamente. En los casos incompletos conserva trazas, resúmenes e incidencias para analizarlos por separado, sin incorporarlos a los agregados comparativos.

4.2 Contrato compartido de *layout*

El contrato de datos más importante de la arquitectura es la representación común del *layout*. En este proyecto, un *layout* se expresa como un *array* de enteros en el que cada posición representa un cúbit lógico y el valor almacenado en esa posición indica el cúbit físico asignado. La convención se mantiene de forma explícita en toda la memoria:

$$\text{layout}[i] = \text{physical_qubit_for_logical_qubit_i}$$

Por ejemplo, si $\text{layout} = [3, 0, 5]$, el cúbit lógico 0 se sitúa sobre el cúbit físico 3, el cúbit lógico 1 sobre el cúbit físico 0 y el cúbit lógico 2 sobre el cúbit físico 5. La Figura 4.2 resume esta interpretación y muestra cómo el mismo contrato sirve de punto de paso entre los módulos principales.

Esta representación se interpreta como una permutación parcial cuando el *backend* tiene más cúbits físicos que el circuito lógico. Para un circuito de n cúbits lógicos y un *backend* con N cúbits físicos, el *array* tiene longitud n y contiene n identificadores físicos distintos escogidos entre los N disponibles. La restricción de unicidad evita asignar dos cúbits lógicos al mismo cúbit físico, mientras que la restricción de pertenencia garantiza que cada valor del *array* corresponde a una posición física válida del *backend*. Los cúbits físicos no utilizados no aparecen en el *array* compartido.

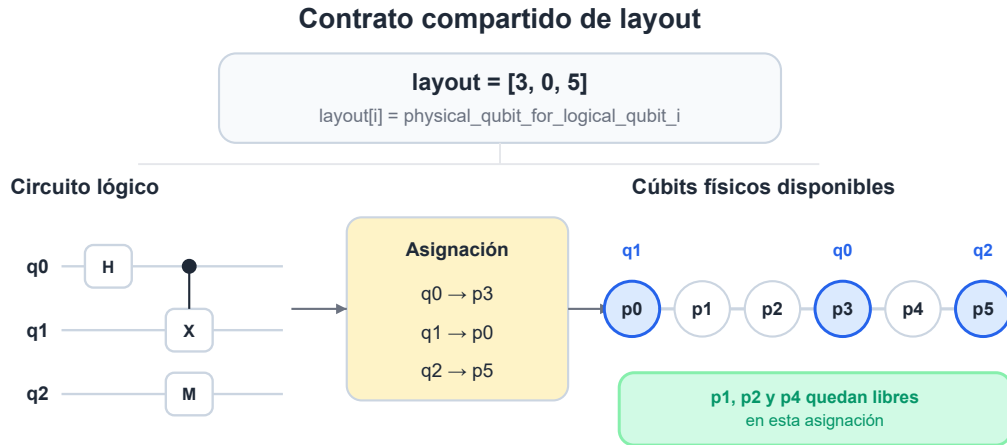


Figura 4.2. Interpretación del contrato compartido de layout.

El módulo `mo_module` utiliza esta representación como individuo dentro de la búsqueda evolutiva. Cada candidato del algoritmo es una asignación lógico–física factible, y los operadores de muestreo, cruce, mutación, validación y reparación trabajan sobre esa forma de *array*. Al finalizar la optimización, el módulo puede devolver un frente de Pareto con múltiples *layouts* y, cuando hace falta ejecutar un escenario concreto, una selección de *layout* determinada por un criterio de compromiso o por una política definida.

En `qiskit_interface`, el mismo *layout* actúa como entrada externa para evaluar el efecto de una asignación inicial sobre el proceso de transpilación. Esto permite comparar un *layout* producido por `mo_module` con una transpilación de referencia sin obligar al resto del sistema a manejar directamente las estructuras internas de Qiskit. La capa de interfaz traduce el contrato compartido al formato que necesita Qiskit y devuelve resultados estructurados con métricas comparables.

En `rl_module`, el *layout* aparece como `initial_layout`. El módulo no decide de dónde procede esa asignación: puede ser trivial, proporcionada por el llamador o seleccionada previamente por `integration`. Desde ese punto de partida, el entorno actualiza la asignación durante el *routing* a medida que el agente aplica operaciones SWAP. Los detalles internos usados para consultar o invertir la asignación pertenecen al propio módulo RL y no modifican el contrato compartido.

La responsabilidad de `integration` es hacer que ese contrato circule sin ambigüedad entre productor y consumidor. En los escenarios basados en optimización multiobjetivo, la capa de integración selecciona un *layout* único a partir de la salida de `mo_module`, valida que la asignación sea compa-

ible con el circuito y el *backend*, y lo entrega a la etapa correspondiente. En *MO_Only*, ese *layout* se evalúa mediante Qiskit; en *MO+RL*, se reutiliza como *initial_layout* del episodio de *routing*. Así, el contrato de *layout* funciona como una frontera estable entre módulos: permite componer técnicas distintas sin mezclar sus responsabilidades internas.

4.3 Responsabilidades por módulo

La división por módulos es una decisión de diseño necesaria para mantener comparables los experimentos. En este punto, la Tabla 4.1 no pretende repetir la descripción anterior, sino fijar los límites entre componentes para evitar solapamientos: cada módulo asume una parte concreta del flujo de transpilación y evita invadir la lógica interna de los demás. Esta separación permite modificar, entrenar o evaluar un bloque sin alterar el significado de los resultados producidos por otro.

Módulo	Responsabilidad principal	Fuera de su alcance
<code>qiskit_interface</code>	Proporciona circuitos, <i>backends</i> simulados, métricas y transpilaciones de referencia. También evalúa <i>layouts</i> externos mediante el flujo de Qiskit.	No orquesta optimización multiobjetivo, aprendizaje por refuerzo ni campañas experimentales.
<code>mo_module</code>	Genera <i>layouts</i> candidatos, evalúa su <i>fitness</i> , construye frentes de Pareto y ofrece mecanismos para seleccionar una solución útil.	No entrena agentes RL, no ejecuta episodios de <i>routing</i> y no define los escenarios públicos.
<code>rl_module</code>	Define el entorno Gymnasium, las observaciones, acciones, recompensas, agentes y herramientas de entrenamiento o evaluación por refuerzo.	No genera <i>layouts</i> iniciales, no decide su origen y no orquesta el <i>handoff</i> desde MO.
<code>integration</code>	Resuelve peticiones de escenario o campaña experimental, conecta los módulos, selecciona <i>layouts</i> cuando procede y estructura los resultados.	No reimplementa la lógica interna de Qiskit, de la optimización multiobjetivo ni del entorno RL.

Tabla 4.1. Responsabilidades y límites principales de cada módulo.

`qiskit_interface` ocupa la frontera entre el proyecto y Qiskit. Su papel es estabilizar el acceso a circuitos, *backends* y métricas para que el resto de la arquitectura trabaje con artefactos homogé-

neos. Cuando se ejecuta un *baseline*, este módulo proporciona la ruta de referencia. Cuando se evalúa un *layout* externo, también es el encargado de traducir esa asignación a una transpilación concreta y devolver métricas estructuradas. Sin embargo, no decide qué *layout* debe usarse ni compara escenarios híbridos por sí mismo.

`mo_module` se limita a la búsqueda multiobjetivo de *layouts*. Su responsabilidad termina en la producción y análisis de candidatos, junto con la posibilidad de seleccionar una solución del frente de Pareto. Esta frontera es importante porque evita presentar el módulo MO como un sistema completo de transpilación. El módulo no dirige el entrenamiento RL ni ejecuta episodios; se comporta como productor de *layouts* que después pueden ser evaluados o reutilizados por una capa externa.

`rl_module` asume la parte secuencial del problema. En el flujo de *routing*, el agente observa el estado del circuito y del *layout* actual, elige acciones de tipo **SWAP** y recibe recompensas asociadas al progreso del episodio. El módulo puede consumir un `initial_layout`, pero no lo genera ni decide si procede de una heurística, de un *layout* trivial o de `mo_module`. Por ese motivo, su frontera con MO se mantiene deliberadamente indirecta: ambos módulos se conectan a través de `integration`.

`integration` actúa como capa de composición. Su tarea consiste en recibir una petición de escenario o campaña experimental, resolver los datos necesarios, llamar al módulo correspondiente y empaquetar la salida en un resultado común. En el caso híbrido, esta capa selecciona un *layout* procedente de MO y lo entrega al módulo RL como *layout* inicial. No obstante, esa coordinación no convierte a `integration` en una nueva implementación de optimización ni de aprendizaje por refuerzo. Su valor está en preservar los contratos, mantener la trazabilidad de las decisiones y hacer explícito qué parte del resultado procede de cada componente.

Esta delimitación también condiciona la lectura experimental de la memoria. Si un resultado mejora al usar `MO_Only`, la mejora se atribuye al efecto del *layout* inicial evaluado con Qiskit. Si cambia el comportamiento de `RL_Only` o `MO+RL`, el análisis debe referirse a métricas de episodio y al *layout* de partida utilizado por el agente. Separar responsabilidades evita atribuir a un módulo efectos que en realidad pertenecen a otro y permite que las comparaciones posteriores sean más honestas.

4.4 Flujo híbrido y escenarios de evaluación

El flujo de evaluación se articula alrededor de `integration`. Toda ejecución parte de una petición de escenario, o de un caso dentro de una campaña experimental, que identifica el circuito, el *backend* y los parámetros necesarios para la ruta seleccionada. A partir de esa petición, la capa de integración resuelve los datos comunes, valida los contratos de entrada y decide qué módulos deben intervenir.

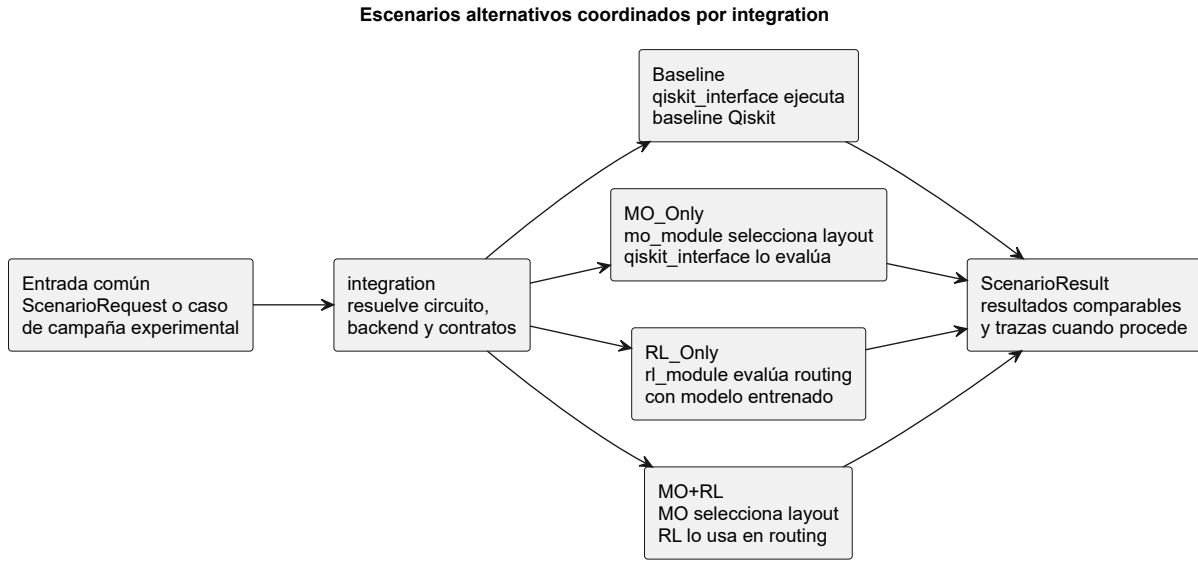


Figura 4.3. Flujo general de escenarios coordinado por *integration*.

La Figura 4.3 sintetiza esa bifurcación en los cuatro escenarios evaluados y su retorno a un resultado común. Tras esa visión gráfica, el texto concreta qué datos circulan por cada ruta.

`ScenarioRequest` representa la entrada normalizada de una ejecución. `ScenarioResult` agrupa las métricas, artefactos y estado final devuelto por cada escenario. El primer paso común es disponer de una descripción coherente del circuito y del *backend*. Para ello, *integration* se apoya en `qiskit_interface`, que carga o genera el circuito, obtiene la información del *backend* y ofrece las métricas necesarias para los escenarios basados en Qiskit. Cuando el escenario necesita un *layout* procedente de optimización multiobjetivo, *integration* invoca `mo_module`, recibe el frente de Pareto o la solución seleccionada y valida que el *layout* sea compatible con el circuito y con el *backend*. Cuando el escenario requiere aprendizaje por refuerzo, *integration* carga el modelo RL indicado y ejecuta la evaluación de *routing* a través de `rl_module`.

La Tabla 4.2 deja ver la diferencia práctica entre escenarios. *Baseline* fija la referencia sin *layout* externo; *MO_Only* mide el efecto de seleccionar un *layout* multiobjetivo y evaluarlo con la misma infraestructura de Qiskit; *RL_Only* estudia el comportamiento del agente de *routing* desde el *layout* inicial de Qiskit; y *MO+RL* estudia la combinación de un *layout* seleccionado por MO con una política de RL aplicada al *routing*. La comparación entre *RL_Only* y *MO+RL* no evalúa “RL para toda la transpilación”, sino el cambio que produce el punto de partida del episodio y la política de *routing* entrenada.

Durante las evaluaciones basadas en RL se registra un resumen de episodio con recompensa, pasos, *swaps*, *layout* inicial, *layout* final y trazas de ejecución. Si el episodio completa, *integration*

Escenario	Ruta principal	Layout inicial	Salida principal
Baseline	Transpilación de referencia mediante <code>qiskit_interface</code> .	Definido por Qiskit, sin <i>layout</i> externo del proyecto.	Métricas de transpilación y resultado Qiskit.
MO_Only	Optimización multiobjetivo en <code>mo_module</code> y evaluación posterior con Qiskit.	<i>Layout</i> seleccionado desde el frente de Pareto.	Métricas de transpilación con <i>layout</i> externo.
RL_Only	Evaluación de <i>routing</i> con <code>rl_module</code> y un modelo entrenado.	<i>Layout</i> inicial de Qiskit, sin <i>layout</i> MO.	Resumen RL y, si completa, métricas <i>post-routing</i> de Qiskit.
MO+RL	Selección de <i>layout</i> en <code>mo_module</code> y evaluación de <i>routing</i> con <code>rl_module</code> .	<i>Layout</i> seleccionado por MO y entregado como <code>initial_layout</code> .	Resumen RL y, si completa, métricas <i>post-routing</i> de Qiskit.

Tabla 4.2. Resumen de los escenarios de evaluación definidos por *integration*.

reconstruye internamente el circuito ruteado y lo entrega a la etapa *post-routing* de Qiskit, de modo que también se obtienen métricas de transpilación comparables. Si el episodio no completa, el escenario conserva la información del episodio, pero no produce ese artefacto *post-routing*.

La consecuencia metodológica es directa: los cuatro escenarios solo pueden compararse mediante métricas de transpilación en los casos donde las rutas RL completan el *routing* y Qiskit evalúa el artefacto *post-routing*. En ejecuciones incompletas, la interpretación debe limitarse al diagnóstico del episodio RL y a las incidencias registradas por *integration*. Así se evita mezclar circuitos finales con trazas parciales.

4.5 Decisiones de diseño y alcance actual

El sistema se plantea como una arquitectura experimental donde cada componente puede evaluarse con una responsabilidad bien definida. Esta decisión condiciona todo el capítulo: `qiskit_interface` estabiliza el acceso a Qiskit, `mo_module` produce y selecciona *layouts*, `rl_module` modela decisiones secuenciales y *integration* compone escenarios reproducibles. La separación permite cambiar una estrategia de *layout*, entrenar una política distinta o repetir una campaña experimental sin redefinir el significado de los resultados.

La segunda decisión relevante es trabajar con *backends* simulados y configuraciones controladas. El uso de *fake backends* permite reproducir topologías, `coupling_map` (mapa de conectividad física

entre cúbits), `basis_gates` (conjunto de puertas nativas disponibles) y restricciones de transpilación sin depender de credenciales, disponibilidad de *hardware* real o cambios de calibración. Esta elección no elimina la orientación *hardware-aware* del sistema, pero sí acota el tipo de afirmaciones que se hacen en la memoria: los resultados analizan el comportamiento de la arquitectura bajo modelos de *backend* reproducibles, no la ejecución física de circuitos en un dispositivo cuántico real.

El alcance principal de *integration* queda centrado en los escenarios de evaluación de *routing* descritos en la sección anterior. En *Baseline* y *MO_Only*, la salida se obtiene directamente mediante Qiskit; en *RL_Only* y *MO+RL*, el sistema ejecuta un episodio RL y, solo si este completa el *routing*, reconstruye el circuito ruteado para evaluarlo en la etapa *post-routing* de Qiskit. La comparación experimental distingue por ello entre casos completos, con métricas de transpilación disponibles para todos los escenarios, y casos incompletos, donde solo procede analizar la traza del episodio y las incidencias registradas.

También existe un modo *synthesis* dentro de `rl_module`, pero su papel en esta memoria debe interpretarse como una expansión de la línea base del proyecto, no como parte del flujo híbrido central $MO \rightarrow RL$. Este modo comparte la infraestructura de entorno, estrategia, recompensa y entrenamiento, aunque cambia la semántica del problema: en lugar de insertar *SWAPs* para desbloquear puertas, el agente selecciona primitivas nativas para reducir un residual de síntesis. Su alcance actual es deliberadamente acotado: trabaja sobre circuitos Clifford, requiere `basis_gates` explícitas y mantiene el *layout* fijo durante el episodio. Por ello se mencionará posteriormente como ampliación funcional en la Sección 9.4, donde encaja mejor su motivación y su contribución respecto a la línea base.

Estas decisiones de alcance ayudan a leer el resto de la memoria. La arquitectura central se evalúa como una composición de *layout* multiobjetivo y *routing* mediante aprendizaje por refuerzo; las capacidades adicionales, como *synthesis*, se presentan como extensiones que amplían el marco experimental sin alterar la interpretación de los escenarios principales. De este modo, el capítulo cierra con una frontera clara entre el núcleo comparativo del trabajo y las ampliaciones que enriquecen la plataforma.

5

Desarrollo del módulo `qiskit_interface`

5.1 Estructura general del módulo

5.1.1 Visión global

`qiskit_interface` se diseñó como una capa de frontera entre Qiskit y los módulos propios del proyecto. Su objetivo no es ocultar por completo la existencia de Qiskit, ya que el sistema trabaja de forma explícita con `QuantumCircuit`, *backends* simulados y resultados de transpilación, sino estabilizar los puntos de contacto que necesita el resto de la arquitectura. De esta forma, `mo_module` e `integration` pueden consumir circuitos, propiedades *hardware* y métricas comparables sin depender de detalles cambiantes de bajo nivel.

La estructura interna del módulo responde a tres responsabilidades principales. En primer lugar, `circuit_utils.py` concentra la gestión de circuitos: carga desde biblioteca interna, lectura y exportación en QASM 2 o QASM 3, conversión entre representaciones y extracción de métricas básicas. QASM es un formato textual estándar para describir circuitos cuánticos, de forma que puedan cargarse desde fichero y reconstruirse como objetos `QuantumCircuit`. En segundo lugar, `backend_info.py` encapsula la consulta de *backends* simulados y traduce sus propiedades a una estructura homogénea. Por último, `transpiler.py` ejecuta la transpilación de referencia con Qiskit, evalúa *layouts* externos y devuelve resultados estructurados para comparación experimental.

Esta separación evita que cada módulo replique su propia forma de crear circuitos, leer topologías o contar puertas. También permite mantener una frontera clara de alcance: `qiskit_interface` no

decide qué escenario se ejecuta, no selecciona soluciones de un frente de Pareto y no entrena agentes de aprendizaje por refuerzo. Su papel es proporcionar artefactos fiables y reutilizables para que esas decisiones se tomen en las capas especializadas correspondientes.

5.1.2 Bloques funcionales y contratos

La fachada pública del paquete se define en `__init__.py`. Este fichero reexporta los símbolos usados con mayor frecuencia para que los consumidores del módulo no tengan que acoplarse a la organización interna de archivos. Así, una capa externa puede importar funciones como `load_circuit`, `get_backend`, `extract_backend_info`, `transpile_circuit` o `run_named_baseline` desde el propio paquete, manteniendo una superficie de uso más compacta. La Tabla 5.1 resume los contratos que estabilizan esta superficie.

Contrato	Productor principal	Información que estabiliza
QuantumCircuit con metadatos	<code>load_circuit</code>	Circuito cargado o generado, junto con su procedencia: tipo de fuente, formato, ruta de fichero cuando existe y nombre resuelto.
CircuitMetrics	<code>circuit_utils.py</code>	Métricas cuantitativas de circuitos originales, transpilados o reconstruidos: profundidad, puertas, anchura, cúbits activos y conteos por tipo de operación.
BackendInfo	<code>backend_info.py</code>	Descripción <i>hardware</i> del <i>backend</i> simulado: conectividad, aristas físicas, puertas nativas, puerta de dos cúbits, errores y tiempos de coherencia.
TranspilationResult	<code>transpiler.py</code>	Resultado completo de una transpilación o <i>baseline</i> : circuitos, métricas pre/post, nivel de optimización, <i>layouts</i> , <i>backend</i> , semilla y artefacto serializable.

Tabla 5.1. Contratos centrales expuestos por *qiskit_interface*.

La función de carga no introduce una clase adicional para el circuito cargado. Devuelve el circuito de Qiskit enriquecido con metadatos en el campo `metadata`. Ese contrato encapsula el circuito ejecutable, su procedencia y los datos mínimos de generación o carga: tipo de fuente, formato, ruta de fichero cuando existe y nombre resuelto. Gracias a ello, el resto del sistema trabaja con el tipo

estándar de Qiskit, pero conserva trazabilidad sobre si el circuito procede de la biblioteca interna o de un fichero externo.

`circuit_utils.py` aporta el contrato `CircuitMetrics`. Este objeto resume un circuito mediante campos serializables como profundidad, número de cúbits, número total de puertas, puertas de dos cúbits, puertas no locales, coste equivalente en CNOT, desglose por tipo de puerta, anchura, bits clásicos y cúbits activos. La distinción entre anchura materializada y cúbits activos resulta útil cuando una transpilación o una reconstrucción posterior ocupa más registro físico del que realmente participa en operaciones relevantes.

`backend_info.py` define el contrato `BackendInfo`. En él se agrupan el nombre y tamaño del *backend*, el `coupling_map`, la lista de aristas físicas, las `basis_gates`, la puerta nativa de dos cúbits, las puertas de un cúbit, tiempos de coherencia, errores de puerta y metadatos de topología. La puerta nativa de dos cúbits se detecta de forma dinámica a partir del *backend*, lo que permite trabajar con dispositivos como `fake_torino`, `fake_sherbrooke` y `fake_brisbane` sin asumir siempre una misma primitiva de interacción.

`transpiler.py` organiza el contrato `TranspilationResult`. Este objeto conserva el circuito original, el circuito transpilado, las métricas antes y después de la transpilación, el nivel de optimización, el *backend* utilizado, la semilla, el *layout* inicial cuando existe, el *layout* inicial interpretado por Qiskit, el *layout* final y el tiempo de ejecución. Además, puede producir diccionarios planos para análisis tabular y artefactos estructurados que `integration` puede persistir o comparar. De esta manera, el resultado de una transpilación mantiene, además del circuito final, contexto suficiente para reproducir e interpretar la ejecución.

5.1.3 Diagrama de componentes

La Figura 5.1 resume la organización interna del módulo mediante un diagrama UML de componentes. La fachada pública concentra el acceso directo desde `integration` y `mo_module`, mientras que los tres bloques funcionales interactúan con Qiskit para obtener circuitos, propiedades de *backend* y resultados de transpilación. `rl_module` no depende directamente de esta capa: su relación con las métricas y la evaluación *post-routing* queda mediada por `integration`. La salida común de esos bloques son artefactos estructurados, no valores sueltos, lo que facilita que las campañas y escenarios posteriores puedan registrar la procedencia de cada resultado.

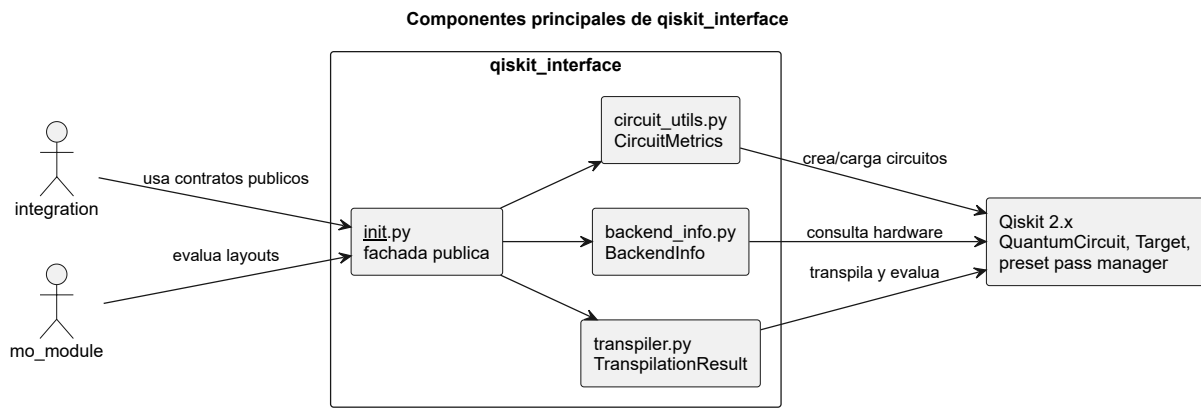


Figura 5.1. Componentes internos y contratos principales de *qiskit_interface*.

El diagrama debe leerse como una vista de paquetes, no como un diagrama de clases exhaustivo. Los contratos `CircuitMetrics`, `BackendInfo` y `TranspilationResult` son los puntos de estabilidad más importantes, pero cada bloque incluye además funciones auxiliares para tareas concretas. Por ejemplo, la carga QASM y la generación de circuitos pertenecen al bloque de circuitos; la extracción de `basis_gates`, errores y tiempos de coherencia pertenece al bloque de *backend*; y las variantes de *baseline* o transpilación con *layout* externo pertenecen al bloque de transpilación.

5.1.4 Flujo principal

El flujo principal de `qiskit_interface` empieza con la obtención de un circuito. El llamador puede solicitar una familia de la biblioteca interna, como `ghz`, `qft`, `random_shallow` o `clifford`, o proporcionar un fichero QASM. En ambos casos, el resultado se normaliza como un `QuantumCircuit` con metadatos de procedencia, y el módulo puede extraer `CircuitMetrics` antes de aplicar restricciones *hardware*.

El segundo paso consiste en resolver el *backend*. `get_backend` selecciona uno de los *backends* simulados disponibles y `extract_backend_info` transforma sus propiedades en un objeto `BackendInfo`. Así, otros módulos razonan sobre conectividad y errores sin manipular directamente la API interna de Qiskit: `mo_module` usa las aristas del `coupling_map` para construir *layouts* candidatos e `integration` registra el resumen *hardware* del escenario.

El tercer paso es la transpilación o evaluación. En la ruta `Baseline`, `run_named_baseline` y `transpile_circuit` ejecutan Qiskit con nivel de optimización y semilla reproducible. En la ruta `MO_Only`, `transpile_with_custom_layout` evalúa un *layout* externo mediante el mismo flujo de Qiskit. Para circuitos ya ruteados, `transpile_post_routing` delega en Qiskit las etapas posteriores al *routing* sin presentarse como una integración RL completa.

El resultado final se encapsula en `TranspilationResult`, con circuito transpilado, métricas previas y posteriores, *layouts* y contexto de ejecución. Así, el flujo completo puede resumirse como *circuito* → *backend* → *transpilación o evaluación* → *métricas y artefacto estructurado*.

5.2 Gestión y generación de circuitos

Una vez fijada la estructura general, la primera responsabilidad concreta de `qiskit_interface` es proporcionar una entrada estable de circuitos para el resto del sistema. En lugar de que cada módulo cree sus propios `QuantumCircuit` o dependa directamente de detalles de Qiskit, esta capa concentra la construcción, carga y conversión de circuitos en `circuit_utils.py`. De este modo, los escenarios definidos en `integration`, la evaluación de *layouts* en `mo_module` y las pruebas de *routing* con `rl_module` parten de objetos homogéneos y con una procedencia trazable.

La vía principal de entrada es `load_circuit`. Esta función distingue entre circuitos de biblioteca y circuitos cargados desde fichero QASM. Cuando se usa la biblioteca interna, la petición especifica el nombre del circuito, el número de cúbits y, si procede, una semilla. El módulo normaliza el nombre solicitado, construye el circuito correspondiente y adjunta metadatos mínimos sobre su origen. Esta información resulta útil posteriormente para identificar el caso experimental sin depender solo del nombre interno del objeto `QuantumCircuit`.

Nombre	Construcción	Uso dentro del proyecto
<code>ghz</code>	Estado GHZ con una puerta H inicial y una cadena de CX .	Circuito de entrelazamiento simple y estructurado para observar el efecto del <i>layout</i> y del <i>routing</i> sobre interacciones encadenadas.
<code>qft</code>	Transformada Cuántica de Fourier generada con la utilidad de síntesis de Qiskit.	Transformada cuántica de Fourier, más densa en interacciones y útil como banco de pruebas con mayor presión sobre la conectividad física.
<code>qft_inv</code>	Variante inversa de <code>qft</code> .	Permite evaluar una familia relacionada, pero con orden de operaciones distinto.
<code>random_shallow</code>	Circuito aleatorio de profundidad baja, con semilla reproducible.	Caso sintético para probar robustez con variación estructural ligera y campañas exploratorias.
<code>random_deep</code>	Circuito aleatorio de mayor profundidad, también controlado por semilla.	Caso sintético más exigente para probar robustez ante mayor profundidad, volumen de puertas y coste de <i>routing</i> .
<code>clifford</code>	Circuito generado a partir de un operador Clifford aleatorio.	Familia útil para experimentos de síntesis Clifford y para las ampliaciones relacionadas con el modo <code>synthesis</code> de <code>rl_module</code> .

Tabla 5.2. Circuitos disponibles en la biblioteca interna de `qiskit_interface`.

La Tabla 5.2 muestra que la biblioteca no pretende ser un catálogo exhaustivo de algoritmos cuánticos, sino un conjunto controlado de familias con propiedades distintas. En concreto, `ghz` representa un circuito de entrelazamiento simple y estructurado; `qft` y `qft_inv` representan la transformada cuántica de Fourier y su inversa, con una densidad mayor de interacciones; los circuitos aleatorios actúan como casos sintéticos para probar robustez bajo estructuras no diseñadas manualmente; y `clifford` proporciona una familia útil para experimentos de síntesis Clifford. Esta diversidad permite probar el sistema con circuitos de distinta dificultad sin cambiar la interfaz de entrada.

Las Figuras 5.2, 5.3 y 5.4 muestran tres ejemplos generados desde esa biblioteca interna con cinco cúbits. Su función dentro de la memoria es hacer visible la diferencia estructural entre una cadena de entrelazamiento, un circuito con interacciones controladas densas y un objetivo Clifford usado como familia compatible con la extensión de síntesis.

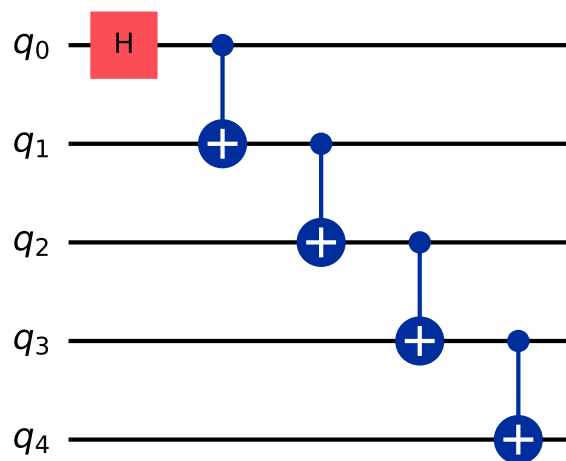


Figura 5.2. Circuito `ghz` de cinco cúbits generado por la biblioteca interna de `qiskit_interface`.

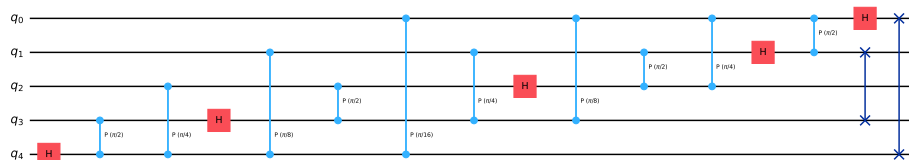


Figura 5.3. Circuito `qft` de cinco cúbits generado por la biblioteca interna de `qiskit_interface`.

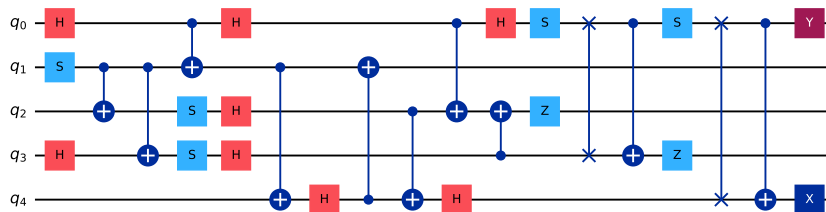


Figura 5.4. Circuito *clifford* de cinco cúbits generado con semilla 42 por la biblioteca interna de *qiskit_interface*.

Como ya se comentó al describir la estructura del módulo, `qiskit_interface` también ofrece soporte de carga y exportación en OpenQASM. En la ruta de fichero, `load_circuit` acepta una entrada `qasm_file`. Después detecta automáticamente si el contenido corresponde a QASM 2 o QASM 3 cuando se usa el modo `auto`, y delega la lectura en las funciones específicas de Qiskit. Esta capacidad no sustituye a la biblioteca interna en las campañas experimentales controladas, pero proporciona una entrada más flexible para experimentos concretos y para reutilizar circuitos definidos fuera del proyecto.

En conjunto, esta sección del módulo actúa como punto de normalización. El resto de componentes no necesita saber si el circuito procede de una familia generada, de un fichero QASM o de una cadena serializada: recibe un `QuantumCircuit` válido, con metadatos de procedencia cuando la ruta lo permite, y puede concentrarse en su propia responsabilidad. Esa separación es especialmente importante para mantener comparables los escenarios, ya que evita que diferencias accidentales en la creación del circuito se mezclen con el efecto del *layout*, del *routing* o de la transpilación.

5.3 Abstracción de *backends* y propiedades *hardware*

Tras la carga y normalización de circuitos, la segunda responsabilidad de `qiskit_interface` es convertir la descripción de un *backend* de Qiskit en una estructura estable para el resto del proyecto. En Qiskit, la información *hardware* aparece distribuida entre objetos como el *backend*, su *target*, el *coupling_map* y las propiedades asociadas a operaciones concretas. Esa representación es adecuada para el transpilador, pero resulta incómoda como contrato entre módulos. Por ello, el bloque de información de *backend* centraliza la extracción de esos datos y los agrupa en el objeto `BackendInfo`.

El catálogo público del módulo se limita a tres *backends* simulados de la familia de *fake backends*: Torino, Sherbrooke y Brisbane. Esta decisión mantiene el entorno reproducible y evita depender de credenciales, disponibilidad de dispositivos reales o calibraciones cambiantes. La función `get_backend` actúa como punto de entrada: normaliza el nombre solicitado, comprueba que pertenezca al registro

interno e instancia el *fake backend* correspondiente. A partir de ahí, la función de extracción construye la descripción *hardware* que consumen los demás módulos.

La parte topológica se representa mediante dos vistas complementarias. Por un lado, se conserva el objeto `coupling_map` de Qiskit, útil cuando una función necesita trabajar con la infraestructura propia del SDK. Por otro, se expone `coupling_edges` como lista de tuplas (`qubit_i`, `qubit_j`). Esta segunda forma es más directa para `mo_module` e `integration`, que necesitan construir grafos, subgrafos de *routing* o espacios de búsqueda de *layouts* sin depender de métodos internos de Qiskit. La Figura 3.4, presentada en el capítulo de tecnologías, muestra visualmente las topologías de los *fake backends* principales y ayuda a interpretar por qué esta lista de aristas es un contrato central. La separación entre ambas vistas evita conversiones repetidas y hace explícita la frontera entre el modelo *hardware* y los algoritmos del proyecto.

Las puertas nativas se extraen desde `backend.target`, que es la vía utilizada por Qiskit 2.x para describir las operaciones soportadas por un *backend*. El módulo filtra operaciones que no son puertas de cómputo, como medidas, reinicios, retardos o control de flujo, y devuelve el conjunto resultante como `basis_gates`. Ese conjunto describe las operaciones disponibles; la puerta nativa de dos cúbits es un dato derivado distinto, obtenido al inspeccionar en el `target` las operaciones con dos argumentos. Así, el sistema puede caracterizar *backends* cuya primitiva de interacción sea `cz`, `ecr` u otra puerta futura, sin asumir que toda conectividad física se expresa mediante `cx`.

Además de topología y puertas nativas, `BackendInfo` almacena propiedades cuantitativas del *hardware* simulado. Cuando el *backend* proporciona estos datos, se extraen; si no están disponibles, el campo queda vacío o nulo para mantener la compatibilidad. Para cada cúbit se registran `T1`, `T2` y frecuencia: `T1` es el tiempo de relajación del cúbit, `T2` es el tiempo de coherencia o decoherencia de fase, y la frecuencia describe la frecuencia física asociada al cúbit. Para las operaciones se recogen tasas de error y duraciones, que actúan como estimadores de calidad y coste temporal de cada puerta. En el caso de las puertas de un cúbit, el módulo selecciona una puerta de referencia disponible, dando prioridad a primitivas habituales como `sx`, `x`, `rz` o `id`. Para las puertas de dos cúbits, utiliza la primitiva detectada dinámicamente. El resultado se guarda en diccionarios indexados por cúbit o por arista física, lo que facilita calcular resúmenes y comparar *layouts* sobre el mismo *backend*.

El contrato también contempla *backends* con una interfaz más ligera, como los *backends* sintéticos que pueden aparecer en capas de integración o en experimentos derivados. En esos casos, el módulo sigue un criterio de compatibilidad por atributos: si existe `target`, se usa la ruta completa de Qiskit; si no existe, se toman campos como `basis_gates`, `coupling_map`, `backend_kind` o

`topology_metadata` cuando están presentes, y las propiedades no disponibles se dejan vacías o nulas. Esta estrategia permite reutilizar la misma estructura `BackendInfo` tanto para *fake backends* como para topologías derivadas, sin presentar estas últimas como dispositivos físicos reales.

Finalmente, `backend_info.py` ofrece utilidades de apoyo sobre la información extraída. Una primera utilidad selecciona un conjunto conexo de cúbits físicos mediante una expansión desde un nodo de alto grado, útil como heurística inicial. Otra calcula estadísticas agregadas para un *layout* concreto, como error medio de puertas de un cúbit, error medio y máximo de puertas de dos cúbits, T_1/T_2 medios y número de aristas disponibles entre los cúbits seleccionados. En el diseño actual, estas estadísticas sirven como apoyo de diagnóstico y comparación, especialmente en torno a `mo_module`, pero no convierten a `qiskit_interface` en un optimizador de *layouts*. El módulo se mantiene como proveedor de información *hardware* normalizada.

5.4 Métricas de circuitos y transpilación *baseline*

La tercera responsabilidad de `qiskit_interface` es proporcionar una medida homogénea del coste de un circuito antes y después de la transpilación. Sin una base común, comparar salidas de `MO_Only`, `RL_Only` o *layouts* triviales sería engañoso, porque cada *backend* puede expresar su conectividad mediante puertas nativas distintas y el transpilador puede introducir descomposiciones o *SWAPs* adicionales. Por eso, `circuit_utils.py` y `transpiler.py` se entienden como un binomio: el primero extrae métricas del circuito original o transpilado, y el segundo ejecuta la línea base estándar de Qiskit y empaqueta el resultado en una estructura serializable para análisis tabular [14, 11]. La Tabla 5.3 resume las métricas más relevantes.

La función `extract_metrics` concentra esa extracción sobre un `QuantumCircuit`. La profundidad se toma con `depth()`, el volumen total con `size()` y el desglose por tipo con `count_ops()`, de acuerdo con la API oficial de Qiskit [14]. Sobre esa base, `two_qubit_gates` cuenta operaciones que actúan sobre dos o más cúbits, `nonlocal_gates` conserva el criterio nativo de Qiskit para puertas no locales y `cnot_equivalent` convierte el circuito a una unidad comparable basada en CNOTs. La combinación evita que el coste dependa solo del nombre de la puerta nativa del *backend* y no de su impacto real sobre el circuito. Además, `active_qubits` diferencia entre la anchura formal del circuito y los cúbits que realmente participan en alguna instrucción, algo útil cuando el registro físico es más amplio que el subcircuito usado.

Sobre esa capa de métricas, `transpiler.py` implementa la línea base estándar de Qiskit. La función `transpile_circuit` usa un `preset_pass_manager` de Qiskit. A partir de ahí, registra el circuito original, el circuito transpilado, las métricas antes y después, el *backend*, el nivel de opti-

Métrica		Qué resume	Por qué importa
<code>depth</code>		Longitud del camino crítico del circuito.	Aproxima la latencia lógica y la exposición a decoherencia.
<code>total_gates</code> <code>gate_counts</code>	y	Volumen total de operaciones y desglose por tipo de puerta.	Permite comparar complejidad estructural más allá de la profundidad.
<code>two_qubit_gates</code> <code>nonlocal_gates</code>	y	Carga de operaciones multi-cúbit.	Captura la parte más sensible al <i>routing</i> y a las tasas de error.
<code>cnot_equivalent</code>		Coste normalizado en unidades de CNOT.	Hace comparables circuitos que usan puertas nativas distintas.
<code>width</code> , <code>num_clbits</code> <code>active_qubits</code>	y	Huella total del circuito y cúbits realmente utilizados.	Distingue el tamaño del registro de la actividad efectiva del programa.

Tabla 5.3. Interpretación de las métricas básicas extraídas por *extract_metrics*.

mización, la semilla y los *layouts* que Qiskit resuelve durante la compilación. El flujo sigue la estructura documentada por Qiskit para las etapas de *layout*, *routing*, *translation* y *optimization*, y trabaja con los niveles de optimización 0–3 como escala de referencia reproducible [11, 17]. El diseño es deliberado: esta capa compara transpilaciones, pero no ejecuta circuitos ni envía trabajos a *hardware* real.

Las funciones auxiliares completan esa línea base. `transpile_all_levels` repite el mismo circuito y *backend* para aislar el efecto del nivel de optimización; `run_baseline` extiende la comparación a varios *backends* simulados y prepara salidas tabulares; y `run_named_baseline` encapsula configuraciones reutilizables. A nivel interpretativo, `TranspilationResult` añade dos indicadores derivados útiles en la memoria: `depth_reduction`, que resume la reducción relativa de profundidad, y `two_qubit_gate_overhead`, que mide el incremento de puertas multi-cúbit respecto al circuito original.

En conjunto, esta sección fija el punto de comparación común sobre el que se apoyan los capítulos posteriores. Primero se cuantifica el circuito; después se aplica la línea base de Qiskit; y, a partir de ahí, las soluciones de `mo_module` y `rl_module` pueden evaluarse con el mismo lenguaje de métricas. La siguiente sección reutiliza precisamente esa base para valorar *layouts* suministrados desde el exterior.

5.5 Evaluación con *layouts* externos

La cuarta responsabilidad de `qiskit_interface` es evaluar un *layout* que no ha sido elegido por el propio transpilador, sino por una capa externa. Esta entrada permite comparar, con el mismo flujo de Qiskit, una disposición trivial, una heurística clásica, un *layout* seleccionado por `mo_module` o cualquier

otra propuesta que llegue desde *integration*. Como ya se definió en el capítulo anterior, el contrato compartido expresa en cada posición lógica el cúbit físico asignado. En esta sección lo relevante no es redefinir ese convenio, sino explicar cómo `qiskit_interface` lo valida y lo entrega a Qiskit para medir el efecto de una propuesta externa. La Tabla 5.4 recoge esas comprobaciones.

Validación	Qué impone	Motivo
Longitud del <i>layout</i>	Debe coincidir con el número de qubits del circuito.	Evita ambigüedades entre posiciones lógicas y qubits físicos.
Unicidad de qubits	Ningún qubit físico puede repetirse en la lista.	Mantiene el mapeo como una asignación uno a uno.
Rango del <i>backend</i>	Todos los índices deben existir en el <i>backend</i> resuelto.	Impide <i>layouts</i> que no son compatibles con el dispositivo o topología elegida.

Tabla 5.4. Comprobaciones básicas aplicadas a un *layout* externo antes de evaluarlo.

Qiskit toma ese `initial_layout` como una permutación inicial y, si el circuito lo necesita, aplica *routing* posterior para adaptarlo a la conectividad del *backend*. La representación interna del resultado conserva esa información mediante `TranspileLayout`: la disposición inicial y la disposición final quedan disponibles para inspección, por lo que la comparación incorpora trazabilidad sobre cómo se llegó al circuito transpileado [12, 16, 11, 17]. En esta capa, `transpile_with_custom_layout` solo actúa como envoltorio. Resuelve el *backend*, valida el vector de entrada y delega la compilación en `transpile_circuit`, sin forzar un *layout* trivial.

El resultado se devuelve como `TranspilationResult`, con el mismo conjunto de métricas que la *baseline* estándar. Eso incluye el circuito original y el transpileado, las métricas pre y post, el *layout* inicial suministrado, el `qiskit_initial_layout` resuelto durante la compilación y el `final_layout` obtenido tras *routing*. Así, *integration* puede comparar `MO_Only` con referencias simples. En la práctica, `qiskit_interface` mide el impacto; la selección del candidato queda fuera, en la capa que orquesta el experimento.

La Figura 5.5 muestra esta responsabilidad como una secuencia de llamadas. El llamador entrega un *layout* bajo el contrato compartido, `qiskit_interface` valida que sea factible para el circuito y el *backend*, y la transpilación queda delegada en el mismo flujo que produce la *baseline*. De este modo, un *layout* procedente de `mo_module` no recibe un tratamiento especial: se evalúa con la misma estructura de métricas que cualquier otra propuesta externa.

La siguiente sección aplica el mismo principio al caso en el que el *routing* también llega desde

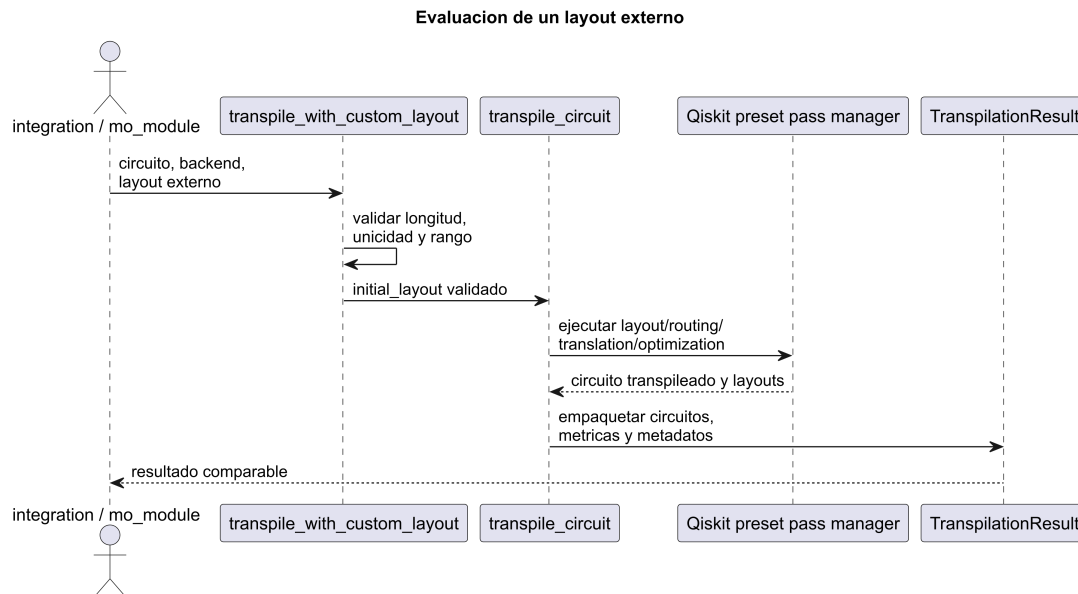


Figura 5.5. Secuencia de evaluación de un layout externo en *qiskit_interface*.

fuera y Qiskit solo completa las fases posteriores.

5.6 Evaluación con *routing* externo

La quinta responsabilidad de *qiskit_interface* es evaluar un circuito cuyo *routing* ya ha sido decidido fuera del transpilador. En este caso la entrada no es una propuesta de *layout*, sino un circuito que ya incorpora la secuencia de intercambios necesaria para respetar la conectividad del *backend*. La función `transpile_post_routing` asume precisamente ese escenario: construye un `preset pass manager`, desactiva las fases de *layout* y *routing* y deja a Qiskit solo las etapas de *translation* y *optimization*, siguiendo la estructura general del proceso de transpilación documentado por Qiskit [11, 17]. La Tabla 5.5 resume las entradas que preservan la trazabilidad de esta ruta.

Elemento	Papel en la evaluación
<code>routed_circuit</code>	Circuito de entrada sobre el que se ejecuta la fase final de Qiskit; se asume que el enrutado ya está materializado.
<code>reference_circuit</code>	Circuito de referencia para calcular las métricas originales cuando la entrada procede de una reconstrucción externa.
<code>initial_layout</code> <code>final_layout</code>	Metadatos opcionales para mantener la trazabilidad del experimento.
<code>optimization_level</code>	Nivel de optimización aplicado tras el <i>routing</i> , comparable con la línea base.

Tabla 5.5. Entradas relevantes de la evaluación con *routing* externo.

Cuando `integration` reconstruye un circuito a partir de una traza RL completa, esta función

le permite recuperar una salida comparable con el resto del proyecto sin volver a decidir *swaps* ni posiciones. El resultado vuelve a ser un `TranspilationResult` con el circuito original, el circuito transpilado y las métricas pre y post, de modo que `RL_Only` y la fase ruteada de `MO+RL` pueden compararse con `Baseline` y `MO_Only` en el mismo espacio de medidas. La diferencia, aquí, es estrictamente de responsabilidad: `qiskit_interface` mide y normaliza la transpilación final, mientras que la decisión de *routing* permanece fuera.

6

Desarrollo del módulo `mo_module`

6.1 Estructura general del módulo

6.1.1 Visión global

`mo_module` actúa como la capa de búsqueda multiobjetivo del sistema: recibe el circuito y el *backend* desde `qiskit_interface`, explora *layouts* candidatos y devuelve un frente de Pareto o una solución de compromiso para *integration*. La Figura 6.1 muestra esa frontera funcional y distingue los bloques internos del módulo de las capas que proporcionan la información de entrada o consumen el resultado.

6.1.2 Bloques funcionales y contratos

La Tabla 6.1 resume los bloques del módulo y los símbolos que forman su superficie pública.

La separación de bloques simplifica el análisis y permite reutilizar el módulo desde *integration* y las utilidades de banco de pruebas.

6.1.3 Modelo estático del optimizador

La Figura 6.2 muestra el modelo estático propio de `mo_module`. `LayoutSearchSpace` define el dominio, `FitnessEvaluator` agrupa las métricas activas, `LayoutOptimizationProblem` adapta la evaluación al optimizador y `OptimizationResult` conserva el frente de Pareto.

`TranspilationCache` es una parte relevante de este modelo porque evita recalcular el mismo

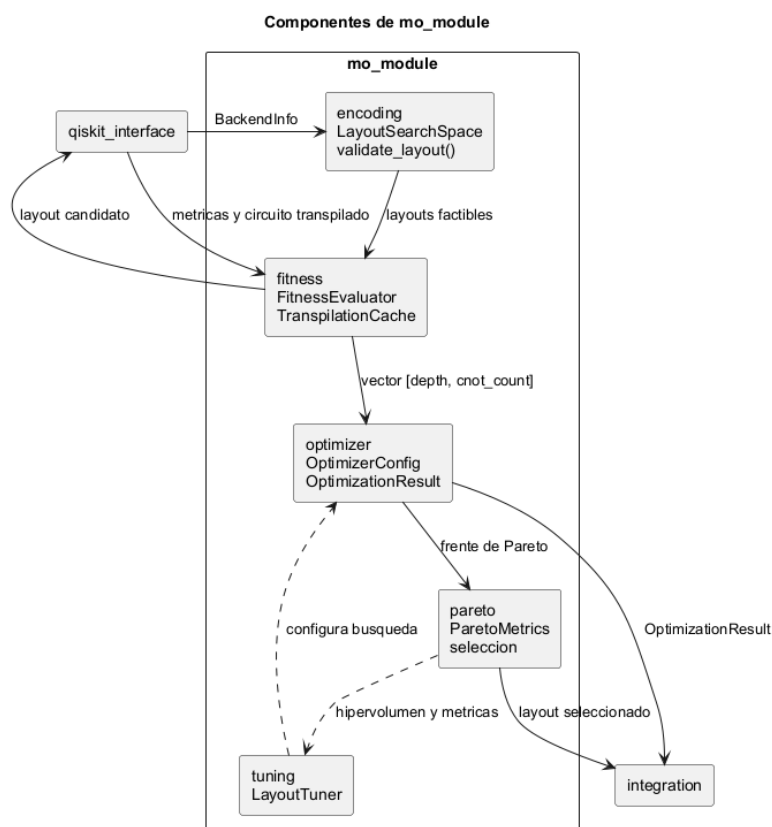


Figura 6.1. Vista de componentes y fronteras funcionales de *mo_module*.

Bloque	Responsabilidad	Contratos o símbolos principales
encoding	Codificar el <i>layout</i> como permutación parcial y generar individuos válidos.	LayoutSearchSpace, validate_layout, repair_layout
fitness	Evaluar el efecto real de cada <i>layout</i> tras la transpilación.	FitnessFunction, FitnessEvaluator, TranspilationCache
optimizer	Orquestar la búsqueda evolutiva y empaquetar el frente de Pareto.	OptimizerConfig, LayoutOptimizationProblem, OptimizationResult
pareto	Analizar el frente y seleccionar una solución concreta cuando hace falta.	ParetoMetrics, select_knee_point, select_weighted
tuning	Ajustar hiperparámetros y fijar el punto de referencia de sesión.	HyperparameterSpace, LayoutTuner, session_ref_point

Tabla 6.1. Bloques funcionales de *mo_module* y contratos que fijan su superficie pública.

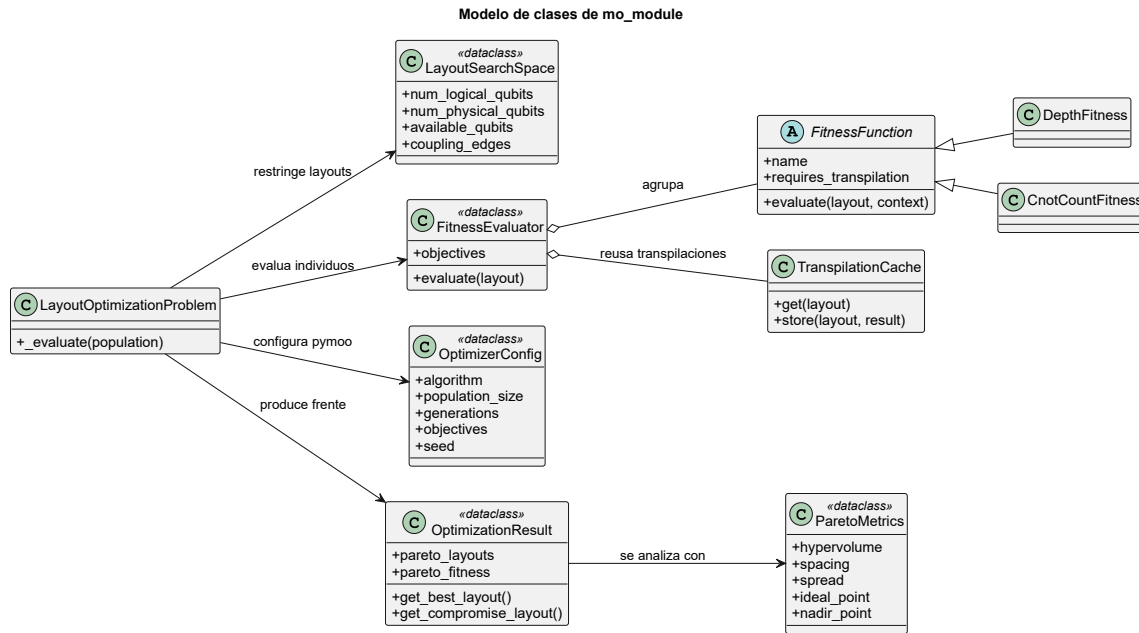


Figura 6.2. Modelo UML de clases de los contratos principales de *mo_module*.

circuito cuando varias funciones objetivo comparten individuo. La caché almacena resultados indexados por la tupla de cúbits físicos del *layout*, tiene un límite de 512 entradas y aplica una política LRU: cuando se supera ese límite, expulsa el resultado usado hace más tiempo. Esta decisión reduce transpilaciones repetidas durante una ejecución evolutiva sin convertir la caché en una estructura de crecimiento ilimitado.

6.1.4 Flujo de optimización multiobjetivo

La Figura 6.3 resume la cadena operativa desde *BackendInfo* hasta la selección del *layout* de compromiso: espacio de búsqueda, evaluación de fitness y optimización evolutiva. Este flujo distingue la generación de candidatos de la decisión final del *layout*, que es la que consume *integration*.

6.2 Formulación del problema y representación del *layout*

La optimización de *mo_module* se formula como un problema combinatorio de asignación entre cúbits lógicos y físicos. Sobre el contrato compartido definido en el Capítulo 4, para un circuito con n cúbits lógicos ejecutado sobre un *backend* con N cúbits físicos, el módulo explora una permutación parcial de longitud n : selecciona n cúbits físicos distintos entre los disponibles y los asigna a las posiciones lógicas del circuito.

Esta representación es deliberadamente posicional. Dos *layouts* con el mismo conjunto de cúbits físicos, pero con los valores intercambiados entre posiciones, son candidatos distintos porque cada

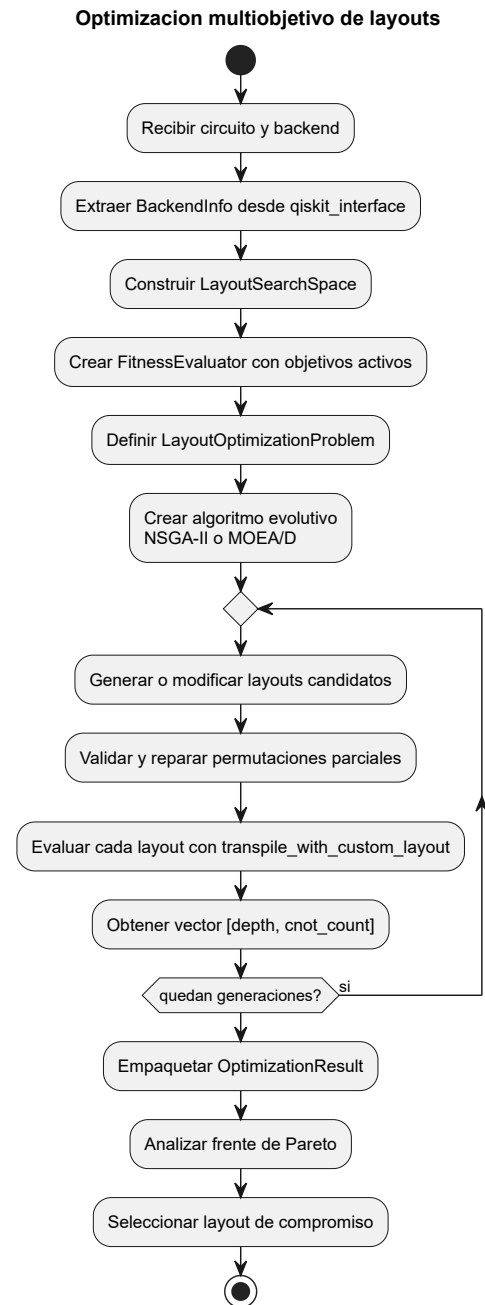


Figura 6.3. Actividad principal de optimización multiobjetivo de layouts en *mo_module*.

Regla	Exigencia	Motivo
Longitud	El vector debe tener exactamente n posiciones.	Cada cúbit lógico necesita una asignación física única.
Rango	Todo valor debe pertenecer a <code>available_qubits</code> .	Evita referencias fuera del <i>backend</i> seleccionado.
Unicidad	Ningún cúbit físico puede repetirse.	Mantiene el mapeo como una asignación inyectiva.

Tabla 6.2. Reglas básicas de factibilidad aplicadas a un layout candidato.

posición del vector corresponde a un cúbit lógico concreto. Esta decisión simplifica la evaluación del espacio de búsqueda y mantiene la compatibilidad con la interfaz que consumen `qiskit_interface` e `integration`, donde el *layout* se interpreta como una asignación directa y no como una estructura relacional más ambigua. La Figura 6.4 resume el recorrido de validación y reparación, mientras que la Tabla 6.2 sintetiza las reglas mínimas que debe cumplir todo candidato.

El espacio factible se encapsula en `LayoutSearchSpace`. Este contrato agrupa el número de cúbits lógicos, el número de cúbits físicos, el conjunto de cúbits disponibles y las aristas del `coupling_map` del *backend*. La construcción `LayoutSearchSpace.from_backend_info()` rechaza de forma explícita los casos en los que el circuito tiene más cúbits lógicos que cúbits físicos disponibles en el *backend*, ya que en esa situación no existe una asignación inyectiva válida. En el caso habitual, `available_qubits` coincide con el conjunto completo de cúbits del *backend*, aunque la abstracción ya deja abierta la puerta a excluir cúbits si en el futuro se trabaja con topologías sintéticas o con recursos defectuosos.

La factibilidad del *layout* se verifica con `validate_layout()`. La función exige que el vector tenga la longitud correcta, que todos sus elementos pertenezcan a `available_qubits` y que no existan cúbits físicos repetidos. Por tanto, el problema se modela como una asignación inyectiva, no como un vector entero genérico. La conectividad del *backend* se conserva como contexto mediante `coupling_edges`, pero no se impone todavía como una restricción dura sobre el *layout*. Esa compatibilidad con la topología física se evalúa más adelante, cuando el candidato se transpila y se puntúa según su efecto real sobre el circuito.

Como los operadores evolutivos pueden producir individuos con duplicados o con huecos, es necesario un mecanismo de corrección posterior al cruce o a la mutación. `repair_layout()` conserva la primera aparición válida de cada cúbit físico, descarta repeticiones y rellena las posiciones vacías

Validacion y reparacion de layouts

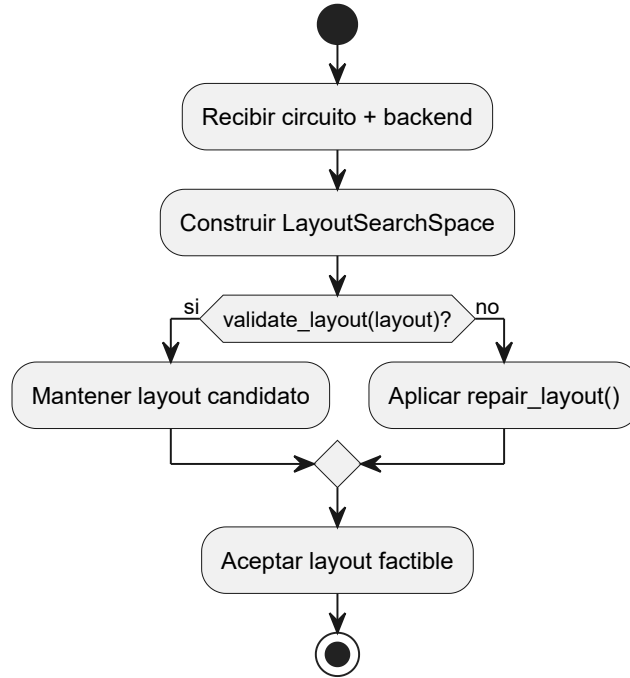


Figura 6.4. Flujo de validación y reparación de un layout candidato en *mo_module*.

con cúbits aún no utilizados en el *backend*. En la implementación actual, esos cúbits disponibles se barajan con un generador pseudoaleatorio y se insertan en ese orden, sin aplicar todavía un criterio *hardware-aware*; cuando el operador recibe una semilla, el procedimiento sigue siendo reproducible. Así, el algoritmo puede explorar recombinaciones agresivas sin abandonar el dominio factible, y el resultado siempre mantiene el contrato esperado por la capa de evaluación. En la práctica, esta formulación reduce la búsqueda a un espacio combinatorio estructurado, donde el objetivo no es encontrar cualquier disposición de cúbits, sino una disposición inyectiva que favorezca la transpilación posterior.

6.3 Objetivos y evaluación de fitness

mo_module no evalúa un *layout* como una propiedad aislada de la topología, sino como una decisión que modifica el resultado de la transpilación. Cada individuo generado por el algoritmo evolutivo se interpreta como una asignación lógica-física válida y se transforma en un vector de fitness. Ese vector resume, para los objetivos activos, qué coste tendría usar dicho *layout* como punto de partida del flujo de Qiskit. De esta forma, la búsqueda multiobjetivo no intenta encontrar un único criterio absoluto de calidad, sino aproximar un conjunto de soluciones no dominadas que equilibran varias medidas relevantes para la ejecución final del circuito.

El primer objetivo activo es `depth`, implementado mediante `DepthFitness`. La profundidad

del circuito representa el número mínimo de capas secuenciales necesarias para ejecutar sus operaciones, equivalente al camino más largo en el grafo dirigido de dependencias del circuito. Reducirla es importante porque un circuito más corto en términos de capas secuenciales requiere menos pasos dependientes y, en un dispositivo real, tiende a exponerse durante menos tiempo a errores acumulados y decoherencia. En el contexto del módulo, `depth` no mide la profundidad del circuito original ni una estimación topológica previa, sino la profundidad obtenida tras aplicar el *layout* al *backend* seleccionado y ejecutar la transpilación correspondiente.

El segundo objetivo activo es `cnot_count`, implementado mediante `CnotCountFitness`. Esta métrica utiliza el coste equivalente en puertas CNOT del circuito transpilado, no el número de CNOTs presentes en el circuito original. De este modo captura el impacto que tiene el *layout* sobre las operaciones de dos cúbits introducidas o conservadas durante el proceso de adaptación al *hardware*. Su interés metodológico se debe a que las puertas multicúbit son más costosas y más sensibles al error que las operaciones de un solo cúbit. Además, como las puertas multicúbit requieren conectividad física entre los cúbits afectados, cuando esa conectividad no existe directamente es necesario introducir *routing* adicional. Por ello, un *layout* que reduzca esa necesidad puede mejorar el coste final del circuito aunque no sea necesariamente el mejor respecto a profundidad.

Ambos objetivos se minimizan. La configuración de objetivos por defecto del módulo es, por tanto, el par `["depth", "cnot_count"]`, y cada evaluación produce un vector numérico con esos dos componentes. Esta formulación conserva la naturaleza multiobjetivo del problema: un candidato puede reducir la profundidad a costa de aumentar el número equivalente de CNOTs, o al contrario. En lugar de forzar una combinación escalar desde el principio, el optimizador trabaja con esos valores separados y deja que el análisis del frente de Pareto determine qué *layouts* representan compromisos razonables.

La evaluación está basada en transpilación. Para cada *layout*, `FitnessEvaluator` delega la transpilación en `qiskit_interface`; la llamada concreta es `transpile_with_custom_layout()`. A partir de ese resultado, recupera las métricas del circuito transpilado y las entrega a las funciones de fitness activas. Esta decisión hace que el candidato se valore por su efecto real dentro del flujo de Qiskit, no solo por propiedades indirectas como distancia media entre cúbits conectados o cercanía al `coupling_map`. La topología del *backend* sigue siendo fundamental para construir el espacio de búsqueda, pero la puntuación final depende de cómo Qiskit materializa esa asignación en un circuito concreto.

Desde el punto de vista de diseño, `FitnessEvaluator` actúa como un evaluador compuesto:

agrupa las funciones objetivo configuradas y devuelve el vector que consume el algoritmo evolutivo. Cuando varios objetivos necesitan la misma transpilación, `TranspilationCache` evita repetir trabajo para un *layout* ya evaluado, almacenando el resultado asociado a la tupla de cúbits físicos. La caché está acotada a 512 entradas y usa reemplazo LRU, por lo que conserva los resultados recientes y expulsa los menos utilizados cuando se llena. Esta política es especialmente útil en un algoritmo evolutivo, donde pueden reaparecer individuos por selección, cruce o mutación, y donde cada transpilación tiene un coste sensiblemente mayor que una métrica puramente algebraica.

El módulo conserva además una frontera clara entre objetivos activos e información diagnóstica. Las propiedades *hardware*, los errores de puertas o la conectividad detallada pueden utilizarse para análisis, comparación de *layouts* o futuras ampliaciones, pero no forman parte de la configuración activa de fitness. En la versión descrita, `mo_module` optimiza *layouts* a partir de profundidad y coste equivalente de CNOTs tras transpilación; la selección posterior de una solución concreta del frente queda separada y se aborda en las secciones siguientes.

6.4 Algoritmos evolutivos y operadores especializados

`mo_module` resuelve la búsqueda de *layouts* mediante algoritmos evolutivos multiobjetivo. Esta elección encaja con la naturaleza combinatoria del problema: el espacio de soluciones crece rápidamente con el número de cúbits físicos disponibles y no existe una única métrica que ordene todos los candidatos. En lugar de optimizar una función escalar cerrada, el módulo mantiene una población de *layouts*, evalúa cada individuo con los objetivos descritos en la sección anterior y aplica operadores de variación para explorar nuevas asignaciones. El resultado de la ejecución es un conjunto de soluciones no dominadas que aproxima el frente de Pareto.

La configuración de la búsqueda se concentra en la clase `OptimizerConfig`. Este contrato selecciona el algoritmo evolutivo, el tamaño de población, el número de generaciones, los objetivos activos, el nivel de optimización de Qiskit usado durante la evaluación, el operador de cruce, las probabilidades de mutación y la semilla. La clase `LayoutOptimizationProblem` adapta el problema al formato esperado por `pymoo`, la biblioteca utilizada para implementar los algoritmos de optimización multiobjetivo: cada individuo es un vector entero de longitud igual al número de cúbits lógicos y la evaluación devuelve el vector de fitness calculado por `FitnessEvaluator`. Finalmente, `create_algorithm()` instancia el algoritmo concreto y le inyecta los operadores especializados del módulo. Esta separación permite cambiar entre estrategias evolutivas sin alterar la representación del *layout* ni la evaluación de fitness.

El algoritmo principal es **NSGA-II**. Su interés para el proyecto procede de dos mecanismos comple-

Operador	Acción principal	Motivo
<code>LayoutSampling</code>	Selecciona cúbits físicos sin reemplazo.	Inicia la población con <i>layouts</i> factibles.
<code>DPXCrossover</code>	Conserva asignaciones comunes y rellena posiciones libres.	Recombina coincidencias posicionales entre padres.
<code>LayoutCrossover</code>	Aplica <i>Order Crossover</i> sobre permutaciones parciales.	Mantiene una alternativa de cruce compatible.
<code>LayoutMutation</code>	Intercambia posiciones o reemplaza por cúbits no usados.	Combina intensificación y exploración.
<code>repair_layout()</code>	Corrige duplicados o huecos tras la variación.	Preserva el contrato de <i>layout</i> válido.

Tabla 6.3. Operadores evolutivos especializados empleados por *mo_module*.

mentarios: la ordenación por no dominancia, que separa las soluciones en frentes según las relaciones de Pareto, y la distancia de *crowding*, que favorece la diversidad dentro de un mismo frente. Esta combinación resulta adecuada cuando se quiere evitar que la población converja prematuramente hacia una sola zona del espacio de objetivos. En la implementación, **NSGA-II** se usa a través de **pymoo**, que proporciona la infraestructura evolutiva general y permite sustituir los operadores genéricos por operadores propios para *layouts* [4, 3].

Como alternativa, el módulo soporta **MOEA/D**. A diferencia de **NSGA-II**, que trabaja directamente con frentes de no dominancia, **MOEA/D** descompone el problema multiobjetivo en subproblemas escalares asociados a direcciones o vectores de referencia [30]. En el código, esas direcciones se generan automáticamente mediante `get_reference_directions("uniform", ...)`, y cada subproblema coopera con una vecindad de soluciones. Esta opción amplía el repertorio experimental del módulo y permite comparar un enfoque basado en dominancia con otro basado en descomposición, manteniendo constantes la codificación del *layout* y las funciones de fitness.

El punto delicado es definir operadores compatibles con la representación. Un *layout* no es un vector entero arbitrario: es una permutación parcial donde cada posición lógica debe contener un cúbit físico válido y no repetido. Si se aplicasen operadores genéricos, sería fácil producir individuos con duplicados, referencias fuera del *backend* o asignaciones que rompen el contrato posicional compartido. Por eso, *mo_module* incorpora operadores propios para muestreo, cruce, mutación y reparación, resumidos en la Tabla 6.3.

El muestreo inicial utiliza selección sin reemplazo sobre el conjunto de cúbits disponibles. Esto produce *layouts* válidos sin necesitar una fase de filtrado posterior. En el cruce, el operador por defecto es `DPXCrossover`; su comportamiento se centra en conservar las posiciones donde ambos padres asignan el mismo cúbit físico al mismo cúbit lógico y completar el resto con información del padre donante. Por ejemplo, si un padre representa $[0, 3, 5, 7]$ y otro representa $[0, 4, 5, 8]$, el cruce conserva las posiciones comunes 0 y 5; las posiciones restantes se rellenan con cúbits del padre donante que no estén ya usados, obteniendo un hijo factible como $[0, 4, 5, 7]$ o $[0, 3, 5, 8]$ según el donante. La alternativa `LayoutCrossover` implementa un cruce tipo *Order Crossover*: copia un segmento de un padre y rellena el resto con valores del otro padre que no introduzcan duplicados. En ambos casos, si aparece una inconsistencia, la reparación restaura la factibilidad antes de que el individuo pase a evaluación.

La mutación se divide en dos operaciones independientes. La mutación por intercambio altera la correspondencia lógico-física entre dos posiciones del mismo *layout* sin cambiar el subconjunto de cúbits físicos seleccionado. Con $[0, 3, 5, 7]$, intercambiar las posiciones 1 y 3 produce $[0, 7, 5, 3]$, de modo que se conserva el mismo conjunto físico pero cambia qué cúbit lógico ocupa cada ubicación. La mutación por reemplazo cambia una posición por un cúbit físico que aún no aparece en el individuo; por ejemplo, si el cúbit físico 9 está disponible, $[0, 3, 5, 7]$ puede pasar a $[0, 9, 5, 7]$. Las probabilidades asociadas a estas mutaciones se guardan en la configuración como categorías discretas, lo que facilita comparar experimentos y reutilizar configuraciones en bancos de pruebas o sesiones de ajuste de hiperparámetros.

Con este diseño, la búsqueda evolutiva queda acoplada al dominio sin perder flexibilidad experimental. `NSGA-II` y `MOEA/D` proporcionan dos políticas de exploración multiobjetivo; los operadores especializados aseguran que las soluciones generadas sigan siendo *layouts* interpretables por `qiskit_interface`; y `OptimizationResult` conserva el frente producido junto con metadatos de ejecución. La decisión de qué solución concreta seleccionar de ese frente no pertenece a los operadores evolutivos, sino al análisis posterior de Pareto que se describe a continuación.

6.5 Análisis del frente de Pareto y selección de soluciones

`mo_module` no termina su trabajo cuando el algoritmo evolutivo devuelve una población final. El resultado útil del proceso es el frente de Pareto contenido en `OptimizationResult`: una colección de *layouts* no dominados junto con sus valores de fitness. En este contexto, no dominado significa que ningún otro *layout* del frente mejora simultáneamente todos los objetivos considerados. Dado que los objetivos activos son `depth` y `cnot_count`, el frente expresa de forma explícita el compromiso entre

Instrumento	Interpretación	Uso en el módulo
Hipervolumen	Área o volumen dominado respecto a un punto de referencia.	Comparar calidad global de frentes y puntuar configuraciones de ajuste de hiperparámetros.
<i>Spacing</i>	Regularidad de las distancias entre soluciones vecinas.	Detectar frentes concentrados o con huecos grandes.
<i>Spread</i>	Amplitud y diversidad del frente.	Valorar si el algoritmo cubre varios compromisos.
Punto ideal y nadir	Mejores y peores valores observados por objetivo dentro del frente.	Normalizar distancias y contextualizar la escala de fitness.
<i>Knee point</i>	Solución situada en una zona de cambio marginal alto.	Recomendar un compromiso cuando no hay preferencias explícitas.
Suma ponderada	Escalarización de los objetivos con pesos configurables.	Seleccionar <i>layouts</i> cuando sí existen prioridades de decisión.

Tabla 6.4. Instrumentos de análisis y selección disponibles para el frente de Pareto de *mo_module*.

circuitos transpilados más poco profundos y circuitos con menor coste equivalente en CNOTs.

El análisis posterior se concentra en `pareto.py`. La función `compute_pareto_metrics()` resume la calidad geométrica del frente mediante `ParetoMetrics`, que incluye hipervolumen, *spacing*, *spread*, punto ideal y punto nadir. El punto ideal recoge el mejor valor observado para cada objetivo, mientras que el punto nadir aproxima el peor valor dentro del frente de Pareto. Estos dos extremos no son necesariamente *layouts* ejecutables por sí mismos, sino referencias para interpretar la escala del frente y normalizar distancias. El hipervolumen mide el volumen dominado por el frente respecto a un punto de referencia peor que las soluciones evaluadas; por tanto, combina cercanía al ideal y extensión del frente en una sola magnitud. *Spacing* y *spread* complementan esa lectura al describir la regularidad y la diversidad de la distribución de soluciones. La Tabla 6.4 resume los instrumentos de análisis y selección usados por el módulo.

La selección de un *layout* único es una decisión metodológica posterior a la optimización. El módulo ofrece varias políticas porque no todas las evaluaciones persiguen el mismo objetivo. La política `select_min_objective()` permite escoger el mejor *layout* en una métrica concreta, por ejemplo la menor profundidad. La política `select_weighted()` aplica una suma ponderada y re-

sulta adecuada cuando el usuario o el experimento fijan una preferencia explícita entre objetivos. La política `select_knee_point()` busca el punto de rodilla, entendido como una solución de compromiso donde pequeñas mejoras en un objetivo tienden a exigir pérdidas relativamente mayores en otro. Esta última opción es especialmente útil como recomendación por defecto, porque evita privilegiar de forma arbitraria uno de los extremos del frente.

En la integración del proyecto, esta reducción del frente a un *layout* concreto se materializa a través de políticas como `compromise` o `best_on_objective`. La primera encaja con la idea de *layout* equilibrado; la segunda permite construir experimentos orientados a una métrica. En ambos casos, se conserva el contrato compartido definido en la arquitectura. Esto mantiene separadas dos fases que conviene no confundir: la optimización multiobjetivo produce diversidad de soluciones no dominadas, mientras que la selección posterior decide qué solución concreta se entrega a `qiskit_interface` o a `integration`.

6.6 Ajuste de hiperparámetros, bancos de pruebas y reproducibilidad

La calidad del frente obtenido depende de decisiones de configuración: algoritmo, tamaño de población, número de generaciones, operador de cruce, probabilidades de mutación y nivel de optimización de Qiskit. Para evitar que esas decisiones queden fijadas solo por prueba manual, `mo_module` incorpora una capa de ajuste de hiperparámetros basada en Optuna [23]. El punto de entrada es `LayoutTuner`, que construye estudios sobre `OptimizerConfig` y evalúa cada configuración a partir del hipervolumen medio obtenido en varias semillas. La semilla no se optimiza; se fija para controlar la reproducibilidad y, cuando procede, se repite la evaluación con varias semillas.

La elección del hipervolumen exige comparar todos los ensayos de una sesión contra el mismo punto de referencia. Si el `ref_point` se recalculase en cada ensayo, los valores medirían áreas dominadas respecto a escalas distintas. Por ello, `LayoutTuner` fija un `session_ref_point`: en modo `manual` lo proporciona el usuario, y en modo `calibrated` se obtiene tras ejecutar configuraciones ancla, observar el peor frente combinado y aplicar un margen conservador del 30 % más un pequeño término numérico.

Esta decisión favorece la comparabilidad, aunque introduce una regla de penalización. Si un frente posterior excede el `session_ref_point`, es decir, si el punto de referencia deja de ser estrictamente peor que el máximo observado en alguna coordenada, ese ensayo recibe hipervolumen cero para ese frente y se registra una advertencia. La sesión continúa, pero la puntuación deja constancia

de que la comparación no sería geométricamente válida bajo el punto fijado. El comportamiento es conservador: prioriza que las puntuaciones de Optuna tengan una interpretación común frente a recalibrar dinámicamente la escala a mitad del estudio.

La reproducibilidad se apoya también en un banco de pruebas estable. Los circuitos por defecto son `ghz_5q`, `qft_4q`, `random_4q_d10` y `clifford_4q`. La selección cubre patrones distintos: entrelazamiento estructurado en GHZ, estructura aritmética en QFT, circuitos aleatorios de profundidad controlada y circuitos Clifford. No pretende agotar el espacio de aplicaciones cuánticas, pero sí proporcionar un conjunto pequeño y repetible con el que detectar diferencias entre configuraciones, operadores y semillas.

Cada ejecución del banco de pruebas compara el comportamiento de `mo_module` sobre los mismos circuitos y *backends*, y puede repetirse con varias semillas. Este punto es relevante porque los algoritmos evolutivos dependen de muestreo inicial, selección y variación estocástica. Una mejora observada en una única semilla podría ser accidental; una tendencia que se mantiene al promediar varias semillas tiene mayor valor experimental. Por eso, el módulo distingue entre el resultado de una ejecución concreta y la estabilidad de una configuración a través de repeticiones.

La interfaz de banco de pruebas y ajuste de hiperparámetros debe entenderse como una extensión de la línea base creada durante el desarrollo para probar temporalmente el funcionamiento del módulo. Su finalidad principal fue facilitar la inspección del progreso de calibración y ajuste, no definir una interfaz final de usuario ni desplazar las responsabilidades centrales de `mo_module`. Durante la calibración se comunican eventos de inicio, avance y finalización con el `ref_point` candidato o definitivo. Durante los ensayos se informa de la puntuación obtenida, la mejor puntuación acumulada y los parámetros sugeridos. Esta trazabilidad no cambia la lógica de optimización, pero permitió validar de forma más cómoda por qué una configuración fue seleccionada y repetir el estudio bajo las mismas condiciones. La Figura 6.5 muestra una captura de esta interfaz.

6.7 Limitaciones actuales y aportación del módulo

El alcance de `mo_module` es deliberadamente acotado. Su responsabilidad es generar *layouts* candidatos, evaluarlos mediante transpilación y organizar las soluciones no dominadas en un frente de Pareto. A partir de ese frente puede devolver un *layout* de compromiso o el mejor *layout* según un objetivo concreto, pero no ejecuta entrenamiento de aprendizaje por refuerzo ni decide por sí mismo el flujo completo MO+RL. El traspaso de un *layout* de MO hacia RL pertenece a *integration*, que es la capa encargada de orquestar escenarios, campañas y comparabilidad entre módulos.

Tampoco reimplementa la lógica de `qiskit_interface`. El módulo depende de esta capa para

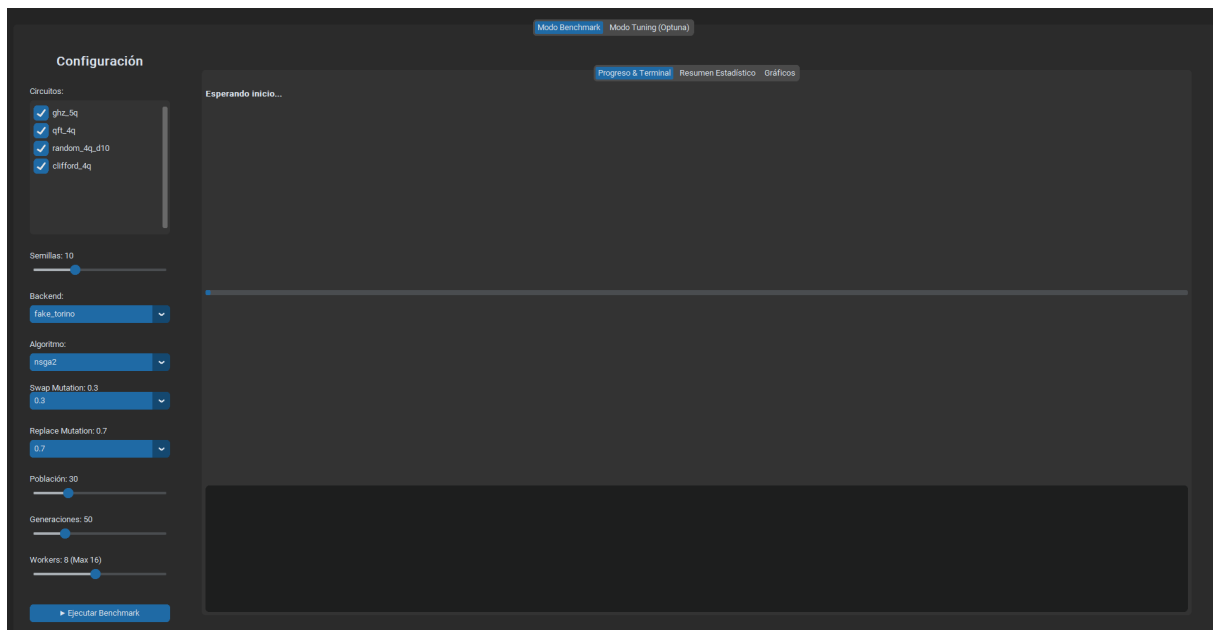


Figura 6.5. Captura de la interfaz gráfica de banco de pruebas y ajuste de hiperparámetros de *mo_module*.

obtener información del *backend*, transpilar con *layouts* externos y extraer métricas del circuito final. En la práctica, *qiskit_interface* define cómo se habla con Qiskit, *mo_module* decide qué *layouts* merece la pena probar e *integration* decide cómo se comparan los escenarios. El resultado de MO es reutilizable por otras rutas, pero no constituye por sí solo un flujo de transpilación completo.

Otra limitación procede de los objetivos activos. La versión descrita optimiza profundidad y coste equivalente de CNOTs tras transpilación. Ambas métricas son relevantes y comparables, pero no capturan todo el comportamiento físico de un *backend*. Errores de puerta, tiempos de coherencia, fidelidad esperada, duración de puertas o congestión topológica podrían incorporarse como objetivos adicionales o como criterios de diagnóstico. El diseño del evaluador, basado en estrategias registrables, facilita esa extensión; sin embargo, en el alcance actual esas magnitudes no forman parte de la configuración de fitness por defecto.

El módulo también hereda las limitaciones propias de una búsqueda evolutiva. No garantiza encontrar el óptimo global y su resultado depende de presupuesto computacional, configuración y semilla. Además, cada evaluación basada en transpilación es relativamente costosa, por lo que existe una tensión entre explorar más *layouts* y mantener tiempos de evaluación razonables. La caché de transpilación reduce trabajo repetido, pero no elimina el coste de evaluar individuos nuevos. Por ello, las conclusiones experimentales deben interpretarse como resultados de una estrategia heurística multi-objetivo, no como una prueba exhaustiva del espacio de *layouts*.

Frente a una línea base clásica de Qiskit, `mo_module` no acepta un único *layout* producido por una heurística interna, sino que expone alternativas y conserva compromisos entre objetivos. Esto permite comparar *layouts* por su posición dentro del frente de Pareto, elegir políticas de selección coherentes con cada experimento y reutilizar el *layout* resultante en escenarios superiores. La decisión metodológica que permanece abierta no es si existe un único *layout* universalmente mejor, sino qué criterio de selección conviene adoptar para cada finalidad experimental.

7

Desarrollo del módulo rl_module

7.1 Formulación del problema y arquitectura del entorno

El módulo `rl_module` constituye la capa de aprendizaje por refuerzo del proyecto. Su alcance base es el *routing*: dado un circuito objetivo, una topología física y un *layout* inicial, el agente aprende una política que decide qué operaciones **SWAP** aplicar sobre el mapa de acoplamiento, `coupling_map`, para desbloquear las puertas pendientes del circuito. Por tanto, el problema se plantea como una secuencia de decisiones condicionadas por el estado actual del *layout*, la frontera visible del circuito y las restricciones de conectividad del *backend*.

`QuantumTranspilationEnv` es la pieza central de esta formulación. Se trata de un entorno *Gymnasium* que transforma el *routing* en el ciclo habitual de aprendizaje por refuerzo: el entorno construye una observación, el agente selecciona una acción, el entorno actualiza su estado, calcula una recompensa y decide si el episodio ha terminado o se ha truncado. Esta abstracción habilita el uso de algoritmos de *Stable-Baselines3* sin acoplar la lógica del problema a una implementación concreta del agente.

Aunque el núcleo inicial del módulo es el *routing*, la arquitectura se diseñó con un mecanismo de extensión mediante el patrón *Strategy*. `RoutingStrategy` define la semántica de acciones y observaciones para el problema base, mientras que `SynthesisStrategy` reutiliza la misma infraestructura para un segundo problema, la síntesis Clifford. Esta síntesis no debe leerse como el objetivo original del capítulo ni como una síntesis general de circuitos arbitrarios: es una extensión funcional posterior, acotada, que demuestra que el entorno puede especializarse hacia otro subproblema sin duplicar toda la mecánica de episodios, recompensas y entrenamiento.

7.2 Estructura general del módulo

7.2.1 Visión global

El módulo se organiza alrededor de una separación clara entre dinámica del entorno, semántica del modo, función de recompensa, agente entrenable e inspección de episodios. Esta división resulta especialmente importante porque el entrenamiento ejecuta miles de transiciones y cualquier mezcla entre responsabilidades haría más difícil ajustar recompensas, cambiar la observación o evaluar modelos ya entrenados.

En el flujo base de *routing*, el entorno mantiene el estado del episodio, la estrategia traduce acciones discretas a SWAPs, la frontera decide qué puertas son visibles y la recompensa asigna crédito a las acciones que producen progreso real. Sobre ese núcleo se sitúan el agente de Stable-Baselines3, el flujo de entrenamiento, los metadatos de los puntos de control y la interfaz gráfica de inspección. El soporte de síntesis, denominado *synthesis* en el código, se incorpora como ampliación de esa misma arquitectura, no como sustitución del objetivo principal de *routing*.

7.2.2 Bloques funcionales y contratos

La Tabla 7.1 resume los bloques del módulo y marca su papel dentro del alcance del capítulo. La distinción entre núcleo, soporte común y extensión evita presentar *synthesis* con el mismo grado de centralidad que el *routing*.

7.2.3 Modelo estático del entorno

La Figura 7.1 muestra la estructura estática que permite reutilizar un mismo entorno para problemas con semánticas distintas. La clase `QuantumTranspilationEnv` conserva el estado común del episodio, mientras que la estrategia activa especializa la acción y la observación. En el alcance base se usa `RoutingStrategy`; en la ampliación posterior se usa `SynthesisStrategy`. La recompensa también se separa mediante `RewardStrategy`, de modo que su moldeado puede evolucionar sin mezclar la lógica de recompensa con la mecánica del entorno.

Bloque	Responsabilidad y alcance	Contratos principales
<code>environment</code>	Núcleo común del episodio: <i>layout</i> , contadores y ciclo <i>reset/step</i> .	<code>QuantumTranspilationEnv</code>
<code>env_strategies</code>	Semántica de acciones y observaciones para <i>routing</i> y síntesis.	<code>RoutingStrategy</code> ; <code>SynthesisStrategy</code>
<code>frontier</code>	Frontera visible y ejecución en cascada de puertas desbloqueadas en <i>routing</i> .	<code>SequentialFrontier</code> ; <code>DagFrontier</code>
<code>rewards</code>	Modelado de recompensa separado de la mecánica del entorno.	<code>RoutingReward</code> ; <code>SynthesisReward</code>
<code>agent</code>	Envoltorio común de Stable-Baselines3 y predicción de acciones.	<code>QuantumRLAgent</code>
<code>training y model_metadata</code>	Semillas, funciones de retorno, puntos de control y metadatos reproducibles.	<code>setup_training_pipeline</code> ; <code>run_metadata.json</code>
<code>gui</code>	Entrenamiento, evaluación e inspección de episodios con vistas por modo.	<code>RLBenchmarkGUI</code>

Tabla 7.1. Bloques funcionales de *rl_module* y relación con el alcance base de *routing*.

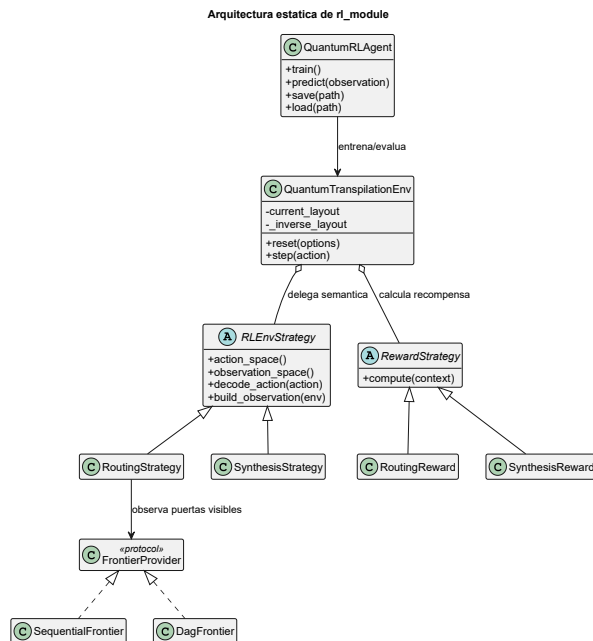


Figura 7.1. Modelo UML de clases de la arquitectura principal de *rl_module*.

7.2.4 Ciclo de episodio

La Figura 7.2 representa el ciclo de vida de un episodio RL. El entorno se inicializa con `reset`, construye una observación, recibe acciones sucesivas del agente y devuelve recompensa, nueva observación e información de terminación o truncado. En *routing*, la actualización modifica el *layout* mediante *SWAPs* y consume puertas cuando quedan físicamente habilitadas. En *synthesis*, como extensión del alcance base, la acción aplica una primitiva y actualiza un residual Clifford, es decir, la operación Clifford pendiente que todavía separa el circuito construido del objetivo.

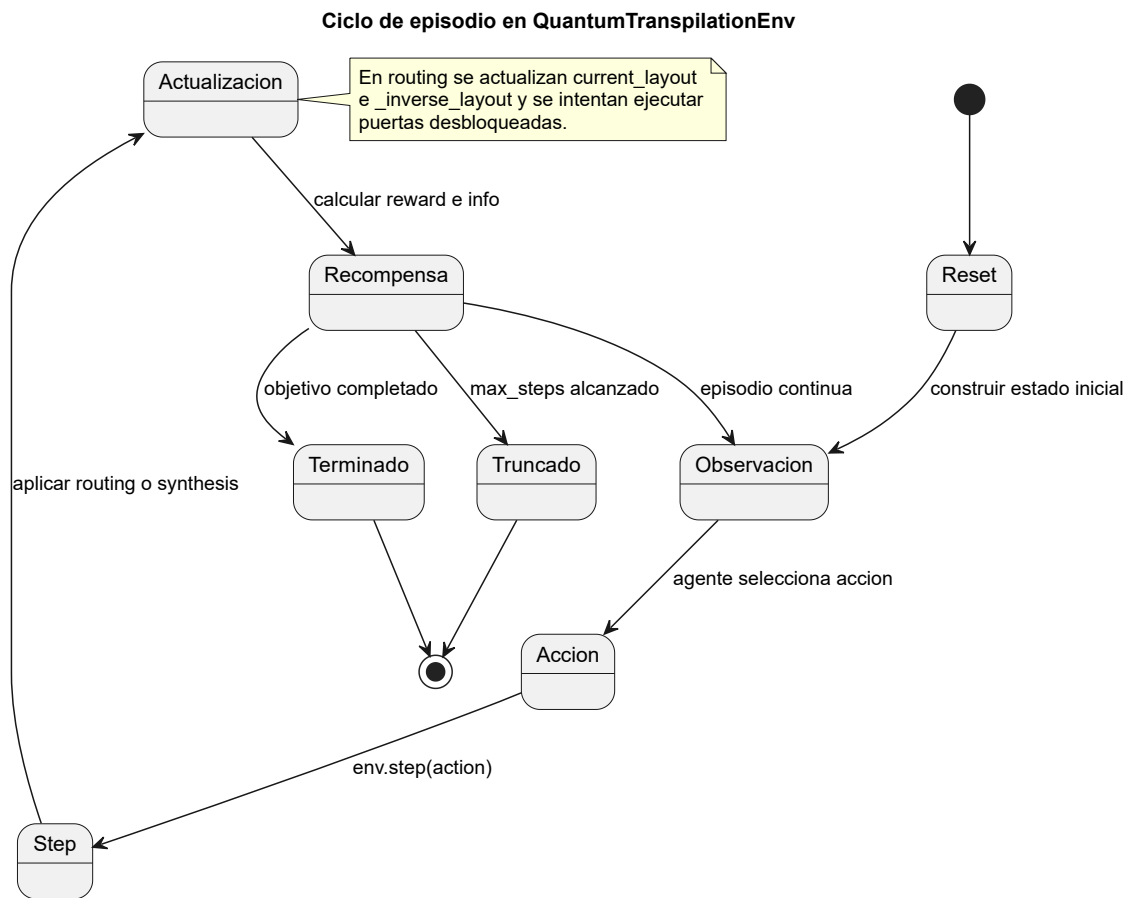


Figura 7.2. Diagrama UML de estados del ciclo de episodio en *QuantumTranspilationEnv*.

7.2.5 Secuencia de un paso de routing

La Figura 7.3 detalla la secuencia de un paso de *routing*, que es el recorrido principal del módulo. Tras recibir la acción elegida por el agente, la estrategia la decodifica como una arista física, el entorno actualiza `current_layout` y `_inverse_layout`, consulta la frontera visible e intenta ejecutar en cascada las puertas desbloqueadas. La recompensa se calcula al final con señales de progreso, acciones inválidas, repeticiones, deshacer el SWAP anterior y completitud del episodio.

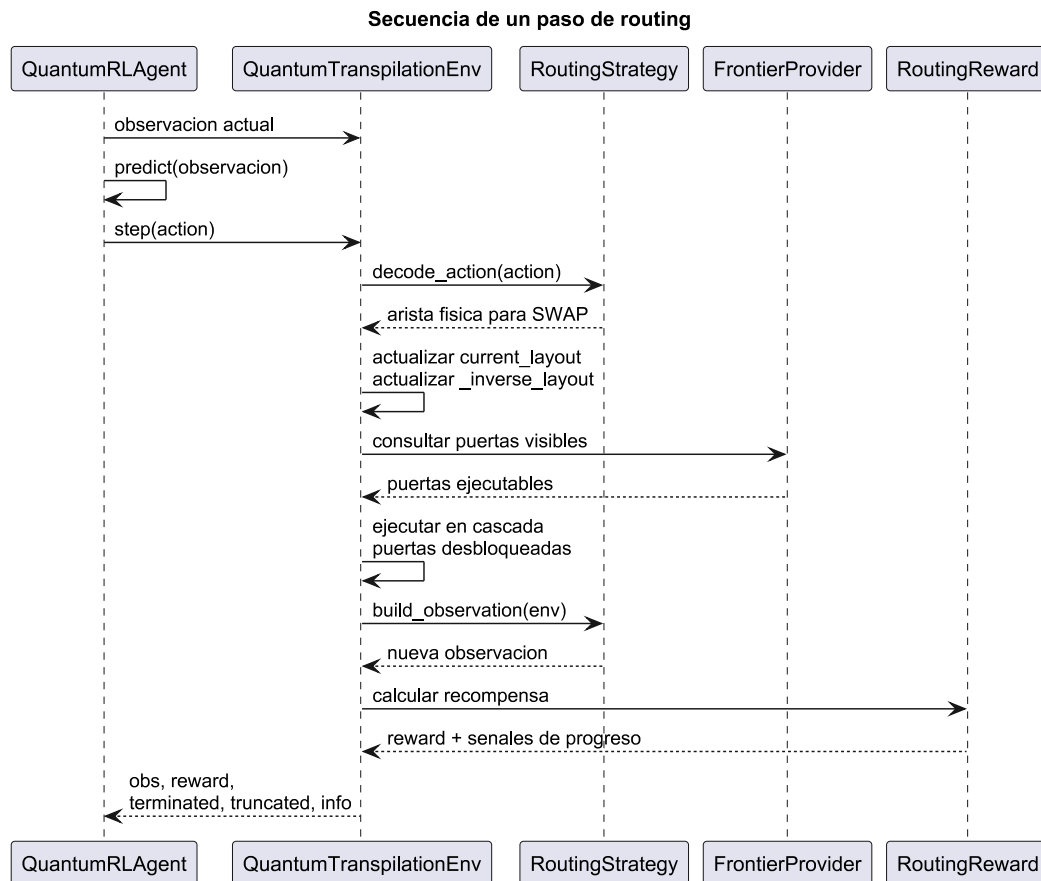


Figura 7.3. Secuencia de interacción entre agente, entorno, frontera y recompensa en un paso de *routing*.

7.2.6 Flujo principal

El flujo principal queda resumido por la cadena `reset` → observación → acción → `step` → recompensa → fin de episodio. En *routing*, las acciones son SWAPs sobre aristas físicas y el progreso se mide por puertas desbloqueadas y reducción de distancia de *routing*. En *synthesis*, la misma cadena se conserva, pero cambia la semántica: las acciones son primitivas conscientes del *hardware* y el progreso se mide por la reducción de un residual Clifford. Esta diferencia es esencial para no confundir

la extensión con el núcleo del módulo.

7.3 Representación interna del estado y del *layout*

En `rl_module` se usa el mismo contrato de *layout* que en todo el proyecto, definido en la Sección 4.2. Por tanto, el capítulo no introduce una representación nueva: el *layout* inicial entra con la misma semántica que emplean `qiskit_interface`, `mo_module` e `integration`, y el módulo RL lo consume como `initial_layout`. Si no se proporciona, el entorno usa un *layout* trivial determinista.

Esta decisión es especialmente importante en aprendizaje por refuerzo porque el episodio no debe reinterpretar los datos que recibe de otras capas. En particular, `rl_module` respeta los contratos ya existentes: no genera *layouts*, no decide su origen y no modifica la forma pública del dato. El *layout* puede venir de una configuración manual, de un escenario de integración o de un *layout* optimizado por `mo_module`, pero la transferencia entre productor y consumidor y la validación intermodular son responsabilidad de `integration`.

Para ejecutar el episodio de forma eficiente, el entorno mantiene dos vistas sincronizadas. La primera, `current_layout`, indica dónde está situado cada cúbit lógico:

```
current_layout[logical_qubit] = physical_qubit.
```

La vista inversa, `_inverse_layout`, responde a la pregunta complementaria:

```
_inverse_layout[physical_qubit] = logical_qubit.
```

Esta duplicidad permite consultar y actualizar el *layout* en tiempo constante durante cada **SWAP**. Cuando el *backend* tiene más cúbits físicos que el circuito lógico, los cúbits físicos libres se representan con `-1` en `_inverse_layout`. En *routing*, esta representación permite saber qué cúbits lógicos intercambia una arista física y cómo cambia la ejecutabilidad de la frontera. En *synthesis*, la misma estructura se reutiliza dentro de la extensión, pero el *layout* permanece fijo durante el episodio.

7.4 Modo *routing* como alcance base

El modo *routing* es el alcance base del módulo. Su objetivo no es construir un circuito desde cero, sino decidir una secuencia de **SWAPs** que haga ejecutables las puertas pendientes de un circuito dado sobre una conectividad física limitada. Cada acción del espacio discreto corresponde a una arista única del mapa de acoplamiento, `coupling_map`; seleccionar una acción significa aplicar un **SWAP** entre los dos cúbits físicos de esa arista.

La observación se construye como un diccionario de tensores, porque el agente necesita información heterogénea: *layout*, frontera lógica, proyección física de la frontera, ejecutabilidad, distancia aproximada de *routing* y progreso temporal. Esta estructura justifica el uso de `MultiInputPolicy` durante el entrenamiento, ya que la política debe procesar varias entradas con semánticas distintas. La Tabla 7.2 enumera esos campos y concreta el papel de cada uno dentro del episodio.

Campo	Tipo conceptual	Papel en <i>routing</i>
<code>layout</code>	Layout lógico-físico	Indica dónde reside cada cúbit lógico bajo el <i>layout</i> actual.
<code>lookahead</code>	Ventana lógica	Codifica las próximas puertas visibles como pares de cúbits lógicos.
<code>lookahead_physical</code>	Proyección física	Muestra qué pares físicos corresponden a la frontera bajo el <i>layout</i> actual.
<code>lookahead_executable</code>	Indicador binario	Señala qué puertas visibles ya pueden ejecutarse sin nuevos SWAPs .
<code>lookahead_routing_distance</code>	Distancia aproximada	Estima cuántos movimientos faltan para conectar puertas bloqueadas.
<code>lookahead_valid_mask</code>	Máscara de relleno	Distingue entradas reales de posiciones vacías en la ventana.
<code>step_progress</code>	Escalar temporal	Aporta contexto sobre el avance relativo dentro del episodio.

Tabla 7.2. Campos principales de la observación en el modo *routing*.

El módulo contempla dos formas de interpretar la frontera. `SequentialFrontier` conserva una lectura lineal de las puertas pendientes y mantiene compatibilidad con el comportamiento inicial del entorno. `DagFrontier`, en cambio, utiliza la `front_layer()` del DAG de Qiskit para exponer el conjunto de puertas que se podrían ejecutar a continuación en el circuito. En ambos casos, después de aplicar un **SWAP**, el entorno intenta ejecutar en cascada todas las puertas que hayan quedado desbloqueadas. Esto evita premiar movimientos abstractos sin efecto y vincula la recompensa al progreso efectivo sobre el circuito.

Una versión ingenua, que permite elegir como acción cualquier arista del grafo, puede dificultar el aprendizaje. Por ello, se desarrolló una variante de *routing* enmascarado, denominada *masked routing*

en el código, que restringe el espacio de acción a las aristas útiles en cada estado. El catálogo discreto sigue siendo el conjunto fijo de aristas del `coupling_map`; `action_masks()` aplica una máscara determinista y dependiente de la frontera para impedir que el agente muestree `SWAPs` poco útiles en el estado actual. Una acción se considera poco útil cuando no reduce la distancia de *routing* de ninguna puerta visible, deshace inmediatamente el `SWAP` anterior o conduce a un *layout* repetido. Esta decisión conserva índices de acción estables y permite entrenar puntos de control nuevos con `MaskablePPO`, mientras que los modelos heredados basados en PPO o DQN siguen evaluándose bajo sus contratos correspondientes.

La recompensa de *routing* combina penalizaciones y bonificaciones densas. Penaliza cada `SWAP`, las acciones inválidas, la repetición de *layouts* recientes, deshacer inmediatamente el `SWAP` anterior y truncar el episodio sin completarlo. A la vez, bonifica cada puerta ejecutada, la reducción de la distancia agregada de *routing* y la completitud del circuito. Esta combinación no garantiza por sí sola una política óptima, pero ofrece al agente señales locales que aproximan el objetivo global: completar el circuito con menos movimientos inútiles.

7.5 Modo de síntesis como extensión del alcance base

El modo `synthesis` no forma parte del núcleo inicial del TFG. Se incorporó como una extensión funcional posterior para comprobar si la infraestructura de `QuantumTranspilationEnv`, estrategias, recompensas, entrenamiento e inspección podía reutilizarse en otro subproblema de transpilación.

En esta extensión, el agente no decide `SWAPs` para desbloquear una cola de puertas. Decide primitivas nativas de un catálogo consciente del *hardware* con el objetivo de reducir un residual Clifford. Para construir ese catálogo no basta con conocer el `coupling_map`: también hacen falta las `basis_gates`, porque la topología indica qué cúbits pueden interactuar, pero no qué puerta de dos cúbits es nativa del *backend*. El catálogo puede incluir primitivas de un cúbit, rotaciones `rz` discretizadas y una puerta nativa de dos cúbits, como `cz`, `ecr` o `cx`, sobre las aristas válidas.

La noción central del modo es el residual Clifford. En este contexto, el residual Clifford es la transformación Clifford que todavía habría que aplicar para que el Clifford actual, construido por las primitivas ya seleccionadas, equivalga al Clifford objetivo remapeado al espacio físico. Si ese residual llega a la identidad, la síntesis se considera completada. Esta primera versión entrenable solo cubre circuitos Clifford, mantiene el *layout* fijo durante el episodio y no debe describirse como síntesis general de circuitos cuánticos arbitrarios. La Tabla 7.3 resume las diferencias principales entre el alcance base de *routing* y esta extensión.

La recompensa de `synthesis` también refleja su carácter centrado en el residual. Penaliza

Aspecto	<i>routing</i>	<i>synthesis</i>
Papel en el capítulo	Alcance base del módulo.	Extensión funcional posterior.
Objetivo	Insertar SWAPs para desbloquear puertas pendientes.	Construir equivalencia Clifford mediante primitivas nativas.
Acción	Arista física del coupling_map .	Primitiva del catálogo consciente del <i>hardware</i> .
Observación principal	<i>Layout</i> , frontera visible, ejecutabilidad y distancia de <i>routing</i> .	<i>Layout</i> fijo, vista físico-lógica y residual Clifford.
Progreso	Puertas ejecutadas y reducción de distancia de <i>routing</i> .	Reducción de distancia residual.
Terminación	No quedan puertas pendientes por enrutar.	El residual es la identidad.
Limitación principal	Riesgo de ciclos u oscilaciones en evaluación determinista.	Solo Clifford, <i>layout</i> fijo y sin síntesis general no-Clifford.

Tabla 7.3. Diferencias entre el alcance base de *routing* y la extensión *synthesis*.

acciones inválidas, cada paso válido y el coste de la primitiva, y bonifica la reducción de distancia residual y la completitud. Esta recompensa es coherente con la extensión, pero no debe confundirse con la recompensa de *routing*: en *synthesis* no hay frontera de puertas pendientes ni ejecución en cascada de puertas desbloqueadas.

7.6 Entrenamiento, evaluación e interpretabilidad

El entrenamiento se articula mediante `QuantumRLAgent`, un envoltorio sobre `Stable-Baselines3` que crea, entrena, guarda y carga modelos. Los algoritmos soportados son PPO, DQN y, para puntos de control nuevos de *routing* enmascarado, `MaskablePPO`. Este último queda restringido al modo *routing* porque depende de máscaras de acción sobre el espacio fijo de aristas; no se presenta como algoritmo general para la extensión *synthesis*.

El agente utiliza por defecto `MultiInputPolicy`, adecuada para observaciones de tipo diccionario. El flujo de entrenamiento fija semillas globales, crea entornos de entrenamiento y evaluación, los envuelve con `Monitor`, configura funciones de retorno, registra métricas para `TensorBoard` y guarda los artefactos del modelo. Además, el bloque de metadatos persiste `run_metadata.json`, que permite reconstruir con qué modo, algoritmo, frontera y parámetros fue entrenado un punto de control.

La Figura 7.4 separa dos responsabilidades que conviene no mezclar. `rl_module` puede entrenar y evaluar agentes, pero la comparación entre escenarios del proyecto y la inyección de *layouts* externos pertenecen a `integration`. En particular, `integration` puede consumir un modelo RL ya entrenado y pasar un `initial_layout`, pero no por ello el módulo RL se convierte en productor de *layouts*.

Entrenamiento y evaluación de modelos RL

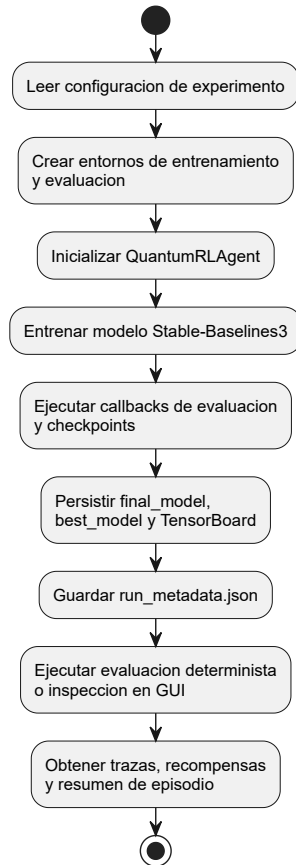


Figura 7.4. Flujo de entrenamiento, persistencia y evaluación de modelos en `rl_module`.

La interfaz gráfica unificada aporta interpretabilidad al proceso. En *routing*, permite inspeccionar *layouts* antes y después del paso, frontera visible, arista de **SWAP**, puertas ejecutadas, progreso de *routing* y señales de ciclo como `repeated_layout` o `undo_swap`. En la extensión *synthesis*, reutiliza la misma infraestructura de inspección, pero muestra primitivas, cúbits físicos, coste de la acción y variación de la distancia residual. Esta diferencia refuerza de nuevo que ambos modos comparten soporte técnico, pero no comparten semántica. La Figura 7.5 muestra una captura de la interfaz gráfica.

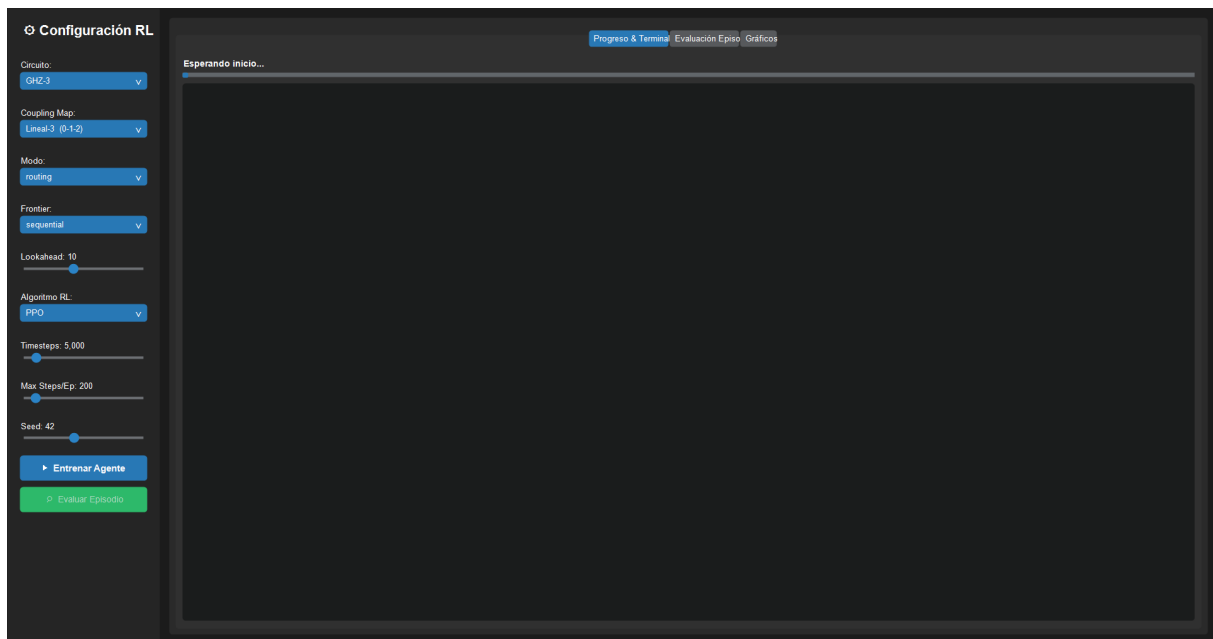


Figura 7.5. Captura de la interfaz gráfica de *rl_module*.

7.7 Limitaciones actuales y aportación del módulo

El módulo debe interpretarse como una infraestructura experimental, no como una reimplementación completa del compilador de Qiskit. En *routing*, la limitación más relevante es la posible oscilación durante evaluación determinista. Una política puede aprender trayectorias razonables durante entrenamiento estocástico y, sin embargo, repetir patrones del tipo $A \rightarrow B \rightarrow A$ cuando se evalúa sin exploración. El módulo incorpora señales de progreso temporal, detección de *layouts* repetidos, penalización por deshacer *SWAPs* y máscaras de acción en el régimen nuevo, pero no dispone todavía de una política recurrente que modele memoria temporal de largo alcance.

También existe una cautela importante sobre la lectura experimental de las salidas. Un episodio RL produce trazas, recompensas, pasos, *SWAPs*, *layouts* iniciales y finales, y metadatos de terminación o truncado. Cuando el *routing* completa, *integration* reconstruye el recorrido y obtiene métricas *post-routing* comparables mediante Qiskit. Cuando la fase de *routing* no completa por falta de entrenamiento del modelo, deben analizarse por separado las métricas de episodio y las incidencias, sin mezclarlas con los agregados homogéneos.

En *synthesis*, las restricciones anteriores se concentran en tres puntos: solo soporta circuitos Clifford, exige *basis_gates* y mantiene el *layout* fijo. Por ello, su aportación actual es demostrar extensibilidad de la infraestructura, no resolver síntesis general no-Clifford.

rl_module formula el *routing* como un problema de decisión secuencial, proporciona observa-

ciones enriquecidas con contexto físico, separa la recompensa de la dinámica del entorno, permite entrenar políticas con herramientas consolidadas y ofrece inspección paso a paso del comportamiento aprendido. La extensión *synthesis* añade una prueba de extensibilidad al adaptar la misma arquitectura a una semántica centrada en el residual.

8

Desarrollo del módulo *integration*

8.1 Papel y alcance de la capa de integración

`integration` actúa como capa de orquestación entre los módulos especializados del proyecto. Su función es decidir qué componentes intervienen en cada escenario, validar los datos intercambiados y conservar resultados comparables y trazables. De esta forma, `qiskit_interface` mantiene la interacción con Qiskit, `mo_module` produce *layouts* candidatos y `rl_module` aporta el entrenamiento y la evaluación de políticas de *routing*.

Esta separación resulta especialmente importante en el camino híbrido. El módulo multiobjetivo devuelve un frente de Pareto; una política de selección reduce ese frente a un *layout* concreto; y la capa de integración entrega esa asignación al entorno RL como `initial_layout`. El contrato compartido definido en la arquitectura permanece inalterado durante el traspaso.

El alcance principal de la versión descrita en este capítulo es el *routing*. La capa permite estudiar cuatro escenarios comparativos, ejecutar campañas reproducibles y obtener métricas homogéneas cuando las rutas RL completan y pueden evaluarse tras el *routing*. Las ampliaciones experimentales que extienden este núcleo se presentan en el capítulo siguiente.

8.2 Estructura general del módulo

8.2.1 Dos niveles de orquestación

La interfaz pública de `integration` distingue dos niveles. Un escenario (`Scenario` en la implementación) representa una evaluación puntual sobre un circuito y un *backend* concretos. Su finalidad es ejecutar una de las rutas `Baseline`, `MO_Only`, `RL_Only` o `MO+RL` y devolver un resultado estructurado, que en las rutas RL puede ser completo o quedar en resumen de episodio. Una campaña (`Campaign` en la implementación), en cambio, organiza una secuencia reproducible de entrenamiento y evaluación sobre uno o varios casos. Cada caso corresponde a una combinación *circuito* $\times$ *backend*.

La distinción evita mezclar dos necesidades diferentes. Los escenarios son útiles para inspeccionar una ruta aislada, mientras que las campañas permiten comparar estrategias bajo una configuración compartida, persistir resultados de forma incremental y conservar incidencias. Ambos niveles reutilizan los mismos contratos de circuito, *backend*, *layout* y métricas.

Para la ejecución operativa, `integration` expone además una interfaz de línea de comandos (CLI) que permite lanzar campañas por lotes a partir de un fichero JSON de configuración. Esta entrada no cambia los contratos del módulo, sino que automatiza la carga de campañas, su validación y la escritura de los artefactos de salida.

8.2.2 Bloques funcionales

La Tabla 8.1 resume los bloques principales de la capa. La organización mantiene separadas la evaluación puntual, el entrenamiento, la selección de *layouts* y la generación de informes.

8.2.3 Diagrama de componentes

La Figura 8.1 representa `integration` como una capa de composición. Los módulos externos conservan sus responsabilidades y se conectan mediante piezas pequeñas: adaptación del *backend*, selección del *layout*, evaluación de *routing*, entrenamiento y generación de informes.

8.3 Contratos y DTOs de integración

Los contratos públicos hacen explícito qué datos circulan entre capas y en qué momento deben rechazarse entradas inconsistentes. DTO, del inglés *Data Transfer Object*, es un objeto usado para transportar datos entre capas sin exponer detalles internos de implementación. La Tabla 8.2 recoge los DTOs principales. Esta interfaz evita que las campañas dependan de detalles internos de Qiskit, pymoo o Stable-Baselines3.

Bloque	Responsabilidad
contracts y campaign_contracts	Definir y validar peticiones, resultados, configuraciones y resúmenes públicos.
scenarios	Ejecutar las cuatro rutas comparativas y coordinar sus dependencias.
layout_policy	Seleccionar un único <i>layout</i> a partir de la salida de <i>mo_module</i> .
routing_evaluator	Evaluar episodios RL, conservar trazas y reconstruir el circuito ruteado.
training_bridge	Traducir una configuración de campaña al flujo de entrenamiento de <i>rl_module</i> .
campaign_runner y campaign_reporting	Orquestar casos, persistir avances y generar informes comparativos.

Tabla 8.1. Bloques funcionales principales de *integration*.

Contrato	Finalidad	Contenido relevante
ScenarioRequest	Describir una evaluación puntual.	Escenario, circuito, <i>backend</i> , semilla, política MO, <i>layout</i> inicial y punto de control RL cuando proceda.
RoutingEpisodeSummary	Resumir la ejecución RL.	Recompensa, pasos, completitud, truncado, <i>layouts</i> , número de SWAPs, <i>swap_trace</i> y <i>executed_gate_trace</i> .
ScenarioResult	Unificar la salida de cualquier escenario.	Éxito, <i>layout</i> seleccionado, métricas de transpilación, artefacto estructurado, resumen RL, errores y notas.
CampaignConfig	Fijar una campaña reproducible.	Circuitos, <i>backends</i> , parámetros RL, esfuerzo MO, política de <i>layout</i> , semilla y topología.
CampaignCase	Identificar una unidad experimental.	Familia de circuito, número de cúbits y <i>backend</i> seleccionado.
CampaignSummary	Agregar el estado de la campaña.	Casos comparables completados, fallidos, incompletos y cancelados.

Tabla 8.2. DTOs públicos utilizados por la capa *integration*.

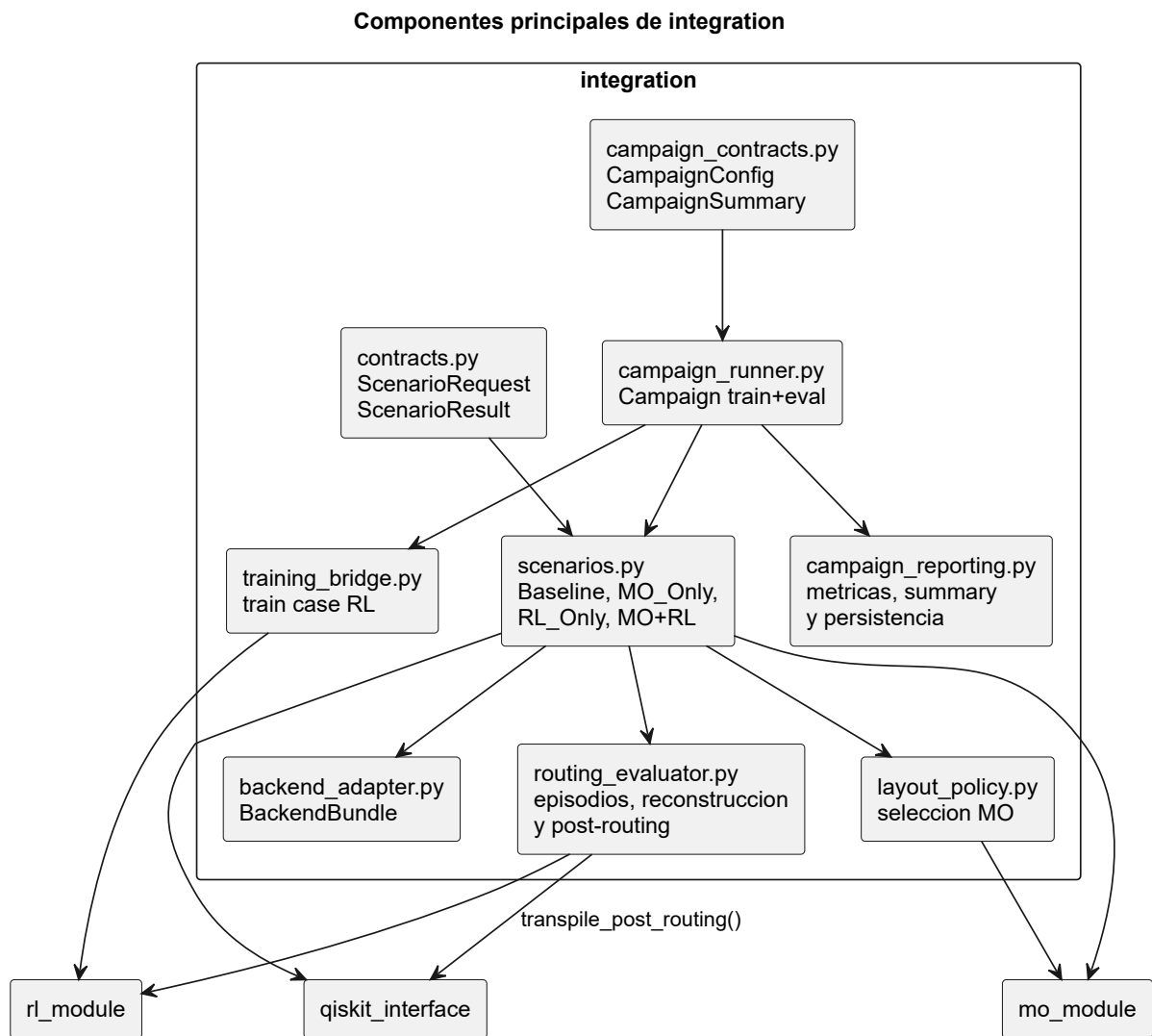


Figura 8.1. Diagrama UML de componentes principales de *integration*.

`ScenarioRequest` aplica restricciones específicas por escenario. Por ejemplo, `Baseline` no admite parámetros RL ni un `layout` externo; `MO_Only` no recibe puntos de control; y las rutas RL requieren la ruta del modelo entrenado. Cuando existe un `layout` de entrada, la validación comprueba longitud, rango físico y ausencia de duplicados. `RoutingEpisodeSummary` añade invariantes propias: el número de `swaps` debe coincidir con la longitud de `swap_trace`, y un episodio no puede aparecer simultáneamente como completado y truncado.

El DTO `ScenarioResult` permite conservar dos familias de información sin perder su procedencia. Las métricas de transpilación sirven para la comparación homogénea entre rutas, mientras que el resumen RL mantiene recompensa, pasos, `swaps` y trazas útiles para analizar el comportamiento del agente. En la capa de campaña, `CampaignSummary` distingue explícitamente los casos comparables de aquellos que finalizaron sin disponer del conjunto completo de métricas.

8.4 Escenarios de evaluación

La Figura 8.2 resume las cuatro rutas públicas. Todas comparten la resolución inicial del circuito y del `backend`, pero difieren en el origen del `layout` y en los módulos especializados que intervienen.

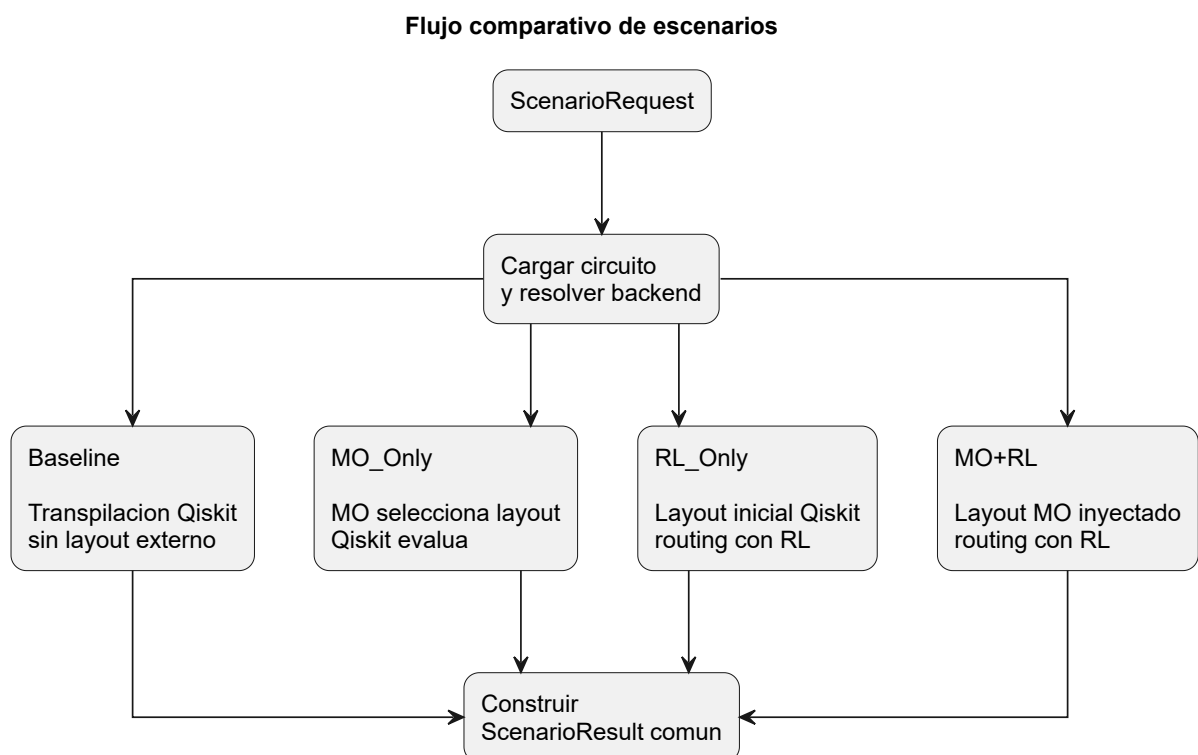


Figura 8.2. Actividad comparativa de los escenarios *Baseline*, *MO_Only*, *RL_Only* y *MO+RL*.

La Tabla 8.3 complementa la figura anterior con una lectura funcional de cada escenario.

Escenario	Origen del <i>layout</i>	Módulos implicados	Resultado cuantitativo
Baseline	<i>Layout</i> resuelto por Qiskit.	<code>qiskit_interface</code> .	Métricas de la transpilación de referencia.
MO_Only	<i>Layout</i> seleccionado del frente de Pareto.	<code>mo_module</code> y <code>qiskit_interface</code> .	Métricas de Qiskit con <i>layout</i> externo.
RL_Only	<i>Layout</i> inicial de Qiskit, sin <i>layout</i> MO.	<code>rl_module</code> y evaluación posterior al <i>routing</i> de <code>qiskit_interface</code> .	Métricas tras <i>routing</i> y resumen RL si el <i>routing</i> completa.
MO+RL	<i>Layout</i> seleccionado por MO e inyectado en RL.	Los tres módulos especializados.	Métricas tras <i>routing</i> y resumen RL si el <i>routing</i> completa.

Tabla 8.3. Comparación funcional de los cuatro escenarios públicos.

8.4.1 Baseline

Baseline establece la referencia clásica del estudio. La capa invoca la transpilación `qiskit_level_1` sin imponer un *layout* externo y conserva métricas como profundidad, puertas de dos cúbits, coste equivalente en CNOT y tiempo de transpilación. Esta ruta permite interpretar las mejoras o regresiones observadas en el resto de escenarios.

8.4.2 MO_Only

MO_Only aísla el efecto del *layout* inicial. Primero ejecuta `mo_module`, obtiene el frente de Pareto y selecciona un *layout* único mediante la política configurada. Después, `qiskit_interface` evalúa la asignación con la referencia `custom_layout_level_1`. La comparación con Baseline responde así a una pregunta concreta: cuánto cambia el resultado de Qiskit cuando el punto de partida procede de una búsqueda multiobjetivo.

La campaña contempla tres modos históricos de selección sobre el mismo frente de Pareto. Estos modos no modifican la ejecución del optimizador, sino el criterio utilizado para reducir el conjunto de soluciones no dominadas a un único *layout*. La Tabla 8.4 resume su semántica.

Internamente, `compromise` se corresponde con la política COMPROMISE. Los modos `best_depth` y `best_cnot_count` especializan la política BEST_ON_OBJECTIVE sobre `depth` y `cnot_count`, respectivamente. En una matriz de campañas pueden evaluarse los tres modos sobre el mismo frente y la misma semilla. Esta reutilización permite comparar el efecto del criterio de selección sin introducir una variación adicional en la búsqueda evolutiva.

Modo	Criterio de selección	Interpretación experimental
<code>compromise</code>	Escoge el <i>layout</i> más cercano al punto ideal en el espacio normalizado de objetivos.	Busca una solución equilibrada entre profundidad y coste equivalente en CNOT, sin privilegiar uno de los extremos del frente.
<code>best_depth</code>	Escoge el <i>layout</i> con el menor valor del objetivo <code>depth</code> .	Prioriza la reducción de la profundidad transpilada, aunque el coste equivalente en CNOT pueda ser mayor.
<code>best_cnot_count</code>	Escoge el <i>layout</i> con el menor valor del objetivo <code>cnot_count</code> .	Prioriza la reducción del coste equivalente en CNOT, aunque la profundidad transpilada pueda aumentar.

Tabla 8.4. Modos históricos de selección de *layout* disponibles para *MO_Only*.

8.4.3 RL_Only

RL_Only permite estudiar la contribución del *routing* aprendido sin introducir un *layout* optimizado por MO. La ruta reutiliza el *layout* inicial resuelto por Qiskit, carga el punto de control RL, resuelve su contrato de metadatos y evalúa el episodio. Cuando el *routing* completa, reconstruye la secuencia ejecutada y delega la evaluación posterior al *routing* en Qiskit.

8.4.4 MO+RL

MO+RL materializa la combinación principal del proyecto. El frente de Pareto generado por `mo_module` se reduce a un *layout* concreto mediante `layout_policy`. La capa valida la asignación y la inyecta en `rl_module` como `initial_layout`. De esta forma, el agente comienza el *routing* desde una solución elegida con criterios multiobjetivo en lugar de depender únicamente del *layout* inicial de referencia.

Los modos de selección descritos para *MO_Only* también afectan a MO+RL. En una matriz de campañas, cada modo escoge su *layout* sobre el mismo frente de Pareto y ese *layout* se utiliza tanto para evaluar *MO_Only* como para iniciar el entrenamiento híbrido y ejecutar MO+RL. Por tanto, `compromise`, `best_depth` y `best_cnot_count` no solo cambian la transpilación clásica con *layout* externo: también modifican el punto de partida desde el que aprende y actúa el agente en la rama híbrida. Si dos modos seleccionan exactamente el mismo *layout* físico, la matriz de campañas reutiliza la misma rama de entrenamiento y evaluación MO+RL, ya que no existe una diferencia efectiva que justifique repetir el cálculo.

La Figura 8.3 muestra el traspaso MO $\rightarrow$ RL completo. Tras un episodio RL completado, la reconstrucción y la evaluación posterior al *routing* permiten analizar el resultado híbrido con las mismas métricas utilizadas en el resto de escenarios comparables.

8.5 Reconstrucción y evaluación tras *routing*

La evaluación RL no termina al registrar recompensa y *swaps*. Cuando el episodio completa, el evaluador de *routing* reconstruye internamente el recorrido antes de enviarlo a Qiskit. La reconstrucción utiliza prioritariamente `executed_gate_trace`, que conserva el orden exacto de las puertas ejecutadas, y materializa los movimientos físicos registrados en `swap_trace`. Durante el proceso mantiene actualizado el *layout* y verifica que las interacciones reconstruidas respeten el grafo de *routing*.

El circuito reconstruido pasa a la evaluación posterior al *routing*. En esa ruta, `qiskit_interface` aplica las etapas posteriores del flujo de Qiskit y devuelve un `TranspilationResult` con el mismo tipo de métricas utilizado por las rutas clásicas. La salida pública de la ruta RL conserva el resumen del episodio junto con esas métricas cuando existen; no expone un `QuantumCircuit` final como contrato independiente.

Si el episodio no completa, `integration` evita producir métricas posteriores al *routing* parciales como si constituyeran una solución válida. El resultado conserva el resumen RL, el motivo de truncado o terminación y las incidencias necesarias para el diagnóstico. Esta política impide que una ejecución incompleta distorsione los agregados experimentales.

8.6 Campañas reproducibles

Una campaña (`Campaign` en la implementación) eleva la comparación desde un escenario aislado hasta una ejecución reproducible. Cada `CampaignCase` representa una combinación *circuito* $\times$ *back-end*. Para cada caso, `campaign_runner` ejecuta la secuencia mostrada en la Figura 8.4: *Baseline*, entrenamiento y evaluación de `RL_Only`, selección y evaluación `MO_Only`, entrenamiento híbrido y evaluación `MO+RL`.

El puente `training_bridge` transforma la configuración pública de la campaña en parámetros del flujo de `rl_module`. Para `RL_Only`, el entrenamiento parte del *layout* inicial de Qiskit. Para `MO+RL`, reutiliza exactamente el *layout* seleccionado por `MO_Only`. Esta simetría permite atribuir las diferencias observadas al punto de partida y al entrenamiento asociado a cada ruta.

La campaña intenta derivar un subgrafo de *routing* expandido por caminos, adaptado al circuito y al *layout* seleccionado. Si esa derivación no resulta aplicable, conserva el `coupling_map` completo y registra el mecanismo de respaldo. Durante el entrenamiento, la selección de puntos de control

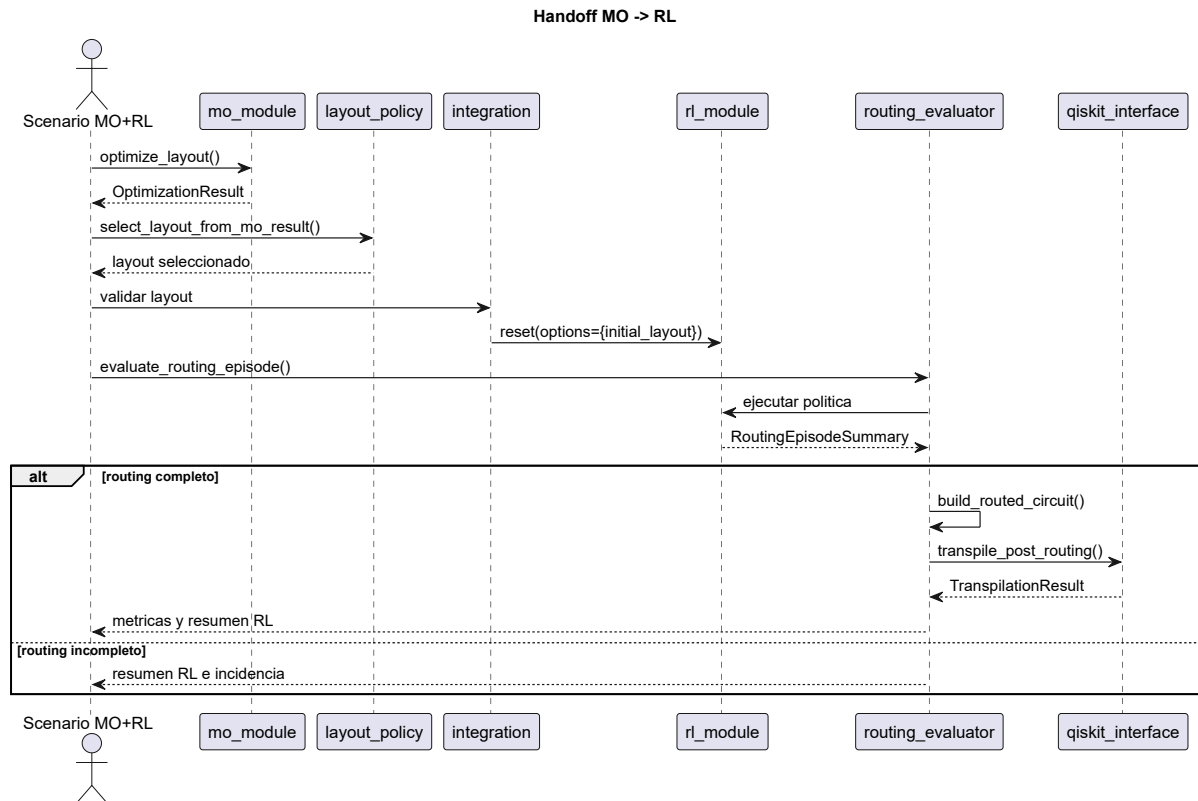


Figura 8.3. Secuencia del traspaso *MO+RL* coordinado por *integration*.

tampoco depende únicamente de la recompensa: cuando existe una solución válida, se evalúa el resultado tras *routing* y se prioriza la tupla (*cnot_equivalent*, *depth*, *swaps*). De este modo, el artefacto escogido queda alineado con las métricas finales del estudio.

Campaña reproducible: flujo de entrenamiento y evaluación

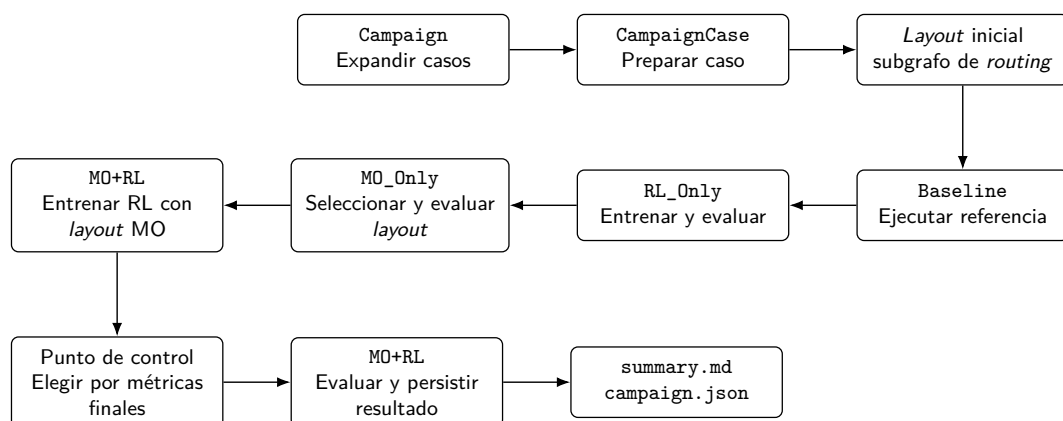


Figura 8.4. Actividad principal de una campaña reproducible en *integration*.

La persistencia incremental evita perder trazabilidad si una ejecución se interrumpe o si un caso falla. La Tabla 8.5 recoge los artefactos públicos esenciales.

Artefacto	Finalidad
<code>summary.md</code>	Documento legible con configuración, métricas agregadas e incidencias.
<code>campaign.json</code>	Salida estructurada completa para análisis automático.
<code>cases/<case>/result.json</code>	Resultado detallado de cada combinación circuito– <i>backend</i> .

Tabla 8.5. Artefactos persistidos por una campaña base.

8.7 Comparabilidad actual entre escenarios

Los cuatro escenarios pueden compararse de forma homogénea cuando las rutas RL completan el *routing* y la evaluación posterior al *routing* finaliza correctamente. En ese caso, *Baseline*, *MO_Only*, *RL_Only* y *MO+RL* disponen de profundidad, número de puertas de dos cúbits, coste equivalente en CNOT y tiempo de transpilación. La comparación permite estudiar tanto el efecto aislado del *layout* multiobjetivo como su combinación con *routing* aprendido.

La condición de completitud forma parte del protocolo, no es un detalle secundario. *campaign_reporting* solo agrega un caso como comparable cuando los cuatro resultados contienen el conjunto completo de métricas. Un caso puede terminar su ejecución técnica y, aun así, clasificarse como incompleto para el análisis homogéneo si falta alguna medida. Además, las rutas RL conservan indicadores específicos como recompensa, pasos, *swaps*, completitud y truncado.

8.8 Alcance actual y ampliaciones

La versión base de *integration* cubre la orquestación de escenarios de *routing*, el traspaso *MO* → *RL*, el entrenamiento por caso y la generación de informes comparativos. *Baseline* y *MO_Only* admiten circuitos procedentes de biblioteca o de fichero QASM. Las rutas basadas en RL se emplean dentro del flujo controlado de circuitos de biblioteca utilizado por las campañas.

Sobre este núcleo se han incorporado ampliaciones orientadas a experimentación más exigente: matrices con varias semillas, comparación de varias políticas de selección *MO*, topologías sintéticas, la sonda determinista *hybrid_probe* y el modo experimental *rl_guided*. Estas capacidades amplían las estrategias de búsqueda, la escala de los experimentos y la trazabilidad disponible, pero pertenecen a una capa posterior al núcleo principal del sistema. Su motivación y alcance se recogen en el capítulo siguiente.

9

Ampliaciones realizadas

9.1 Motivación y criterio de ampliación

Los capítulos anteriores han descrito el núcleo modular del sistema: una capa de interacción con Qiskit, un módulo de optimización multiobjetivo, un entorno de aprendizaje por refuerzo y una capa de integración que compone escenarios de *routing*. Sobre esa base se incorporaron varias ampliaciones orientadas a resolver necesidades que aparecieron durante el desarrollo experimental. Algunas amplían la entrada del sistema, otras mejoran la estabilidad del aprendizaje, otras permiten explorar estrategias híbridas más ambiciosas y otras refuerzan la trazabilidad de las campañas.

El capítulo distingue el papel de cada ampliación dentro del proyecto. La lectura de circuitos en QASM y el ajuste de la optimización multiobjetivo amplían capacidades de uso y configuración. El *routing* enmascarado con `MaskablePPO` modifica la forma en que se entrena y evalúa el agente en los experimentos nuevos de *routing*. El modo `synthesis` prueba la reutilización de `rl_module` en otro subproblema. Las estrategias `hybrid_probe` y `rl_guided` actúan como mecanismos de selección avanzada dentro de campañas, no como sustitutos de los escenarios base.

También se añadieron ampliaciones específicas de la capa *integration*. En particular, la derivación de subgrafos de *routing* permite entrenar y evaluar una rama RL sobre una conectividad adaptada al circuito y al *layout* seleccionado, conservando un mecanismo de respaldo explícito al mapa completo cuando no es aplicable. Por último, las campañas incorporan matrices multisentida, ejecución de varios modos de selección y artefactos persistidos que facilitan auditar por qué un caso es comparable, incompleto o experimentalmente descartado.

9.2 Lectura de circuitos en QASM

La biblioteca interna de circuitos resulta suficiente para comparar familias controladas como `ghz`, `qft`, `random_shallow`, `random_deep` o `clifford`. Sin embargo, limitar el sistema a circuitos generados dentro del proyecto reduce su utilidad como plataforma experimental. Por ese motivo, `qiskit_interface` amplió su capa de entrada para aceptar circuitos procedentes de ficheros QASM, de forma que una ejecución pueda partir de una descripción externa y no solo de una familia predefinida.

La ampliación se concentra en las funciones de carga y normalización de `circuit_utils.py`. El módulo detecta si el fichero corresponde a OpenQASM 2 u OpenQASM 3, lee el contenido como texto y delega en los analizadores modernos de Qiskit para reconstruir un `QuantumCircuit`. A partir de ese punto, el resto del flujo trabaja con el mismo tipo de objeto que usaría para un circuito de biblioteca. Esta decisión evita que `mo_module` o `integration` tengan que distinguir entre procedencias: reciben un circuito normalizado y pueden aplicar sus propios contratos de métricas, *backend* y evaluación.

El alcance actual es intencionadamente limitado. Los escenarios `Baseline` y `MO_Only` aceptan entrada `qasm_file`, porque ambos siguen una ruta directamente apoyada en Qiskit: transpilación de referencia o evaluación de un *layout* externo mediante el flujo clásico. En cambio, las rutas `RL_Only` y `MO+RL` todavía no exponen una entrada pública QASM equivalente. Estas rutas dependen de puntos de control, trazas de episodio y reconstrucción posterior al *routing*, por lo que integrar QASM en ellas requiere fijar con más cuidado cómo se entrenan y reutilizan los modelos para circuitos externos. En la memoria, por tanto, QASM debe leerse como una ampliación real de entrada para las rutas apoyadas directamente en Qiskit, no como soporte universal para todos los escenarios.

9.3 Routing enmascarado con MaskablePPO

El *routing* por aprendizaje por refuerzo se formula como una secuencia de decisiones sobre aristas físicas del mapa de acoplamiento. En la versión no enmascarada, el agente puede seleccionar cualquier acción del espacio discreto aunque muchas de ellas sean válidas solo de forma débil o resulten poco prometedoras en el estado actual. Esa libertad aumenta el coste de exploración y facilita trayectorias con movimientos improductivos, especialmente cuando el circuito requiere varias decisiones encadenadas para desbloquear una puerta.

El régimen de *routing* enmascarado introduce una restricción explícita de candidatos sin rediseñar el espacio de acciones. El catálogo de acciones permanece fijo sobre las aristas del `coupling_map`; lo que cambia en cada paso es la máscara devuelta por `action_masks()`. Esta máscara se calcula

a partir de la frontera visible, de la proyección física de las puertas pendientes y de señales de estabilidad del episodio. De este modo, *MaskablePPO* puede entrenar una política que solo muestrea acciones habilitadas en el estado actual, manteniendo índices estables para los puntos de control y compatibilidad con el contrato general del entorno.

La evolución de este régimen se conserva mediante metadatos versionados. Esta decisión es importante porque los puntos de control entrenados con semánticas distintas no deben evaluarse como si compartieran exactamente el mismo comportamiento. La Tabla 9.1 resume la progresión de versiones y el tipo de control que añade cada una.

Versión	Aportación principal	Efecto sobre el <i>routing</i>
v1	Semántica histórica de <i>routing</i> enmascarado.	Conserva la evaluación de los primeros puntos de control con máscara, sin reinterpretar sus decisiones bajo filtros posteriores.
v2	Filtro anti-undo.	Evita deshacer inmediatamente el último SWAP , reduciendo ciclos locales triviales entre dos estados consecutivos.
v3	Filtro anti-ciclo, estancamiento y top-k SABRE opcional.	Detecta ciclos sobre estado y revisión de frontera, permite terminar episodios estancados mediante stagnation_patience , usa cycle_window y conserva mecanismos de respaldo cuando un filtro vaciaría la máscara.
v4	Decaimiento SABRE.	Penaliza la reutilización serial de los mismos cúbits físicos, favoreciendo rutas que distribuyen mejor los movimientos cuando hay alternativas comparables.
v5	Aristas preparatorias a un salto.	Mantiene las mejoras de v4 y permite conservar candidatos preparatorios alrededor de la frontera bloqueada, incluidos candidatos no productivos hasta k para no cerrar rutas útiles demasiado pronto.

Tabla 9.1. Evolución de las versiones de *routing* enmascarado usadas por puntos de control *MaskablePPO*.

El efecto conceptual de estas versiones es desplazar parte del control desde la recompensa hacia la factibilidad operativa del episodio. La recompensa sigue siendo necesaria para valorar progreso, puertas ejecutadas, penalización de **SWAPs** y completitud, pero la máscara evita que el entrenamiento desperdicie demasiados pasos en acciones claramente dominadas. Aun así, el régimen no debe interpretarse como una garantía de optimalidad. Es una capa heurística y versionada que mejora la estabilidad práctica de los experimentos nuevos, mientras que los puntos de control históricos de PPO, DQN y versiones anteriores permanecen evaluables bajo su contrato original. En este contexto, **top-k** significa conservar solo los k candidatos mejor puntuados por el criterio SABRE cuando esa poda está activada.

9.4 Modo *synthesis*

El modo *synthesis* amplía `rl_module` hacia un problema distinto del *routing*. En *routing*, el circuito objetivo ya existe y el agente decide qué *SWAPs* aplicar para que las puertas pendientes sean ejecutables sobre una conectividad limitada. En *synthesis*, en cambio, el agente no desbloquea una cola de puertas mediante movimientos de cúbits. Su acción consiste en aplicar primitivas nativas de un catálogo discreto para construir progresivamente un circuito equivalente al objetivo.

Esta diferencia obliga a cambiar la semántica de la observación y de la recompensa. El progreso no se mide por puertas ejecutadas ni por distancia de *routing* de la frontera, sino por la reducción de un residual Clifford. El entorno mantiene un estado que compara el circuito construido con el objetivo y considera completado el episodio cuando ese residual llega a la identidad. Por ello, *synthesis* reutiliza la infraestructura de entorno, estrategias, agente, entrenamiento e inspección, pero no comparte la interpretación operativa del *routing*.

El ejemplo Clifford de la Figura 5.4 ayuda a situar esta ampliación: el objetivo ya es un circuito completo, y el agente debe aproximarlos mediante primitivas nativas en lugar de desplazar cúbits para desbloquear una frontera de puertas.

El alcance actual de esta ampliación es acotado. Solo se consideran circuitos Clifford y las `basis_gates` son obligatorias, porque la conectividad no determina por sí sola las primitivas nativas disponibles. Además, el *layout* permanece fijo durante el episodio y `MaskablePPO` no se presenta como algoritmo general para *synthesis*; su uso queda ligado al *routing* enmascarado. En consecuencia, este modo debe entenderse como una prueba de extensibilidad de `rl_module` y como una línea funcional prometedora, pero no como parte del flujo principal de *integration* ni como una síntesis cuántica general.

9.5 Modelo de ajuste para optimización multiobjetivo

La calidad de `mo_module` depende tanto de los objetivos elegidos como de los hiperparámetros que gobiernan la búsqueda evolutiva. Tamaño de población, número de generaciones, operador de cruce y probabilidades de mutación pueden cambiar el frente de Pareto obtenido y, por tanto, el *layout* seleccionado después por *integration*. Para estudiar esa sensibilidad se incorporó `LayoutTuner`, una capa de ajuste basada en Optuna. Las semillas no forman parte del espacio de búsqueda de hiperparámetros. Se fijan para controlar la reproducibilidad y, cuando procede, se repite la evaluación con varias semillas para estimar la estabilidad de la configuración.

La herramienta de ajuste de hiperparámetros no sustituye al análisis experimental posterior ni

a la optimización multiobjetivo; únicamente ayuda a seleccionar configuraciones razonables del optimizador bajo un criterio común. Su papel es ejecutar varias configuraciones de `OptimizerConfig`, evaluar los frentes resultantes y asignar una puntuación comparable a cada prueba. Esa puntuación se basa en el hipervolumen, una métrica habitual para valorar la extensión y calidad de un frente de Pareto. La decisión relevante del diseño es fijar un `ref_point` de sesión: todas las pruebas de una misma sesión se comparan contra el mismo punto de referencia, evitando que cada frente redefina su propia escala y haga menos interpretable la puntuación.

El modo `calibrated` ejecuta una calibración previa con configuraciones ancla y construye automáticamente un punto de referencia conservador antes de iniciar las pruebas. El modo `manual`, por su parte, exige que el usuario proporcione explícitamente ese punto y evita la calibración previa. En ambos casos, una prueba puede evaluarse sobre varias semillas y la puntuación final se obtiene agregando los hipervolumenes resultantes. Esta ampliación aporta una forma más sistemática de justificar configuraciones de `mo_module`, especialmente cuando se quieren comparar campañas o estudiar si una mejora procede de la estrategia híbrida o de una parametrización concreta de la búsqueda.

9.6 Estrategias avanzadas de selección híbrida

Las rutas `MO_Only` y `MO+RL` necesitan reducir un frente de Pareto a un *layout* concreto. Los modos históricos, como `compromise`, `best_depth` y `best_cnot_count`, resuelven esa elección con criterios definidos sobre las métricas de la optimización y de la transpilación. Las ampliaciones `hybrid_probe` y `rl_guided` exploran una idea más ambiciosa: usar información de *routing* para decidir qué candidato del frente resulta más interesante en una ejecución híbrida. SABRE es una heurística clásica de *routing* que guía la inserción de **SWAPs** para adaptar el circuito a la conectividad física del *backend*.

9.6.1 `hybrid_probe`

`hybrid_probe` es un modo opcional de campaña que evalúa *layouts* del frente mediante una heurística determinista de preevaluación inspirada en SABRE. Su objetivo no es sustituir al entrenamiento RL, sino estimar qué *layouts* parecen facilitar un *routing* completo antes de elegir el candidato final. Primero deduplica *layouts* físicos equivalentes para no repetir trabajo experimental. Después ejecuta una estimación de *routing* sobre cada candidato y reconstruye las soluciones que completan. Cuando existe una solución válida, la selección se realiza mediante la tupla (`cnot_equivalent`, `depth`, `swaps`), que está más alineada con las métricas finales del estudio que una decisión basada solo en la geometría del frente MO.

El modo conserva un control diagnóstico con el *layout* inicial de Qiskit, pero no lo utiliza como candidato MO seleccionable. Si ninguna preevaluación sobre *layouts* MO completa el *routing*, registra el incidente y activa de forma explícita el modo *compromise* como mecanismo de respaldo. Esta política evita que una ampliación experimental convierta una campaña en un resultado opaco. Además, *hybrid_probe* requiere *MaskablePPO*, porque su semántica de candidatos y evaluación está ligada al régimen de *routing* enmascarado.

9.6.2 *rl_guided*

rl_guided lleva la idea un paso más allá. En lugar de usar una heurística determinista de preevaluación, ejecuta una segunda búsqueda multiobjetivo en la que cada *layout* candidato se evalúa con un punto de control *RL_Only* congelado. Los episodios completos se reconstruyen y se valoran con métricas posteriores al *routing*; los episodios incompletos o erróneos se penalizan con valores dominados y quedan fuera de la selección válida. Si no aparece ningún candidato que complete *routing*, el modo falla de forma controlada antes de continuar el entrenamiento.

El *layout* seleccionado por esta segunda búsqueda puede utilizarse después para continuar el entrenamiento con el presupuesto adicional *rl.finetune_timesteps*. El modo exige campaña avanzada, *MaskablePPO* y una configuración sintética controlada para mantener estable el significado de las acciones durante preentrenamiento, búsqueda, ajuste fino y evaluación. Por ello se mantiene como mecanismo experimental de selección híbrida, separado de los escenarios base *Baseline*, *MO_Only*, *RL_Only* y *MO+RL*.

9.7 Subgrafos de *routing*

Las campañas híbridas pueden entrenar y evaluar RL sobre el *coupling_map* completo del *backend*, pero esa decisión no siempre refleja la parte del *hardware* que realmente interviene en un caso. Si un circuito solo contiene ciertos pares lógicos de interacción y el *layout* seleccionado sitúa esos cúbits sobre zonas concretas del *backend*, gran parte del grafo físico puede ser irrelevante para resolver ese *routing*. La ampliación de subgrafos aborda este problema derivando una conectividad más ajustada al caso.

La función de derivación identifica primero los pares lógicos que participan en puertas de dos cúbits. Después proyecta esos pares sobre los cúbits físicos fijados por el *layout* seleccionado y busca caminos mínimos en el grafo físico del *backend*. El subgrafo resultante contiene las aristas necesarias para conectar esas interacciones, incluyendo cúbits intermedios cuando los extremos no son adyacentes. La Figura 9.1 resume este flujo.

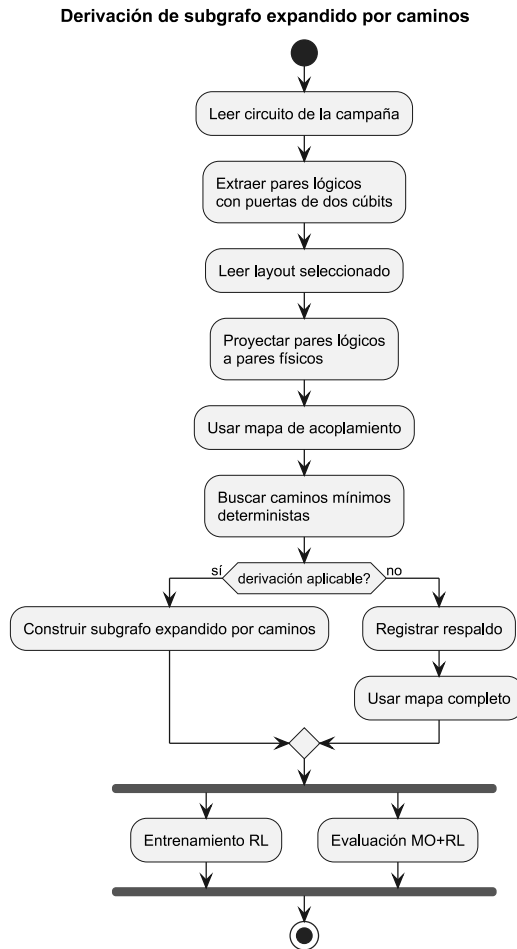


Figura 9.1. Derivación de un subgrafo de routing expandido por caminos dentro de una campaña híbrida.

El resultado no cambia el contrato público de los escenarios aislados. Es una decisión propia de campaña y se aplica de forma simétrica: el entrenamiento RL y la evaluación MO+RL usan el mismo subgrafo derivado. Esta simetría evita que el agente aprenda sobre una conectividad y se evalúe sobre otra distinta. También permite registrar metadatos útiles, como número de nodos, número de aristas, pares interactuantes y cúbits intermedios añadidos.

La derivación conserva un mecanismo de respaldo explícito. Si el circuito no contiene pares de dos cúbits, si falta algún camino en el *backend* o si el subgrafo derivado queda vacío, la campaña usa el `coupling_map` completo y deja constancia del motivo. Esta política es importante para no convertir una optimización de conectividad en una fuente silenciosa de errores experimentales. En términos de responsabilidades, `mo_module` sigue produciendo *layouts* y `rl_module` sigue ejecutando episodios; la decisión de reducir o no el grafo de *routing* pertenece a *integration*.

9.8 Ampliaciones de campañas y trazabilidad

La matriz experimental de campaña agrupa varias semillas y varios criterios de selección sobre una misma base experimental. Para cada caso puede ejecutarse una sola vez *Baseline*, entrenarse la rama *RL_Only*, generarse un frente MO compartido y seleccionar *layouts* mediante distintos modos. Si dos modos seleccionan el mismo *layout* físico, la campaña reutiliza la rama correspondiente en lugar de repetir un entrenamiento equivalente.

La trazabilidad se refuerza mediante artefactos persistidos. `summary.md` resume la configuración, las métricas y las incidencias en un formato legible. `campaign.json` conserva la salida estructurada completa y cada caso mantiene su propio `result.json`. Las ampliaciones avanzadas añaden artefactos específicos, como `hybrid_layout_probe.json` para la preevaluación híbrida y `rl_guided_mo.json` para la búsqueda guiada por RL. Estos ficheros permiten reconstruir qué *layout* se eligió, qué candidatos se descartaron, qué mecanismo de respaldo se activó y qué punto de control intervino.

Un caso puede terminar su ejecución técnica y, aun así, no disponer de métricas homogéneas para los cuatro escenarios. Por ello, el sistema de informes distingue casos completos, incompletos, fallidos y no comparables. Esta separación evita que las rutas RL incompletas se mezclen con resultados posteriores al *routing* válidos. Además, las notas de campaña registran configuraciones efectivas de máscara, paciencia de estancamiento, selección de puntos de control y modo de grafo de *routing*.

10

Validación y diseño experimental

10.1 Preguntas de investigación e hipótesis

La validación experimental de este trabajo se plantea como una comparación controlada entre cuatro escenarios de transpilación: *Baseline*, *MO_Only*, *RL_Only* y *MO+RL*. El objetivo no es demostrar una superioridad universal de una técnica sobre otra, sino estudiar bajo qué condiciones cada componente aporta valor y qué limitaciones aparecen cuando se combinan optimización multiobjetivo y aprendizaje por refuerzo en un flujo de *routing*.

La primera pregunta de investigación (**RQ1**) es si la optimización multiobjetivo obtiene *layouts* iniciales que mejoren las métricas estructurales de la transpilación frente a la referencia de Qiskit. En particular, interesa observar si *MO_Only* reduce la profundidad del circuito o el número de puertas de dos cúbits respecto a *Baseline*. La hipótesis asociada es que la búsqueda multiobjetivo puede encontrar asignaciones competitivas cuando la topología impone restricciones relevantes, aunque no necesariamente dominará en todos los casos ni en todas las métricas a la vez.

La segunda pregunta (**RQ2**) se centra en el efecto del aprendizaje por refuerzo cuando se entrena por caso. En *RL_Only*, el agente aprende una política de *routing* para una combinación concreta de circuito y topología. La hipótesis es que este entrenamiento puede producir rutas útiles en casos acotados, pero su rendimiento dependerá de la dificultad del circuito, del tamaño de la topología y de la estabilidad de la política aprendida. Por tanto, el resultado de RL debe interpretarse junto con indicadores de completitud, truncado y número de pasos, no solo mediante las métricas finales del circuito reconstruido.

La tercera pregunta (**RQ3**) estudia la combinación híbrida MO+RL. La expectativa inicial es que un *layout* seleccionado por `mo_module` pueda facilitar el *routing* posterior del agente. Sin embargo, existe una discrepancia metodológica importante: el *layout* de MO se evalúa mediante la transpilación base de Qiskit, mientras que el agente RL optimiza una dinámica secuencial distinta, guiada por observaciones, recompensas y restricciones de acción. Por ello, un *layout* favorable para Qiskit no tiene por qué ser igualmente favorable para RL.

Por último, el diseño experimental pregunta (**RQ4**) cómo varían estas relaciones al cambiar la familia de circuito, el número de cúbits y la topología sintética. La hipótesis es que los circuitos `ghz` y `qft` ejercen presiones distintas sobre el *routing*: `ghz` tiene una estructura más lineal y localizada, mientras que `qft` introduce una interacción más densa entre cúbits. Del mismo modo, las topologías `ring` y `t` ofrecen restricciones de conectividad diferentes y permiten evaluar si las conclusiones dependen de una forma física concreta.

10.2 Circuitos de prueba

Como se describió en la Sección 5.2, las familias `ghz` y `qft` representan patrones de interacción diferentes dentro de la biblioteca interna de `qiskit_interface`. El banco de pruebas principal utiliza ambas familias, generadas de forma programática como `QuantumCircuit`, lo que evita depender de ficheros externos y facilita reproducir el mismo conjunto de casos en todas las campañas experimentales.

La familia `ghz` representa un circuito estructurado de entrelazamiento. Su construcción combina una puerta de Hadamard inicial con una cadena de puertas `CX`, produciendo un patrón de dependencias sencillo de interpretar. Esta familia de circuitos resulta adecuada para estudiar escenarios estructurados, porque concentra el coste en una cadena de interacciones fácilmente interpretable.

La familia `qft`, por el contrario, introduce un patrón más denso de interacciones controladas. Aunque sigue siendo una familia canónica y reproducible, su estructura genera más presión sobre el proceso de *routing* a medida que aumenta el número de cúbits. Por este motivo, `qft` permite observar mejor cómo escalan las decisiones de *layout* y *routing* cuando el circuito contiene relaciones menos locales.

Para cada familia se consideran tres tamaños: 5, 8 y 11 cúbits. Estos valores ofrecen una progresión gradual. El tamaño de 5 cúbits permite inspeccionar casos pequeños y reduce el coste de entrenamiento; 8 cúbits introduce una complejidad intermedia; y 11 cúbits aumenta el espacio de búsqueda y la dificultad de *routing* sin abandonar un rango manejable para ejecutar el conjunto de pruebas repetidas. En conjunto, la matriz de circuitos contiene las tres versiones de `ghz` y las tres

versiones de qft.

10.3 Topologías sintéticas seleccionadas

En el protocolo experimental de esta memoria se descartan los *fake backends* de Qiskit y se emplean exclusivamente topologías sintéticas. Esta decisión responde a un criterio metodológico: estos *backends* simulados reproducen estructuras y metadatos inspirados en dispositivos reales, pero también introducen heterogeneidad adicional en tamaño, conectividad, puertas nativas y propiedades simuladas. Para un banco de pruebas centrado en comparar escenarios de *routing*, esa riqueza puede dificultar atribuir las diferencias observadas a la técnica evaluada. Este enfoque también sigue la línea de trabajos recientes que utilizan topologías controladas para aislar el efecto de la conectividad en estudios de *routing* y transpilación [21, 24].

Las topologías sintéticas permiten controlar el número de cúbits físicos y la forma de la conectividad. En todos los casos se usa el mismo número de cúbits físicos que cúbits lógicos del circuito, y se comparan dos formas: *ring* y *t*. La Figura 10.1 muestra ambas topologías para los tres tamaños utilizados en el protocolo. La topología *ring* conecta los cúbits formando un ciclo, de modo que cada nodo mantiene conectividad local con dos vecinos. Esta estructura es regular y simétrica, por lo que ofrece un banco de pruebas homogéneo para observar decisiones de *routing* sin zonas privilegiadas.

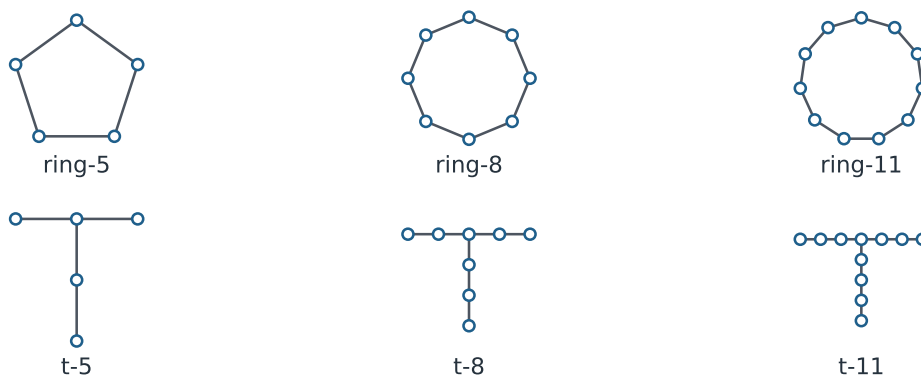


Figura 10.1. Topologías sintéticas *ring* y *t* utilizadas en el protocolo experimental para 5, 8 y 11 cúbits.

La topología *t* introduce una forma menos uniforme. Su estructura contiene una barra y un tallo, lo que genera regiones con distinto papel dentro del grafo. Esta asimetría permite comprobar si las estrategias de *layout* y *routing* se adaptan a topologías donde algunos cúbits ocupan posiciones más centrales que otros. Al comparar *ring* y *t* con los mismos circuitos y tamaños, el estudio puede separar mejor el efecto de la conectividad del efecto propio de la familia de circuito.

El uso de topologías sintéticas también mejora la homogeneidad del protocolo. Todas las combinaciones comparten una base de puertas sintética compatible con CX, y la diferencia principal entre casos queda concentrada en la forma del *coupling map*. Esta simplificación reduce el realismo *hardware*, pero refuerza la claridad de la validación experimental.

10.4 Métricas e indicadores

Como se detalló en capítulos previos, las métricas principales del estudio son la profundidad del circuito transpilado y el número de puertas de dos cúbits, referido en los artefactos como `depth` y `cnot_count`. La profundidad aproxima la longitud crítica del circuito resultante y afecta a la exposición temporal a errores en *hardware* real. Las puertas de dos cúbits son especialmente relevantes porque, frente a las puertas de un cúbit, introducen un error mayor y porque el *routing* tiende a añadir SWAPs, que se descomponen en operaciones de dos cúbits adicionales.

Aunque el tiempo de ejecución es un indicador importante, y en la práctica limita el tamaño de la experimentación, no se usa como métrica principal porque las ejecuciones se realizaron bajo condiciones de *hardware* distintas y su comparación directa no sería justa. Por ello, se interpreta solo como indicador orientativo de viabilidad. A modo de referencia, las combinaciones más ligeras, con 5 cúbits y los cuatro modos comparables, podían requerir entre tres y cinco minutos para completar la ejecución agregada, mientras que configuraciones más exigentes, como `qft` de 11 cúbits con rutas basadas en RL, podían alcanzar varias horas y llegar aproximadamente a ocho horas.

Además de las métricas estructurales comunes, las rutas basadas en RL requieren indicadores propios. Un episodio puede completar el *routing*, truncarse o quedar incompleto. Cuando completa, `integration` puede reconstruir el circuito ruteado y delegar las etapas posteriores en Qiskit, obteniendo métricas comparables con `Baseline` y `MO_Only`. Cuando no completa, el resultado sigue siendo informativo para analizar el comportamiento del agente, pero no debe mezclarse con los agregados de circuitos finales como si fuera una solución válida.

Por este motivo, la comparación homogénea entre los cuatro escenarios se reserva a los casos donde todas las rutas disponen de métricas finales comparables. Los indicadores de recompensa, pasos, *swaps*, completitud y truncado se analizan como contexto explicativo de `RL_Only` y `MO+RL`, mientras que `depth` y `cnot_count` constituyen el núcleo cuantitativo de la comparación entre escenarios.

10.5 Protocolo experimental

La matriz experimental principal cruza dos familias de circuito, tres tamaños y dos topologías sintéticas. Para cada combinación se ejecutan diez semillas, del 1 al 10, y se reporta la media de las métricas obtenidas. Esta repetición permite reducir el peso de variaciones asociadas a la búsqueda evolutiva, al entrenamiento RL y a la selección de soluciones dentro del frente multiobjetivo. La Figura 10.2 resume esta composición factorial y el número de combinaciones base resultante.

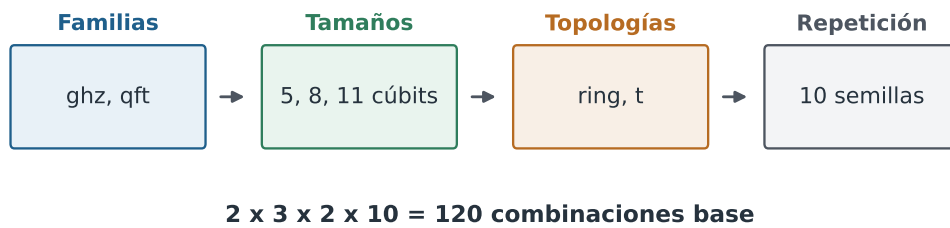


Figura 10.2. Composición de la matriz experimental principal.

En cada combinación se evalúan ocho ejecuciones conceptuales, desglosadas en la Figura 10.3. La primera es *Baseline*, que actúa como referencia de Qiskit. La segunda es *RL_Only*, donde el agente se entrena y se evalúa sin recibir un *layout* procedente de MO. Las otras seis proceden de los tres modos de selección multiobjetivo: *compromise*, *best_depth* y *best_cnot_count*. Para cada modo se ejecuta *MO_Only*, que evalúa el *layout* con Qiskit, y *MO+RL*, que utiliza ese *layout* como punto de partida para el entrenamiento y la evaluación de RL.

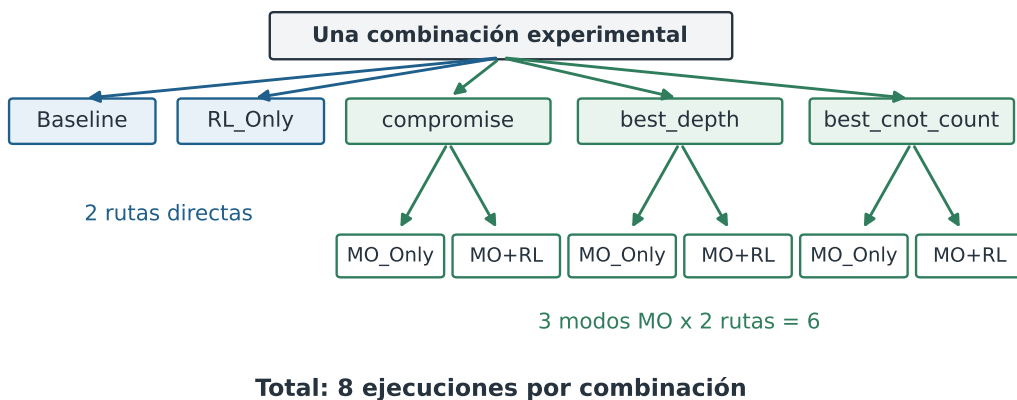


Figura 10.3. Desglose de las ocho ejecuciones evaluadas para cada combinación experimental.

La Tabla 10.1 recopila los factores del protocolo para dejar explícitos los valores que definen cada campaña y las métricas usadas en la comparación.

Factor	Valores
Familias de circuito	ghz, qft
Tamaños	5, 8 y 11 cúbits
Topologías sintéticas	ring, t
Semillas	10 semillas por combinación
Escenarios base	Baseline, RL_Only
Modos MO	compromise, best_depth, best_cnot_count
Rutas por modo MO	MO_Only, MO+RL
Métricas principales	depth, cnot_count

Tabla 10.1. Factores principales del protocolo experimental.

El entrenamiento RL se realiza de forma secuencial para cada combinación. No se reutilizan modelos preentrenados entre circuitos, tamaños, topologías o semillas. Esta decisión limita la ambición generalista del agente, pero mantiene una interpretación clara: cada política evaluada corresponde al caso concreto que se está midiendo. El algoritmo utilizado es `MaskablePPO`, seleccionado porque en las pruebas preliminares ofreció mejores resultados y tiempos de entrenamiento sensiblemente menores que las alternativas consideradas. La frontera visible se configura como `dag`; con ello el agente observa la capa frontal del DAG del circuito y no solo una cola lineal de puertas pendientes, lo que se ajusta mejor a la estructura de dependencias del circuito. La tasa de aprendizaje se fija en 0.0001 para favorecer actualizaciones estables en entrenamientos largos. `clip_range` se establece en 0.1 para limitar cambios bruscos de la política entre iteraciones, y `target_kl` se fija en 0.03 como control adicional frente a desviaciones grandes entre la política anterior y la política actualizada.

La elección de estos parámetros se apoyó en pruebas preliminares informales orientadas a comprobar la viabilidad de cada configuración. En esas pruebas se buscó evitar presupuestos insuficientes que truncaran sistemáticamente los episodios, pero también mantener tiempos de ejecución asumibles para repetir la matriz experimental completa.

Los demás hiperparámetros regulan el tamaño del episodio y la información local disponible para decidir. `max_steps` define el número máximo de acciones de *routing* antes de truncar un episodio, por lo que debe crecer cuando aumenta el tamaño del circuito y el número esperado de decisiones. La ventana de *lookahead* determina cuántas puertas futuras se incluyen en la observación; una ventana mayor aporta más contexto, pero también aumenta la complejidad de la señal que recibe el agente. Por último, `sabre_top_k` limita el conjunto de candidatos inspirado en SABRE que se consideran

en cada estado, actuando como un compromiso entre diversidad de acciones y coste de exploración. Para *qft*, estos presupuestos se escalan con el tamaño del circuito. En 5 cúbits se emplean 125000 pasos de entrenamiento, *max_steps* igual a 100, ventana de *lookahead* igual a 15 y *sabre_top_k* igual a 4. En 8 cúbits se usan 500000 pasos de entrenamiento, *max_steps* igual a 150, *lookahead* igual a 15 y *sabre_top_k* igual a 4. En 11 cúbits se aumenta el presupuesto hasta 2000000 pasos, *max_steps* igual a 300, *lookahead* igual a 25 y *sabre_top_k* igual a 5. Estos valores proceden de las configuraciones de campaña empleadas para los circuitos *qft* y reflejan que esta familia introduce interacciones más densas conforme crece el número de cúbits.

Para *ghz*, se fija un presupuesto RL común de 75000 pasos de entrenamiento en todas las combinaciones. Esta elección se debe a que, dentro de este banco de pruebas, dicho presupuesto resultó suficiente para alcanzar convergencia. Mantenerlo constante evita introducir diferencias artificiales de presupuesto entre tamaños y topologías en una familia de circuito más estructurada.

La optimización multiobjetivo se ejecuta con los objetivos *depth* y *cnot_count*. En todos los casos se evalúan los tres modos históricos de selección. *compromise* representa una solución equilibrada del frente de Pareto; *best_depth* prioriza la menor profundidad; y *best_cnot_count* prioriza el menor número de puertas de dos cúbits. En *qft*, se emplean población 80 y 80 generaciones para 5 y 8 cúbits, y población 150 con 150 generaciones para 11 cúbits. En *ghz*, se usan población 50 y 50 generaciones para todas las combinaciones.

El protocolo no incluye modos avanzados de selección híbrida dentro de la comparación principal. Aunque forman parte de las ampliaciones del sistema, su semántica introduce estrategias adicionales que dificultarían una lectura homogénea del banco de pruebas base. La validación se centra, por tanto, en los cuatro escenarios comparables y en los tres criterios de selección MO ya descritos.

10.6 Amenazas a la validez

La primera amenaza a la validez externa procede del alcance deliberadamente acotado del entrenamiento RL. Los modelos se entrenan para una combinación concreta de circuito, topología y semilla. En consecuencia, los resultados no deben interpretarse como evidencia de que una política aprendida generaliza a familias de circuitos no vistas, a otros tamaños o a *hardware* real. Esta decisión responde a un compromiso metodológico: se prefiere un banco de pruebas simple y controlado antes que un protocolo más amplio pero menos interpretable.

La segunda amenaza afecta a la validez constructiva de las métricas. *depth* y *cnot_count* son indicadores adecuados para comparar coste estructural del circuito, pero no agotan la calidad de una transpilación cuántica. En *hardware* real también intervienen tasas de error, tiempos de coherencia,

calibraciones y fidelidades específicas. Al utilizar topologías sintéticas y descartar *fake backends*, el estudio gana homogeneidad, pero renuncia a capturar parte de esa variabilidad física.

La tercera amenaza se relaciona con la interacción entre MO y RL. La optimización multiobjetivo selecciona *layouts* usando evaluaciones apoyadas en la transpilación de Qiskit. El agente RL, en cambio, opera mediante una secuencia de decisiones y una función de recompensa propia. Por tanto, una solución buena para el transpilador base puede no ser el mejor punto de partida para el agente. Esta divergencia es especialmente importante al interpretar MO+RL: una ausencia de mejora no implica necesariamente que MO no encuentre buenos *layouts*, sino que esos *layouts* pueden no estar alineados con la dinámica aprendida por RL.

Otra amenaza procede del tamaño del banco de pruebas. Aunque se utilizan diez semillas por combinación, el conjunto de familias se limita a *ghz* y *qft*, y los tamaños se restringen a 5, 8 y 11 cúbits. Esta cobertura permite estudiar tendencias iniciales y comparar comportamientos bajo condiciones controladas, pero no justifica conclusiones generales sobre todos los circuitos cuánticos ni sobre todas las topologías posibles.

También existe una amenaza asociada a la agregación. La media por semillas resume el comportamiento global de una combinación, pero puede ocultar ejecuciones fallidas, episodios truncados o variabilidad entre semillas. Por ello, el análisis de resultados debe acompañar las medias con la información de completitud y, cuando proceda, con medidas de dispersión. Esta precaución es especialmente relevante para las rutas RL, donde el entrenamiento puede producir políticas con comportamientos diferentes incluso bajo el mismo presupuesto.

Finalmente, la exclusión de *fake backends* tiene un doble efecto. Por un lado, fortalece la validez interna al mantener constante el tipo de conectividad y evitar propiedades *hardware* heterogéneas. Por otro lado, reduce la validez externa respecto a dispositivos realistas. La decisión es adecuada para un capítulo de validación metodológica centrado en comparar técnicas, pero los resultados deben leerse como evidencia sobre un entorno experimental sintético, no como una evaluación directa de rendimiento en *hardware* cuántico real.

11

Análisis experimental y discusión

11.1 Criterios de lectura de los resultados

El análisis de resultados se organiza siguiendo la matriz experimental definida en el capítulo anterior: familia de circuito, número de cúbits, topología sintética y escenario de transpilación. Esta estructura evita presentar los resultados como una evaluación aislada de módulos y los sitúa en la condición experimental que realmente se ejecutó. En cada combinación se consideran las topologías `ring` y `t`, diez semillas y los escenarios `Baseline`, `MO_Only`, `RL_Only` y `MO+RL`. Para los caminos que dependen de optimización multiobjetivo, se distinguen los modos `compromise`, `best_depth` y `best_cnot_count`.

Las tablas de cada subapartado reportan medias agregadas sobre las diez semillas disponibles. Las métricas principales son `trans_depth` y `trans_cnot_equivalent`, que corresponden respectivamente a la profundidad y al coste equivalente en puertas de dos cúbits del circuito final evaluado tras la transpilación o reconstrucción posterior al *routing*. En los agregados incorporados aquí, los resúmenes de campaña registran diez casos comparables y cero ejecuciones fallidas, incompletas o canceladas por modo de selección. Por tanto, los valores de `RL_Only` y `MO+RL` de estas tablas se comparan de forma homogénea con los escenarios `Baseline` y `MO_Only`.

Esta distinción es importante porque un episodio de aprendizaje por refuerzo no equivale automáticamente a un circuito final comparable. Cuando el *routing* no completa, los indicadores internos del episodio siguen siendo útiles para diagnosticar al agente, pero no deben mezclarse con medias de calidad de circuito. En las matrices incorporadas en este capítulo no aparece esta restricción prác-

tica, ya que todos los agregados principales reportados son comparables. A diferencia del capítulo de validación, aquí no se introducen figuras nuevas: la evidencia cuantitativa se concentra en las tablas referenciadas explícitamente en cada subapartado.

11.2 Resultados en circuitos GHZ

Los circuitos *ghz* representan la familia más estructurada del banco de pruebas. Su patrón de entrelazamiento se aproxima a una cadena, por lo que se espera que las topologías sintéticas no impongan una presión de *routing* tan severa como en *qft*. Los resultados confirman esta lectura general: en *ring*, todos los escenarios producen exactamente las mismas medias para los tres tamaños, mientras que en *t* aparecen diferencias pequeñas, sobre todo cuando el circuito crece.

11.2.1 GHZ de 5 cúbits

La Tabla 11.1 resume el primer tamaño de la familia *ghz* y sirve como referencia para leer la evolución posterior.

Top.	Modo MO	trans_depth				trans_cnot_equivalent			
		Base	RL	MO	MO+RL	Base	RL	MO	MO+RL
ring	compromise	7.00	7.00	7.00	7.00	4.00	4.00	4.00	4.00
ring	best_depth	7.00	7.00	7.00	7.00	4.00	4.00	4.00	4.00
ring	best_cnot_count	7.00	7.00	7.00	7.00	4.00	4.00	4.00	4.00
t	compromise	8.00	9.40	8.00	8.60	5.00	6.40	5.00	5.60
t	best_depth	8.00	9.40	8.00	8.60	5.00	6.40	5.00	5.60
t	best_cnot_count	8.00	9.40	8.00	8.60	5.00	6.40	5.00	5.60

Tabla 11.1. Resultados agregados para *ghz* de 5 cúbits, separados por topología y modo de selección MO.

En 5 cúbits, *ghz* muestra un comportamiento muy estable. Sobre *ring*, todos los escenarios alcanzan profundidad media 7.00 y coste equivalente de dos cúbits 4.00. Esta igualdad indica que la conectividad cíclica ya permite resolver la estructura del circuito sin que el *layout* inicial o el escenario basado en RL introduzcan una diferencia medible en las métricas finales.

La topología *t* introduce una restricción algo más visible. El *Baseline* y *MO_Only* mantienen profundidad 8.00 y coste 5.00 en los tres modos MO, mientras que *RL_Only* sube a 9.40 y 6.40. La combinación *MO+RL* reduce parcialmente esa diferencia, con 8.60 de profundidad y 5.60 de coste, pero no mejora al *Baseline* ni al escenario puramente multiobjetivo. En este tamaño, las métricas finales de los escenarios basados en RL quedan por encima de la solución directa.

11.2.2 GHZ de 8 cúbits

La Tabla 11.2 permite comparar el caso de 8 cúbits con el tamaño anterior sin cambiar de familia ni de topologías.

Top.	Modo MO	trans_depth				trans_cnot_equivalent			
		Base	RL	MO	MO+RL	Base	RL	MO	MO+RL
ring	compromise	10.00	10.00	10.00	10.00	7.00	7.00	7.00	7.00
ring	best_depth	10.00	10.00	10.00	10.00	7.00	7.00	7.00	7.00
ring	best_cnot_count	10.00	10.00	10.00	10.00	7.00	7.00	7.00	7.00
t	compromise	14.00	15.20	13.90	15.30	11.00	12.20	12.30	13.40
t	best_depth	14.00	15.20	13.70	15.10	11.00	12.20	14.30	15.40
t	best_cnot_count	14.00	15.20	14.00	14.60	11.00	12.20	11.00	11.60

Tabla 11.2. Resultados agregados para *ghz* de 8 cúbits, separados por topología y modo de selección MO.

En 8 cúbits se conserva la misma pauta en *ring*: todos los escenarios producen profundidad 10.00 y coste 7.00. La familia *ghz* sigue estando bien alineada con la conectividad cíclica, por lo que la comparación de la Tabla 11.2 no muestra margen práctico para que *MO_Only* o *MO+RL* destaquen.

La topología *t* genera una separación mayor. El *Baseline* obtiene profundidad 14.00 y coste 11.00. *MO_Only* baja ligeramente la profundidad en *compromise* y *best_depth*, pero este último modo aumenta de forma apreciable el coste equivalente. Cuando se prioriza *best_cnot_count*, *MO_Only* conserva el coste 11.00 sin reducir profundidad. Al igual que en 5 cúbits, *RL_Only* queda por encima de la referencia en la topología *t*, lo que indica que para esta familia estructurada el agente no compensa el coste adicional introducido por sus decisiones de *routing*. *MO+RL* reduce esos valores respecto a *RL_Only* en el modo orientado a coste, pero queda por encima de *Baseline* y de *MO_Only*.

11.2.3 GHZ de 11 cúbits

La Tabla 11.3 cierra la lectura por tamaños de *ghz* y permite comprobar si las diferencias detectadas en *t* se amplifican al crecer el circuito.

En 11 cúbits, *ring* vuelve a producir igualdad completa entre escenarios: profundidad 13.00 y coste 10.00. La regularidad de la topología y la estructura de *ghz* siguen evitando que aparezcan diferencias relevantes, como muestra la Tabla 11.3.

En *t*, el caso crece en dificultad. El comportamiento es también similar al observado en esta

topología con menos cúbits: **MO_Only** puede ajustar ligeramente una métrica concreta, pero las rutas basadas en RL no mejoran de forma estable la referencia. El **Baseline** alcanza profundidad 20.00 y coste 17.00. **MO_Only** consigue una reducción ligera de profundidad hasta 19.60 en el modo **best_depth**, pero a costa de elevar el coste equivalente. Los modos **compromise** y **best_cnot_count** conservan el coste 17.00 sin mejorar la profundidad. **RL_Only** queda en 20.60 y 17.60, y **MO+RL** no mejora la referencia en ningún modo. En este caso, las métricas finales favorecen a la ruta directa.

Top.	Modo MO	trans_depth				trans_cnot_equivalent			
		Base	RL	MO	MO+RL	Base	RL	MO	MO+RL
ring	compromise	13.00	13.00	13.00	13.00	10.00	10.00	10.00	10.00
ring	best_depth	13.00	13.00	13.00	13.00	10.00	10.00	10.00	10.00
ring	best_cnot_count	13.00	13.00	13.00	13.00	10.00	10.00	10.00	10.00
t	compromise	20.00	20.60	20.00	21.20	17.00	17.60	17.00	18.20
t	best_depth	20.00	20.60	19.60	20.90	17.00	17.60	18.20	19.60
t	best_cnot_count	20.00	20.60	20.00	21.20	17.00	17.60	17.00	18.20

Tabla 11.3. Resultados agregados para *ghz* de 11 cúbits, separados por topología y modo de selección MO.

11.2.4 Resumen de resultados en GHZ

La conclusión principal para *ghz*, apoyada en las Tablas 11.1, 11.2 y 11.3, es que la familia no ofrece un terreno donde el enfoque híbrido pueda mostrar una mejora clara. En **ring**, la solución de referencia ya es suficiente y todos los escenarios coinciden. En **t**, **MO_Only** puede aportar pequeñas mejoras de profundidad en 8 y 11 cúbits, pero estas mejoras dependen del modo de selección y pueden ir acompañadas de un mayor coste equivalente. **RL_Only** y **MO+RL** tienden a introducir coste adicional. La lectura se limita a esta familia y a estas métricas: para circuitos lineales y relativamente ordenados, la evaluación directa con Qiskit o con *layout* MO resulta más favorable que los escenarios basados en RL.

11.3 Resultados en circuitos QFT

Los circuitos *qft* presentan una estructura más densa que *ghz*. Por ello, ejercen más presión sobre el *layout* inicial y sobre las decisiones de *routing*. Los resultados de 5, 8 y 11 cúbits muestran mejoras claras frente a **Baseline** en varias condiciones, especialmente en profundidad. También revelan una

tensión importante: reducir profundidad puede aumentar el coste equivalente de puertas de dos cúbits, y el mejor escenario depende de la métrica que se priorice.

11.3.1 QFT de 5 cúbits

La Tabla 11.4 recoge el primer caso *qft*, donde ya aparece una separación mayor entre escenarios que en *ghz*.

Top.	Modo MO	trans_depth				trans_cnot_equivalent			
		Base	RL	MO	MO+RL	Base	RL	MO	MO+RL
ring	compromise	53.00	49.60	51.00	48.60	41.00	44.00	41.00	44.00
ring	best_depth	53.00	49.60	51.00	48.60	41.00	44.00	41.00	44.00
ring	best_cnot_count	53.00	49.60	51.00	48.60	41.00	44.00	41.00	44.00
t	compromise	67.00	53.50	57.00	55.90	50.00	40.70	42.20	43.40
t	best_depth	67.00	53.50	56.00	59.00	50.00	40.70	44.00	47.00
t	best_cnot_count	67.00	53.50	57.40	53.50	50.00	40.70	41.00	41.00

Tabla 11.4. Resultados agregados para *qft* de 5 cúbits, separados por topología y modo de selección MO.

En 5 cúbits, *qft* ya muestra una diferencia notable respecto a *ghz*. Sobre *ring*, el *Baseline* obtiene profundidad 53.00 y coste 41.00. *RL_Only* reduce la profundidad a 49.60, y *MO+RL* baja hasta 48.60 en los tres modos, pero ambos elevan el coste equivalente a 44.00. *MO_Only* ofrece una mejora más conservadora: profundidad 51.00 manteniendo el coste 41.00. Por tanto, si se prioriza profundidad, el escenario híbrido obtiene el mejor valor; si se prioriza coste de dos cúbits, *Baseline* y *MO_Only* son preferibles.

En la topología *t*, los escenarios basados en *RL* son más competitivos, aunque el modo *MO* elegido sí cambia la lectura. El *Baseline* parte de profundidad 67.00 y coste 50.00. *RL_Only* reduce esos valores a 53.50 y 40.70. En *MO+RL*, el modo *best_cnot_count* iguala la profundidad de *RL_Only* y queda cerca de su coste, mientras que *best_depth* empeora ambas métricas frente a *RL_Only*. *MO_Only* también mejora a *Baseline*, pero no domina a *RL* en esta configuración.

11.3.2 QFT de 8 cúbits

La Tabla 11.5 muestra el salto de complejidad al pasar a 8 cúbits en la familia *qft*.

En 8 cúbits sobre *ring*, todos los escenarios alternativos mejoran al *Baseline*. La referencia obtiene profundidad 129.70 y coste 167.30. *RL_Only* baja a 113.60 y 156.20. *MO_Only* consigue los

Top.	Modo MO	trans_depth				trans_cnot_equivalent			
		Base	RL	MO	MO+RL	Base	RL	MO	MO+RL
ring	compromise	129.70	113.60	111.60	121.00	167.30	156.20	153.50	157.70
ring	best_depth	129.70	113.60	111.00	121.20	167.30	156.20	154.10	158.90
ring	best_cnot_count	129.70	113.60	112.10	113.60	167.30	156.20	151.70	153.50
t	compromise	157.40	143.80	128.70	147.60	170.00	146.30	146.30	154.40
t	best_depth	157.40	143.80	122.80	142.70	170.00	146.30	152.00	152.90
t	best_cnot_count	157.40	143.80	137.20	143.70	170.00	146.30	143.00	144.80

Tabla 11.5. Resultados agregados para *qft* de 8 cúbits, separados por topología y modo de selección MO.

mejores valores agregados: profundidad 111.00 con *best_depth* y coste 151.70 con *best_cnot_count*. En modo *best_cnot_count*, MO+RL queda cerca de RL. En cambio, con *compromise* y *best_depth* aumenta la profundidad frente a *RL_Only*. En esta configuración, MO aporta una ventaja clara como selección de *layout* evaluada por Qiskit, mientras que el escenario híbrido mejora *Baseline*, pero no alcanza los mejores valores de *MO_Only*.

Sobre *t*, la reducción respecto a *Baseline* es todavía más marcada. La referencia produce profundidad 157.40 y coste 170.00. *RL_Only* alcanza 143.80 y 146.30. *MO_Only* destaca especialmente en profundidad con *best_depth*, que baja a 122.80, aunque su coste sube hasta 152.00. Cuando se prioriza *best_cnot_count*, *MO_Only* obtiene el menor coste equivalente, 143.00. MO+RL mejora a *Baseline* en todos los modos, pero queda por debajo de la mejor selección MO. En *qft* de 8 cúbits, el *layout* multiobjetivo por sí solo es el resultado más fuerte.

11.3.3 QFT de 11 cúbits

La Tabla 11.6 incorpora el caso de mayor tamaño de la familia *qft*. En esta campaña se mantienen diez semillas, los tres modos de selección MO y cero ejecuciones fallidas, incompletas o canceladas, por lo que las medias son comparables entre escenarios.

Sobre *ring*, el crecimiento a 11 cúbits amplifica la separación entre *Baseline* y los escenarios alternativos. La referencia obtiene profundidad 271.80 y coste 375.80. *RL_Only* reduce esos valores a 201.50 y 323.00, una mejora clara en ambas métricas. Sin embargo, el resultado más fuerte vuelve a corresponder a *MO_Only*: con *best_depth* alcanza la menor profundidad, 175.60, y con *best_cnot_count* obtiene el menor coste equivalente, 296.60. El modo *compromise* queda muy cerca de esos extremos y ofrece 178.10 de profundidad y 303.50 de coste. MO+RL también mejora a

Top.	Modo MO	trans_depth				trans_cnot_equivalent			
		Base	RL	MO	MO+RL	Base	RL	MO	MO+RL
ring	compromise	271.80	201.50	178.10	193.50	375.80	323.00	303.50	326.90
ring	best_depth	271.80	201.50	175.60	203.70	375.80	323.00	310.70	327.80
ring	best_cnot_count	271.80	201.50	184.40	196.50	375.80	323.00	296.60	319.10
t	compromise	284.00	257.70	215.70	240.60	364.10	338.90	320.30	339.80
t	best_depth	284.00	257.70	207.10	258.40	364.10	338.90	342.50	354.20
t	best_cnot_count	284.00	257.70	238.70	261.00	364.10	338.90	308.30	338.90

Tabla 11.6. Resultados agregados para *qft* de 11 cúbits, separados por topología y modo de selección MO.

Baseline, pero no conserva toda la ventaja del *layout* seleccionado por MO; en los tres modos queda por encima de MO_Only y, salvo en coste con *best_cnot_count*, no mejora a RL_Only.

En la topología *t* se observa una pauta parecida, aunque con una penalización mayor para los escenarios que pasan por RL. El Baseline parte de profundidad 284.00 y coste 364.10, mientras que RL_Only baja a 257.70 y 338.90. MO_Only vuelve a marcar los mejores valores: *best_depth* reduce la profundidad hasta 207.10, aunque con coste 342.50, y *best_cnot_count* obtiene el menor coste, 308.30, con profundidad 238.70. El modo *compromise* mantiene un equilibrio razonable, con profundidad 215.70 y coste 320.30. En cambio, MO+RL mejora a Baseline pero pierde frente a MO_Only; con *best_depth* incluso queda prácticamente alineado con RL_Only en profundidad y por encima en coste. En *qft* de 11 cúbits, por tanto, la evidencia refuerza la conclusión observada en 8 cúbits: el *layout* multiobjetivo es el componente que más reduce las métricas finales, y el paso posterior por RL no garantiza una mejora adicional.

11.3.4 Resumen de resultados en QFT

En *qft*, las técnicas evaluadas encuentran margen de mejora real frente a Baseline. En 5 cúbits, el escenario híbrido y RL_Only reducen profundidad, aunque en *ring* lo hacen aumentando el coste equivalente de dos cúbits. En *t*, *best_cnot_count* es el modo híbrido más coherente con el objetivo de coste. A partir de 8 cúbits, MO_Only se convierte en el escenario más fuerte en las dos topologías, y la campaña de 11 cúbits confirma esa pauta con más claridad: *best_depth* logra las menores profundidades y *best_cnot_count* los menores costes equivalentes. La evidencia de las Tablas 11.4, 11.5 y 11.6 apoya la utilidad de la optimización multiobjetivo para seleccionar *layouts* en circuitos densos; el valor adicional de pasar esos *layouts* por RL depende de la métrica y, en los tamaños

mayores, tiende a diluir parte de la mejora conseguida por MO.

11.4 Comparación transversal entre configuraciones

11.4.1 Efecto de la familia de circuito

La diferencia entre *ghz* y *qft* es la pauta más clara del estudio. En *ghz*, la estructura del circuito encaja bien con las topologías sintéticas, especialmente con *ring*, y los escenarios apenas se separan. En *qft*, la mayor densidad de interacciones genera una presión de *routing* más alta y abre espacio para que el *layout* inicial y los escenarios basados en RL alteren de forma visible las métricas finales. Esto confirma la hipótesis metodológica del capítulo anterior: las conclusiones sobre MO y RL dependen de la familia de circuito, no solo del tamaño.

11.4.2 Efecto del tamaño

El tamaño afecta de forma distinta a cada familia. En *ghz*, pasar de 5 a 11 cúbits incrementa las métricas de forma esperable, pero no cambia la relación entre escenarios en *ring* y solo introduce diferencias moderadas en *t*. En *qft*, el aumento de 5 a 8 cúbits produce un salto mucho más visible en profundidad y coste equivalente, y el caso de 11 cúbits consolida esa tendencia. También hace que *MO_Only* gane protagonismo: en 8 y 11 cúbits, la selección de *layout* es el mecanismo que obtiene las mejores medias agregadas en las dos topologías.

11.4.3 Efecto de la topología

La topología *ring* actúa como un entorno regular y simétrico. Para *ghz*, esa regularidad elimina prácticamente las diferencias entre escenarios. Para *qft*, *ring* sigue siendo exigente, pero permite reducciones muy amplias cuando el *layout* se optimiza, como se aprecia en 11 cúbits. La topología *t*, al ser menos uniforme, amplifica la importancia de la posición inicial de los cúbits y del *routing* posterior. Esto se observa en *ghz*, donde algunos escenarios basados en RL presentan valores superiores, y sobre todo en *qft*, donde los escenarios alternativos reducen de forma significativa las métricas de la referencia. En los tamaños mayores, no obstante, *MO+RL* acusa más esa irregularidad que *MO_Only*.

11.4.4 Efecto de los escenarios de transpilación

Baseline es una referencia sólida en *ghz*, pero deja margen de mejora en *qft*. *MO_Only* es el escenario más consistente cuando existe margen para optimizar el *layout*, especialmente en *qft* de 8 y 11 cúbits. *RL_Only* puede mejorar a *Baseline* en *qft*, aunque en *ghz* sobre *t* introduce coste

adicional. **MO+RL** muestra un comportamiento intermedio: puede mejorar a **RL_Only** en algunas configuraciones y a **Baseline** en **qft**, pero no supera de forma estable a **MO_Only**. Esta última observación es importante porque el *layout* seleccionado por **MO** se evalúa con una lógica de Qiskit, mientras que el agente **RL** optimiza una dinámica secuencial diferente. La campaña **qft** de 11 cúbits hace más visible este desajuste: el buen punto de partida de **MO** no se traduce automáticamente en una trayectoria **RL** final mejor.

11.4.5 Efecto de los modos de selección **MO**

En los casos simples de **ghz** sobre **ring**, los tres modos **MO** producen resultados indistinguibles. En configuraciones más restrictivas, el modo de selección empieza a importar. En **ghz** sobre **t**, **best_depth** puede reducir ligeramente la profundidad, pero no siempre conserva el menor coste equivalente. En **qft**, la separación es más clara: **best_depth** favorece reducciones de profundidad y **best_cnot_count** tiende a ser preferible cuando se prioriza el coste de dos cúbits. En 11 cúbits esta distinción se mantiene en las dos topologías, aunque el modo **compromise** ofrece una alternativa competitiva cuando no se desea elegir una sola métrica como criterio dominante. Por tanto, **compromise** resulta útil como punto equilibrado, pero no debe interpretarse como universalmente superior.

11.5 Discusión crítica

Los resultados disponibles responden de forma matizada a las preguntas de investigación. La optimización multiobjetivo sí aporta valor cuando el circuito y la topología dejan margen para que el *layout* inicial cambie el coste final (**RQ1**). Esto se aprecia con claridad en **qft**, especialmente en 8 y 11 cúbits, como muestran las Tablas 11.4, 11.5 y 11.6. En cambio, para **ghz** sobre **ring**, el problema queda tan alineado con la conectividad que todos los escenarios alcanzan el mismo resultado, tal como recogen las Tablas 11.1, 11.2 y 11.3.

Los escenarios basados en aprendizaje por refuerzo producen circuitos finales comparables en las matrices analizadas, aunque su ventaja no es uniforme (**RQ2**). En **qft** pueden mejorar a **Baseline**, mientras que en **ghz** sobre **t** tienden a introducir profundidad y coste adicional. La interpretación debe centrarse en las métricas finales de cada familia y en la comparación directa entre escenarios.

La combinación **MO+RL** es viable como arquitectura experimental, pero no domina de forma sistemática a **MO_Only** ni a **RL_Only** (**RQ3**). En algunos casos reduce la diferencia respecto a **RL_Only** o mejora a **Baseline**, pero en **qft** de 8 y 11 cúbits queda por debajo de **MO_Only**. La razón metodológica es coherente con el diseño del sistema: un *layout* bueno para la evaluación multiob-

jetivo basada en Qiskit no tiene por qué ser el mejor punto de partida para la dinámica secuencial del agente. Por tanto, la integración híbrida debe leerse como una base funcional para experimentar con traspasos MO hacia RL, no como una demostración cerrada de superioridad.

Las diferencias entre `ghz` y `qft`, así como entre `ring` y `t`, confirman que las conclusiones dependen de la familia, el tamaño y la topología (RQ4). La evidencia también muestra el valor de la modularidad del proyecto. La separación entre `qiskit_interface`, `mo_module`, `rl_module` e `integration` permite identificar dónde aparece cada efecto: calidad de *Baseline*, selección del *layout*, comportamiento del episodio RL y reconstrucción posterior al *routing*. Esta organización evita atribuciones excesivas y permite formular trabajos futuros concretos, como alinear mejor la selección MO con la dinámica del agente, incorporar memoria temporal en la política y ampliar la cobertura experimental hacia casos más exigentes.

12

Conclusiones y líneas futuras

12.1 Conclusiones

Este trabajo ha desarrollado y evaluado una arquitectura modular para estudiar la transpilación cuántica desde una perspectiva híbrida, combinando optimización multiobjetivo y aprendizaje por refuerzo sobre una capa de integración reproducible. La contribución principal reside en la separación clara entre responsabilidades: `qiskit_interface` proporciona circuitos, *backends*, métricas y transpilaciones de referencia; `mo_module` busca *layouts* iniciales mediante algoritmos evolutivos; `rl_module` formula el *routing* como un problema de decisiones secuenciales; e `integration` coordina escenarios, campañas y artefactos de comparación. Esta división ha permitido analizar cada componente sin confundir su papel dentro del flujo completo.

Respecto a la primera pregunta de investigación (**RQ1**), los resultados muestran que la optimización multiobjetivo puede aportar valor cuando el circuito y la topología ofrecen margen real para que el *layout* inicial influya en el coste final. Este efecto aparece con especial claridad en la familia `qft`: las Tablas 11.4, 11.5 y 11.6 muestran mejoras frente a `Baseline`, y en 8 y 11 cúbits `MO_Only` obtiene las mejores medias agregadas en varias combinaciones. En cambio, la evidencia de las Tablas 11.1, 11.2 y 11.3 indica que `ghz`, especialmente sobre `ring`, deja poco margen a la optimización porque su estructura encaja bien con la conectividad. Por tanto, la utilidad de `MO` no debe formularse como una ventaja universal, sino como una mejora dependiente de la familia de circuito, la topología y la métrica priorizada.

Respecto a la segunda pregunta de investigación (**RQ2**), el aprendizaje por refuerzo muestra un

comportamiento útil pero variable. `RL_Only` puede generar circuitos finales comparables y mejorar a `Baseline` en configuraciones exigentes, como ocurre en `qft` según las Tablas 11.4, 11.5 y 11.6. Sin embargo, también puede introducir profundidad o coste adicional en casos más estructurados, como `ghz` sobre `t`, donde las Tablas 11.1, 11.2 y 11.3 muestran que las rutas basadas en RL quedan por encima de la referencia directa. Esta variabilidad confirma que el agente no debe evaluarse solo por completar episodios, sino por el efecto que sus decisiones tienen sobre las métricas finales del circuito reconstruido. El uso de `MaskablePPO`, máscaras de acciones y frontera basada en DAG aporta estabilidad práctica, pero no elimina por completo la dependencia del entrenamiento, de la recompensa y de la geometría del caso.

Respecto a la tercera pregunta de investigación (**RQ3**), la arquitectura `MO+RL` es viable y permite estudiar la interacción entre selección de *layout* y *routing* aprendido, pero los resultados no muestran una superioridad sistemática frente a las estrategias individuales. `MO+RL` puede mejorar a `Baseline` o reducir la distancia respecto a `RL_Only` en algunos casos, como se observa en `qft` de 5 cúbits en la Tabla 11.4. No obstante, en `qft` de 8 y 11 cúbits queda por debajo de los mejores valores de `MO_Only`, recogidos en las Tablas 11.5 y 11.6. La explicación metodológica es coherente con el diseño descrito en el Capítulo 10: un *layout* seleccionado mediante evaluación con Qiskit no tiene por qué ser el mejor punto de partida para una dinámica secuencial aprendida por RL. La conclusión, por tanto, no es que la combinación mejore siempre la transpilación, sino que el traspaso entre `mo_module` y `rl_module` funciona dentro de `integration` y deja visible un desajuste que debe resolverse alineando mejor el criterio de selección con el agente que consume el *layout*.

Respecto a la cuarta pregunta de investigación (**RQ4**), las diferencias entre familias, tamaños y topologías condicionan de forma directa la lectura de los resultados. La familia `ghz` actúa como un banco de pruebas estructurado donde el margen de mejora es reducido, mientras que `qft` expone mejor las diferencias entre *layout*, *routing* y criterio de selección, tal como sintetiza la discusión del Capítulo 11. La topología `ring` tiende a ofrecer un entorno regular, y la topología `t` introduce asimetrías que hacen más visible la importancia de la posición inicial de los cúbits. Del mismo modo, los modos `best_depth` y `best_cnot_count` muestran que reducir profundidad y reducir coste equivalente de puertas de dos cúbits no siempre son objetivos compatibles. El proyecto, por tanto, permite estudiar compromisos reales en lugar de reducir la transpilación a una única métrica.

También se han identificado límites importantes. La capa `integration` debe entenderse como una orquestación de escenarios de *routing* y campañas reproducibles, no como una solución definitiva de transpilación híbrida. Los escenarios basados en RL solo producen métricas finales comparables

cuando el episodio completa y puede reconstruirse el circuito ruteado. Asimismo, las ampliaciones descritas en el Capítulo 9, como `synthesis`, `hybrid_probe` o `rl_guided`, amplían el espacio experimental, pero no sustituyen al protocolo principal evaluado. En conjunto, el trabajo ofrece una base funcional, medible y extensible para estudiar estrategias híbridas, con resultados prometedores en casos concretos y con una delimitación explícita de sus restricciones actuales.

12.2 Líneas futuras

Una primera línea de trabajo consiste en ampliar la cobertura experimental. El banco de pruebas actual ya permite observar diferencias entre familias, tamaños y topologías sintéticas, como recogen las Tablas 11.1–11.3 y 11.4–11.6, pero podría enriquecerse con más familias de circuitos, más tamaños y conectividades adicionales. Esta ampliación permitiría comprobar si las tendencias observadas se mantienen en circuitos con patrones de interacción distintos y si las mejoras de `MO_Only`, `RL_Only` o `MO+RL` dependen de estructuras concretas del circuito.

Una línea especialmente ambiciosa sería avanzar hacia un modelo RL preentrenado y condicionado por información del *backend*. En el protocolo actual, cada política se entrena para una combinación concreta de circuito, topología y semilla, lo que favorece una interpretación controlada pero limita la generalización. Un modelo de mayor alcance entrenado sobre un conjunto amplio de campañas podría aprender una política condicionada por descriptores del *backend*, del circuito y del *layout* inicial, reduciendo la necesidad de entrenar desde cero para cada caso. Esta dirección exigiría definir observaciones más generales, conjuntos de datos de entrenamiento suficientemente diversos y criterios de evaluación sobre combinaciones no vistas, de modo que la mejora midiera también la calidad de circuito bajo cambios de entrada.

Otra dirección prioritaria es mejorar la alineación entre la selección multiobjetivo y el agente de *routing*. Los modos históricos de selección, como `compromise`, `best_depth` y `best_cnot_count`, se apoyan en métricas obtenidas mediante evaluación con Qiskit. Una extensión natural es consolidar estrategias que incorporen señales más cercanas al comportamiento posterior de RL. En este sentido, `hybrid_probe` y `rl_guided` abren una vía interesante: seleccionar *layouts* no solo por su posición en el frente de Pareto, sino por su capacidad de facilitar episodios de *routing* completos y competitivos. Su maduración requeriría estudiar coste computacional, estabilidad y criterios de comparabilidad antes de incorporarlos al banco de pruebas principal.

También resulta conveniente extender el soporte de circuitos externos en la línea general del proyecto. La lectura de QASM ya aporta flexibilidad, pero una evolución deseable sería integrarla de forma más uniforme en los flujos de campaña y en la interfaz de línea de comandos (CLI). Esto per-

mitiría lanzar experimentos completos desde la CLI de *integration* a partir de circuitos definidos fuera de la biblioteca interna, manteniendo la misma trazabilidad de configuración, resultados y artefactos persistidos.

En el módulo de aprendizaje por refuerzo, una línea relevante es reducir oscilaciones y mejorar la estabilidad de evaluación. Las máscaras de acción, las penalizaciones por ciclos y el uso de frontera DAG ya actúan como mecanismos de control, pero las políticas actuales siguen siendo esencialmente no recurrentes. Explorar arquitecturas con memoria temporal, observaciones históricas más ricas o mecanismos de selección que penalicen patrones repetitivos podría ayudar a que el agente distinguiera mejor estados localmente parecidos y evitara secuencias improductivas.

El modo *synthesis* ofrece otra ampliación clara. Actualmente funciona como una prueba de extensibilidad de *rl_module* sobre circuitos Clifford, con primitivas dependientes de *basis_gates* y *layout* fijo. Un desarrollo futuro podría integrar esta capacidad en *integration*, definir campañas específicas de síntesis y estudiar cómo cambia la recompensa cuando el objetivo deja de ser desbloquear puertas mediante *SWAPs* y pasa a ser construir directamente un circuito equivalente. Esta línea debería mantenerse separada conceptualmente del *routing*, porque ambos problemas comparten infraestructura, pero no semántica.

Por último, sería valioso acercar la evaluación a condiciones de *hardware* más realistas cuando el objetivo deje de ser solo controlar el protocolo. Las topologías sintéticas han sido adecuadas para aislar el efecto de la conectividad, pero no capturan propiedades como tasas de error, fidelidades, tiempos de coherencia o puertas nativas heterogéneas. Incorporar *backends* simulados más ricos o criterios de coste dependientes del *hardware* permitiría estudiar si los *layouts* y rutas que mejoran profundidad o coste equivalente también son preferibles cuando se consideran restricciones físicas más detalladas.



Guía de Despliegue

A.1 Requisitos de instalación

La reproducción operativa del proyecto se realiza desde la raíz del repositorio del proyecto. Este anexo no depende de rutas absolutas del equipo local: todas las rutas indicadas son relativas a dicho repositorio.

El código fuente puede descargarse desde el repositorio público del proyecto:

<https://github.com/edugbau/TFG-AI-Quantum-Transpiler>

Una vez clonado, las instrucciones siguientes deben ejecutarse desde la raíz del repositorio.

```
git clone https://github.com/edugbau/TFG-AI-Quantum-Transpiler.git
cd TFG-AI-Quantum-Transpiler
```

El entorno recomendado es Python 3.10 con un entorno virtual local en `.venv`. Si el entorno todavía no existe, debe crearse desde la raíz del repositorio antes de activarlo. Las dependencias del proyecto se instalan desde `requirements.txt`, donde se recogen las librerías necesarias para la interacción con Qiskit, la optimización multiobjetivo, el aprendizaje por refuerzo y la generación de informes. Antes de lanzar una campaña conviene comprobar que el entorno virtual está activado y que la instalación de dependencias ha finalizado sin errores.

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

A.2 Ejecución básica

La ejecución reproducible de las campañas se realiza mediante la CLI de `integration`. El archivo `experiment\experimental_campaigns.json` contiene las campañas utilizadas para repetir el experimento principal, por lo que puede emplearse directamente como entrada del modo por lotes.

Como primer paso, se recomienda validar el JSON sin ejecutar entrenamientos ni generar resultados finales. Para ello se usa la opción `--dry-run`, que comprueba que el archivo puede cargarse como una definición válida de campañas.

```
python -m src.integration.campaign_cli --input experiment\experimental_campaigns.json --dry-run
```

Una vez validada la configuración, la ejecución completa se lanza con el mismo archivo de entrada. La opción `--verbose` hace que la CLI muestre información de progreso y las rutas finales de salida, lo que facilita comprobar dónde se han escrito los artefactos.

```
python -m src.integration.campaign_cli --input experiment\experimental_campaigns.json --verbose
```

Si no se indica otra ubicación, la salida queda organizada en `campaigns/` e incluye los resúmenes y ficheros estructurados generados por `integration`. Estos artefactos son la base para revisar la ejecución, consultar incidencias y recuperar las métricas agregadas sin volver a lanzar el entrenamiento.

A.3 Uso de la CLI

La entrada principal para campañas es el módulo `src.integration.campaign_cli`. Su uso por lotes se controla mediante opciones de línea de comandos precedidas por `--`. La opción `--input` recibe la ruta relativa del JSON que define las campañas. En el protocolo de este trabajo, el valor esperado es `experiment\experimental_campaigns.json`.

La opción `--output-root` permite cambiar el directorio relativo donde se guardarán los resultados. Como la CLI ya utiliza `campaigns` por defecto, solo es necesario indicarla cuando se quiera separar una repetición en otra carpeta.

La opción `--dry-run` valida la entrada sin ejecutar las campañas. Es útil para detectar errores de formato, rutas incorrectas o configuraciones no aceptadas antes de iniciar procesos largos. La opción `--verbose` activa una salida más informativa durante la ejecución y muestra las rutas de los documentos de resumen y salidas estructuradas al finalizar.

La CLI también puede iniciarse sin `--input`; en ese caso entra en modo guiado e interactivo. Para repetir el experimento descrito en este trabajo, sin embargo, debe usarse el modo por lotes con `--input`, ya que el JSON de `experiment/` fija la configuración que se desea reproducir.

En las rutas que incluyen aprendizaje por refuerzo, la comparación final solo debe interpretarse como homogénea cuando el episodio de *routing* completa y `integration` puede reconstruir y evaluar el resultado. Cuando no se cumple esa condición, los resúmenes siguen siendo útiles para diagnosticar la ejecución, pero no deben tratarse como circuitos finales comparables.

Bibliografía

- [1] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. *Neural Combinatorial Optimization with Reinforcement Learning*. 2016. arXiv: 1611.09940 [cs.LG]. URL: <https://arxiv.org/abs/1611.09940>.
- [2] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. *Machine Learning for Combinatorial Optimization: a Methodological Tour d'Horizon*. 2020. arXiv: 1811.06128 [cs.LG]. URL: <https://arxiv.org/abs/1811.06128>.
- [3] Julian Blank and Kalyanmoy Deb. "pymoo: Multi-Objective Optimization in Python". In: *IEEE Access* 8 (2020), pp. 89497–89509. DOI: 10.1109/ACCESS.2020.2990567.
- [4] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. "A fast and elitist multi-objective genetic algorithm: NSGA-II". In: *IEEE Transactions on Evolutionary Computation* 6.2 (2002), pp. 182–197. DOI: 10.1109/4235.996017.
- [5] Donald E. Grierson. "Pareto multi-criteria decision making". In: *Advanced Engineering Informatics* 22.3 (2008), pp. 371–384. DOI: 10.1016/j.aei.2008.03.001.
- [6] Gymnasium Documentation. *Env*. Accessed 2026-05-22. 2026. URL: <https://gymnasium.farama.org/api/env/>.
- [7] IBM Quantum Documentation. *Construct circuits*. Accessed 2026-05-21. 2026. URL: <https://docs.quantum.ibm.com/guides/construct-circuits>.
- [8] IBM Quantum Documentation. *CouplingMap*. Accessed 2026-05-22. 2026. URL: <https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.transpiler.CouplingMap>.
- [9] IBM Quantum Documentation. *Fake provider*. Accessed 2026-05-22. 2026. URL: <https://quantum.cloud.ibm.com/docs/api/qiskit-ibm-runtime/fake-provider>.
- [10] IBM Quantum Documentation. *Hello world*. Accessed 2026-05-21. 2026. URL: <https://docs.quantum.ibm.com/guides/hello-world>.
- [11] IBM Quantum Documentation. *Introduction to transpilation*. Accessed 2026-05-21. 2026. URL: <https://docs.quantum.ibm.com/guides/transpile>.

- [12] IBM Quantum Documentation. *Layout*. Accessed 2026-05-25. 2026. URL: <https://qiskit.qotlabs.org/docs/api/qiskit/2.0/qiskit.transpiler.Layout>.
- [13] IBM Quantum Documentation. *Measure qubits*. Accessed 2026-05-21. 2026. URL: <https://docs.quantum.ibm.com/guides/measure-qubits>.
- [14] IBM Quantum Documentation. *Quantum circuit model*. Accessed 2026-05-21. 2026. URL: <https://docs.quantum.ibm.com/api/qiskit/circuit>.
- [15] IBM Quantum Documentation. *SWAP gate*. Accessed 2026-05-21. 2026. URL: <https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.SwapGate>.
- [16] IBM Quantum Documentation. *TranspileLayout*. Accessed 2026-05-25. 2026. URL: <https://qiskit.qotlabs.org/api/qiskit/qiskit.transpiler.TranspileLayout>.
- [17] IBM Quantum Documentation. *Transpiler stages*. Accessed 2026-05-21. 2026. URL: <https://docs.quantum.ibm.com/guides/transpiler-stages>.
- [18] Michael Kölle, Tom Schubert, Philipp Altmann, Maximilian Zorn, Jonas Stein, and Claudia Linnhoff-Popien. *A Reinforcement Learning Environment for Directed Quantum Circuit Synthesis*. 2024. arXiv: 2401.07054 [quant-ph]. URL: <https://arxiv.org/abs/2401.07054>.
- [19] David Kremer, Victor Villar, Hanhee Paik, Ivan Duran, Ismael Faro, and Juan Cruz-Benito. *Practical and efficient quantum circuit synthesis and transpiling with Reinforcement Learning*. 2024. arXiv: 2405.13196 [quant-ph]. URL: <https://arxiv.org/abs/2405.13196>.
- [20] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning". In: *nature* 521.7553 (2015), pp. 436–444.
- [21] Gushu Li, Yufei Ding, and Yuan Xie. *Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices*. 2018. arXiv: 1809.02573 [quant-ph]. URL: <https://arxiv.org/abs/1809.02573>.
- [22] Siyuan Niu, Adrien Suau, Gabriel Staffelbach, and Aida Todri-Sanial. *A Hardware-Aware Heuristic for the Qubit Mapping Problem in the NISQ Era*. 2020. arXiv: 2010.03397 [quant-ph]. URL: <https://arxiv.org/abs/2010.03397>.
- [23] Optuna Documentation. *optuna.study.Study*. Accessed 2026-05-22. 2026. URL: <https://optuna.readthedocs.io/en/stable/reference/generated/optuna.study.Study.html>.

- [24] Matteo G. Pozzi, Steven J. Herbert, Akash Sengupta, and Robert D. Mullins. *Using Reinforcement Learning to Perform Qubit Routing in Quantum Compilers*. 2020. arXiv: 2007.15957 [quant-ph]. URL: <https://arxiv.org/abs/2007.15957>.
- [25] John Preskill. "Quantum Computing in the NISQ Era and Beyond". In: *Quantum* 2 (2018), p. 79. DOI: 10.22331/q-2018-08-06-79. arXiv: 1801.00862. URL: <https://arxiv.org/abs/1801.00862>.
- [26] pymoo Documentation. *pymoo: Multi-objective Optimization in Python*. Accessed 2026-05-22. 2026. URL: <https://pymoo.org/>.
- [27] Stable-Baselines3 Documentation. *Stable-Baselines3 Docs: Reliable Reinforcement Learning Implementations*. Accessed 2026-05-22. 2026. URL: <https://stable-baselines3.readthedocs.io/>.
- [28] Stable-Baselines3 Documentation. *Tensorboard Integration*. Accessed 2026-05-22. 2026. URL: <https://stable-baselines3.readthedocs.io/en/v2.5.0/guide/tensorboard.html>.
- [29] TensorFlow Documentation. *TensorBoard*. Accessed 2026-05-22. 2026. URL: <https://www.tensorflow.org/tensorboard>.
- [30] Qingfu Zhang and Hui Li. "MOEA/D: A Multiobjective Evolutionary Algorithm Based on Decomposition". In: *IEEE Transactions on Evolutionary Computation* 11.6 (2007), pp. 712–731. DOI: 10.1109/TEVC.2007.892759.



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

ETS. de Ingeniería Informática

Bulevar Louis Pasteur, 35

Campus de Teatinos

29071 Málaga